

Caso 1 · ¿Ocurre el evento? ¿Y cuándo?

Respuestas binarias, politómicas y tiempo hasta el evento — Modelos Lineales Generalizados con R

A.M.Mayoral, con asistencia de Claude Opus 4.8

30/7/26

Tabla de contenidos

Resultados de aprendizaje del Caso 1	3
Presentación del caso	5
Dato real — GLOW	6
Cohorte simulada	8
1 · Cuando la recta no llega: del dato binario al marco GLM	13
1.1 El modelo lineal en GLOW	14
1.2 Por qué OLS no sirve para una respuesta binaria	19
1.3 Los tres componentes de un GLM	19
1.4 GLM en datos binarios	22
Puente a la Unidad 2	27
2 · Formulación de los GLM: estimación, inferencia, interpretación y evaluación	27
2.1 Familia exponencial y relación media–varianza	28
2.2 El enlace: canónico y alternativas	32
2.3 Interpretación: del coeficiente a la probabilidad	36
Interpretación en la escala de los odds y log-odds	37
Interpretación en la escala de la probabilidad: efectos marginales	41
2.4 Estimación: máxima verosimilitud e IWLS	45
2.5 Inferencia y selección de modelos	48
La Deviance	48
Inferencia sobre los coeficientes	50
Inferencia sobre bloques: el contraste de razón de verosimilitudes (LRT)	51
Comparar modelos no anidados: AIC y BIC	53
2.6 Bondad de ajuste y diagnóstico	60
El contraste global: ¿aportar o ajustar? (y por qué falla en binaria)	60

Residuos	61
Calibración: ¿son fiables las probabilidades?	62
Separación: cuando el ajuste “perfecto” rompe la inferencia.	65
2.7 Evaluación como clasificador	67
Del umbral a la matriz de confusión.	67
ROC y AUC	70
Puente a la Unidad 3	74
3 · Extensión de la respuesta binaria: binomial y politómica	74
3.1 Del dato individual al agrupado: respuesta binomial	75
3.2 Bondad de ajuste y diagnóstico en datos binomiales	78
Residuos	78
Bondad de ajuste	81
Calibración	83
Sobredispersión	84
3.3 Más de dos categorías sin orden: politómica nominal	86
3.4 Categorías ordenadas: el modelo de odds proporcionales	92
4 · Efectos Aleatorios y Modelos Mixtos	97
4.1 Qué son y para qué sirven	100
4.2 Intercepto aleatorio	101
Intercepto aleatorio en cohorte	101
1. Componentes de varianza	105
2. Condicional vs marginal	107
3. Encogimiento o <i>shrinkage</i> (<i>partial pooling</i>).	110
4.3 Pendiente aleatoria: cuando el efecto varía por grupo	113
4.4 Bondad de ajuste y diagnóstico del modelo mixto	117
4.5 Extensiones: estructuras aleatorias más ricas	123
4.6 El modelo mixto sobre datos reales: GLOW por centro	124
5 · Supervivencia: del hazard al GLM en tiempo discreto	128
5.1 La función de supervivencia y la función de riesgo en tiempo discreto	130
5.2 Precedentes en clave de supervivencia: Kaplan–Meier y Cox	131
Kaplan–Meier: dejar hablar a los datos	132
Riesgos proporcionales, en tiempo discreto	134
Cox: el mismo supuesto en tiempo continuo	138
5.3 La modelización GLM del problema de supervivencia	138
De los datos al modelo: qué es T y qué asumimos	139
El enlace complementary log-log y por qué es el natural	140
Ajuste en tiempo discreto e interpretación	142
Cloglog frente a logit: ¿cuánto importa el enlace?	145
Qué aporta el GLM en supervivencia	147
Conclusiones para la práctica clínica	148

6 · Estudio de caso, exposición y evaluación	151
6.1 El estudio de caso (entregable)	151
El encargo	151
Tu trabajo	152
El formato: un informe técnico	153
6.2 Checklist de tareas	154
¿Respondida cada pregunta del cliente?	154
Tareas analíticas del proyecto	155
Reproducibilidad	156

Resultados de aprendizaje del Caso 1

Al completar el Caso 1, el estudiante será capaz de:

- **RA1 · Marco GLM frente a OLS.** Reconocer por qué el modelo lineal (OLS) falla ante respuestas discretas y describir los tres componentes de un GLM: componente aleatorio, predictor lineal y función de enlace.
- **RA2 · Formulación y estimación.** Formular un GLM en la familia exponencial, ajustarlo por máxima verosimilitud (IWLS) y hacer inferencia (deviance, Wald, razón de verosimilitudes).
- **RA3 · Interpretación en la escala correcta.** Interpretar los efectos en la escala propia de cada respuesta: odds ratios y efectos marginales (binaria), odds ratios relativos (nominal), odds ratios acumulados (ordinal) y hazard ratios (supervivencia).
- **RA4 · Diagnóstico y bondad de ajuste.** Evaluar el modelo con residuos, separación, calibración, el contraste χ^2 de bondad de ajuste y la sobredispersión.
- **RA5 · Selección y comparación de modelos.** Comparar y seleccionar modelos con deviance, AIC/BIC y contrastes de razón de verosimilitudes.
- **RA6 · Efectos aleatorios.** Ajustar e interpretar modelos mixtos (GLMM) con intercepto y pendiente aleatorios: componentes de varianza, ICC, *shrinkage* y la distinción condicional vs marginal.
- **RA7 · Tiempo-a-evento y censura.** Reformular la supervivencia en tiempo discreto como un GLM binomial con enlace *cloglog* sobre datos persona-periodo, tratando la censura y prediciendo hazard y supervivencia.
- **RA8 · Estudio de caso reproducible.** Desarrollar, presentar y defender un estudio de caso reproducible que integre las lentes anteriores.

i Mapa del caso

En esta unidad vamos a trabajar sobre respuestas categóricas. Iniciamos con respuesta binaria, seguimos con su agrupación natural en respuestas binomiales, y extendemos a respuesta poltómica y multinomial. Por el camino incorporamos los efectos aleatorios

en la modelización, y por último estudiamos también el abordaje como un problema de supervivencia para ir más allá de la predicción del evento, incorporando datos censurados y prediciendo también cuándo se va a producir el evento.

Trabajaremos con la base de datos GLOW y usaremos la cohorte simulada para el estudio de supervivencia y el estudio de caso sobre el que trabajarás tú estudiante, para demostrar que has aprendido a modelizar datos discretos.

Unidad	Contenido	Objetivo y datos
1.1	«¿Ocurre el evento?» — Cuando la recta no llega: OLS falla en binaria; tres componentes del GLM; tipología de respuestas; primer glm.	RA1 (y primer contacto con RA2) Datos: glow
1.2	«¿Cómo se ajusta e interpreta una logística, y cómo de bien clasifica?» — Formulación de los GLM: familia exponencial, enlace, IWLS, <i>deviance</i> ; inferencia; interpretación (OR, efectos marginales); evaluación y comparación de clasificadores.	RA2, RA3, RA4, RA5 Datos: glow
1.3	«¿Y si la respuesta tiene más de dos categorías?» — Respuesta categórica generalizada: binomial agrupado (ajuste y bondad de ajuste con ²) y respuestas politómicas — multinomial nominal y ordinal (odds proporcionales).	RA2–RA5 (nuevos tipos de respuesta) Datos: glow (agrupado y politómico)
1.4	«¿Y si los sujetos vienen agrupados?» — Efectos aleatorios (GLMM logístico, intercepto y pendiente aleatorios) sobre la cohorte y validación con datos reales (GLOW); puente a la supervivencia.	RA6 (y RA4 en el diagnóstico del mixto) Datos: cohorte, glow
1.5	«¿Y cuándo ocurre el evento?» — Supervivencia: función de supervivencia, hazard, riesgos proporcionales (Cox como referencia, detalle en el Caso 3); el tiempo discreto como GLM (cloglog persona-periodo).	RA7 (y RA3 vía hazard ratios) Datos: cohorte (persona-periodo)
1.6	«¿Sabes ya modelizar datos discretos?» — Estudio de caso, exposiciones y evaluación (actividades, instrucciones, rúbricas).	RA8 (integra RA1–RA7) Datos: banco del estudiante

Caja de herramientas de R por unidad. Las funciones y paquetes que vertebran cada sesión (uso comentado en cada `.qmd` y en los scripts de `caso1/scripts/`):

UnidadFunciones y paquetes clave

- 1.1 `glm(family = binomial) · predict(type = "response") · performance::binned_residuals`
 - 1.2 `glm(binomial) · broom::tidy · pROC::roc · performance::performance_hosmer · DHARMA::simulateResiduals · emmeans / marginaeffects`
 - 1.3 `glm(binomial, weights = ) (agrupado) · nnet::multinom (nominal) · MASS::polr (ordinal)`
 - 1.4 `lme4::glmer · performance::icc / check_singularity · broom.mixed::tidy`
 - 1.5 `glm(binomial("cloglog")) (persona-periodo) · survival::Surv / survfit · coxph (referencia)`
 - 1.6 `Quarto (.qmd) · renv · Git — reproducibilidad y comunicación`
-

Presentación del caso

Iniciamos pues la unidad, presentando los bancos de datos con los que vamos a trabajar. La pregunta que vertebra el caso tiene dos tiempos básicos:

1. «**¿Ocurre el evento?**»: una respuesta binaria que motiva el marco GLM y la regresión logística.
2. «**¿Cuándo?**»: el mismo evento como **tiempo hasta que ocurre**, con **censura**, que se reformula como un GLM sobre datos persona-periodo.

En el recorrido, surgen dos matices que también trataremos:

- los sujetos vienen **agrupados** (por centro), e introducimos los **efectos aleatorios** y los **modelos mixtos**,
- y la respuesta puede tener **más de dos categorías**, motivando la extensión politómica, nominal u ordinal.

Para trabajar todas estas cuestiones utilizaremos dos fuentes de datos con papeles distintos: un conjunto **real** para el ejemplo logístico, y una **cohorte simulada** con proceso generador conocido, que vertebra la extensión de la modelización a la supervivencia.

Dato real — GLOW

glow500 procede del estudio **GLOW** (*Global Longitudinal Study of Osteoporosis in Women*) y se distribuye en el paquete **aplore3** (licencia GPL-3), compañero del manual de Hosmer et al. (2013). Es una muestra de **500 mujeres** seguidas un año en **6 centros**. La respuesta es binaria —**fracture**: alguna fractura en el primer año— y el resto son factores de riesgo basales:

Variable	Descripción	Tipo
fracture	Fractura en el primer año (No/Yes) — respuesta	Binaria
age	Edad al ingreso (años)	Continua
weight	Peso al ingreso (kg)	Continua
height	Estatura al ingreso (cm)	Continua
bmi	Índice de masa corporal (kg/m ²)	Continua
priorfrac	Antecedente de fractura previa (No/Yes)	Binaria
preno	Menopausia antes de los 45 (No/Yes)	Binaria
momfrac	Fractura de cadera en la madre (No/Yes)	Binaria
armassist	Necesita los brazos para levantarse de una silla (No/Yes)	Binaria
smoke	Fumadora actual o pasada (No/Yes)	Binaria
raterisk	Riesgo de fractura autopercebido (Less/Same/Greater)	Ordinal (3)
fracscore	Puntuación compuesta de riesgo de fractura	Discreta
sub_id, site_id, phy_id	Identificadores de paciente, centro y médico	ID

Creamos también una versión 0/1 de la respuesta **fracture** para el experimento, que utilizaremos en esta unidad.

```
data(glow500)
glow <- glow500 |>
  as_tibble() |>
  mutate(fractura01 = as.integer(fracture) - 1L) # No -> 0, Yes -> 1

glow |> count(fracture) |> mutate(prop = round(n / sum(n), 3))
```

```
# A tibble: 2 x 3
  fracture      n prop
  <fct>    <int> <dbl>
1 No         375  0.75
2 Yes        125  0.25
```

Visualizamos en la Figura 1 los datos y las relaciones entre variables en GLOW.

```
# Covariables sustantivas de GLOW (dejo fuera los identificadores)
vars <- c("age", "weight", "height", "bmi", "fracscore",      # continuas / discretas
          "priorfrac", "premeno", "momfrac",                # factores de riesgo binarios
          "armassist", "smoke", "raterisk")                  # binarios + ordinal

glow |>
  select(fracture, all_of(vars)) |>
  ggpairs(
    columns = vars,
    mapping = aes(color = fracture, alpha = 0.6),
    legend=1,
    upper = list(continuous = wrap("cor", size = 2.3),      # correlaciones (global y por g
                      combo   = wrap("box_no_facet"),      # continua ~ factor: boxplots
                      discrete = wrap("count")),           # factor ~ factor: recuento
    lower = list(continuous = wrap("points", alpha = 0.25, size = 0.4),
                  combo     = wrap("facethist", bins = 20),
                  discrete  = wrap("facetbar")),
    diag  = list(continuous = wrap("densityDiag", alpha = 0.5),
                  discrete  = wrap("barDiag")),
    progress = FALSE
  ) +
  theme_bw(base_size = 7) +
  theme(strip.text = element_text(size = 6))+
  theme(legend.position = "bottom")
```



Figura 1: Descripción de GLOW. En la diagonal distribución de las variables, diferenciadas en color por evento. En los cruces la relación entre ellas.

Cohorte simulada

Está inspirada en el problema de GLOW y simula el seguimiento de las pacientes en el tiempo a lo largo de 6 años, para una cohorte de mujeres, repartidas en varias clínicas (24).

El simulador `simular_cohorte()` está disponible en `R/dgp_cohorte.R` y parte de unas condiciones establecidas sobre las relaciones entre las variables consideradas (recuperables en `str(attr(cohorte, "verdad"))`).

Cada paciente acude a **revisiones periódicas anuales** —seis como máximo— y en cada una se constata si **ha sufrido una fractura desde la revisión anterior**. No observamos el **instante exacto** de la fractura, solo el **periodo** en que se detecta: por eso **el tiempo es discreto**. Así, `tiempo` $\in \{1, \dots, 6\}$ representa el periodo en que **termina la observación** de cada mujer —bien porque se le detecta una fractura (`evento` = 1) o porque el seguimiento acaba sin ella (`evento` = 0, **censura**)—. `tiempo` = 1 significa que el desenlace ocurre en la **primera** revisión; `tiempo` = 6, que llega a la **sexta y última**. La expansión **persona-periodo** convierte a cada mujer en una fila por cada revisión a la que llega sin fractura, con `y` = 1 solo en aquella donde se detecta.

El simulador utiliza dos predictores `x1` y `x2` abstractos, que podrían representar distintas características; a lo largo del curso **adoptamos una caracterización fija** para facilitar la interpretación:

- `x1` = **fragilidad ósea** (a partir de la densidad mineral, estandarizada en puntuación `z`): factor de riesgo **continuo**; a mayor valor, hueso más frágil y **más** riesgo de fractura ($\beta_1 > 0$).
- `x2` = **tratamiento** preventivo (1 = lo recibe, 0 = no): factor **binario** y **protector**; reduce el riesgo ($\beta_2 < 0$).
- `centro` = la **clínica** (24 en total); su intercepto aleatorio recoge que unas clínicas tienen más riesgo basal que otras —la *fragilidad* estadística / efecto mixto—.

Las variables simuladas son:

Variable	Significado (lectura adoptada)	Tipo	Se usa en
<code>id</code>	Identificador de la paciente	—	—
<code>centro</code>	Clínica (grupo del efecto aleatorio / fragilidad)	Factor (24 niveles)	1.4, 1.5
<code>x1</code>	Fragilidad ósea (baja densidad mineral, estandarizada)	Continua	1.4, 1.5
<code>x2</code>	Tratamiento preventivo (1 = sí, 0 = no)	Binaria	1.4, 1.5
<code>tiempo</code>	Periodo de revisión (1–6) en que termina la observación	Entero	1.5
<code>evento</code>	1 = fractura observada en ese periodo; 0 = censura	Binaria	1.5
<code>ever</code>	¿Hubo fractura durante el seguimiento? (= <code>evento</code>)	Binaria	1.4
<code>sever_ord</code>	Severidad basal ordinal de la osteoporosis (Leve < Moderado < Grave)	Factor ordenado (3)	1.4

Variable	Significado (lectura adoptada)	Tipo	Se usa en
<code>clase_nom</code>	Endotipo nominal de fragilidad ósea A/B/C (sin orden) — ilustra el contraste; el multinomial del curso se ajusta sobre GLOW (<code>raterisk</code>)	Factor (3)	—

Las respuestas se generan a partir de un modelo de **riesgo en tiempo discreto** con enlace `cloglog` (lo estudiaremos más adelante):

$$\text{cloglog}(h_{ijt}) = \alpha_t + \beta_1 x_{1i} + \beta_2 x_{2i} + u_j, \quad u_j \sim \mathcal{N}(0, \sigma_u^2),$$

con **intercepto aleatorio por clínica** u_j (fragilidad) y **censura administrativa no informativa**. De ahí salen la respuesta binaria «¿hubo fractura?» y el reloj de supervivencia (en qué periodo se produjo). Construimos también una variable de **severidad ordinal** (`sever_ord`), que se genera aparte, con un cumulative logit más intercepto aleatorio por clínica, disponible como ordinal con verdad conocida para ejercicios. Las respuestas categóricas son **independientes** del reloj de supervivencia —comparten covariables y clínicas, pero no se derivan de si hubo fractura, ni de cuándo la hubo—, una decisión de diseño para no entrar en riesgos competitivos (fuera del alcance de esta unidad).

Las dos respuestas categóricas, en clave clínica.

Aunque comparten las covariables `x1` y `x2` con el reloj de supervivencia, `sever_ord` y `clase_nom` cuentan historias clínicas distintas y se generan con estructuras distintas; conviene fijar su lectura para interpretarlas con soltura.

- `sever_ord` es una **gradación de severidad**: un único eje latente ordena los tres niveles —el supuesto de *odds proporcionales*— y las clínicas difieren en su reparto basal (intercepto aleatorio por centro). Admite dos lecturas equivalentes: como **estadio densitométrico de la osteoporosis** (Leve < Moderado < Grave osteopenia < osteoporosis < osteoporosis establecida) o como el **grado semicuantitativo de fractura vertebral de Genant** (grados 1 < 2 < 3), que la literatura etiqueta literalmente leve/moderado/grave. Nos quedamos con la primera: “estadio densitométrico de la osteoporosis”. Las hipótesis del simulador, **en tendencia**: a **mayor fragilidad ósea** (`x1`) la severidad tiende a **desplazarse hacia grados más altos**; el **tratamiento** (`x2`) se asocia a grados **más leves** (lectura protectora); y, a igualdad de paciente, **unas clínicas tienden a registrar severidades mayores que otras** (distinto *case-mix* o criterio del lector). La relación con el tratamiento es **asociativa**, no causal.
- `clase_nom` es, en cambio, un **fenotipo cualitativo sin orden**: cada categoría sigue su propio patrón —efectos específicos de categoría— y la clínica **no** interviene (sin efecto aleatorio), de modo que es un rasgo del **paciente**. La leemos como un **endotipo de fragilidad ósea** con referencia **A** (baja densidad sin etiología secundaria), un fenotipo

B de caídas / fragilidad musculoesquelética y un fenotipo **C** de osteoporosis secundaria (corticoidea o endocrina). En tendencia: a **mayor fragilidad ósea** (x1), más probable **alejarse de la referencia** hacia B y, sobre todo, hacia C; y el **tratamiento** (x2) **separa** ambos fenotipos —tiende a asociarse **menos** con el de caídas (que se maneja con prevención de caídas) y **más** con la osteoporosis secundaria (donde la profilaxis ósea casi se protocoliza)—. Como antes, se trata de una asociación de **indicación/selección**, no de causalidad.

Simulación de los datos Aplicamos el simulador generado y construimos la **cohorte** completa.

```
library(readr)
source("R/dgp_cohorte.R")
cohorte <- cargar_cohorte()
# y guardamos las simulaciones
#write_csv(cohorte, "cohorte.csv")
# resumen
glimpse(cohorte)
```

```
Rows: 877
Columns: 9
$ id      <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 1~
$ centro  <fct> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
$ x1      <dbl> -1.67363040, 1.09293799, -0.35266071, 0.29762674, -0.3236454~
$ x2      <int> 0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, ~
$ tiempo  <int> 4, 3, 6, 4, 6, 4, 6, 3, 4, 5, 6, 5, 4, 1, 6, 4, 4, 5, 3, 6, ~
$ evento  <int> 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, ~
$ ever    <int> 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, ~
$ clase_nom <fct> A, B, A, C, A, A, C, A, A, B, A, A, B, B, A, C, C, A, C, B, ~
$ sever_ord <ord> Leve, Grave, Moderado, Moderado, Leve, Leve, Leve, Grave, Mo~
```

```
head(cohorte)
```

	id	centro	x1	x2	tiempo	evento	ever	clase_nom	sever_ord
1	1	1	-1.6736304	0	4	1	1	A	Leve
2	2	1	1.0929380	1	3	1	1	B	Grave
3	3	1	-0.3526607	1	6	0	0	A	Moderado
4	4	1	0.2976267	0	4	0	0	C	Moderado
5	5	1	-0.3236454	1	6	0	0	A	Leve
6	6	1	-1.6879930	1	4	0	0	A	Leve

Más tarde, para trabajar la supervivencia (Unidad 1.5), transformaremos esta `cohort` a un formato **persona-periodo** —una fila por paciente y periodo en riesgo—; esa expansión se construye y se explica allí, donde se usa.

Visualizamos en la Figura 2 los datos simulados en la `cohort`.

```
# Preparo factores legibles (x2, tiempo y el desenlace como categóricos)
cohort_eda <- cohort |>
  mutate(
    evento = factor(evento, levels = c(0, 1), labels = c("Sin fractura", "Fractura")),
    x2      = factor(x2,      levels = c(0, 1), labels = c("No", "Sí")),
    tiempo = factor(tiempo, levels = 1:6)
  )

# Variables sustantivas (fuera 'id', 'centro' y 'ever' = duplicado de 'evento')
vars <- c("x1", "x2", "tiempo", "clase_nom", "sever_ord")

cohort_eda |>
  select(evento, all_of(vars)) |>
  ggpairs(
    columns = vars,
    mapping = aes(color = evento, alpha = 0.6),
    legend=1,
    upper = list(continuous = wrap("cor", size = 2.5),
                  combo      = wrap("box_no_facet"),
                  discrete   = wrap("count")),
    lower = list(continuous = wrap("points", alpha = 0.25, size = 0.4),
                  combo      = wrap("facethist", bins = 20),
                  discrete   = wrap("facetbar")),
    diag  = list(continuous = wrap("densityDiag", alpha = 0.5),
                  discrete   = wrap("barDiag")),
    progress = FALSE
  ) +
  theme_bw(base_size = 8)+
  theme(legend.position = "bottom")
```

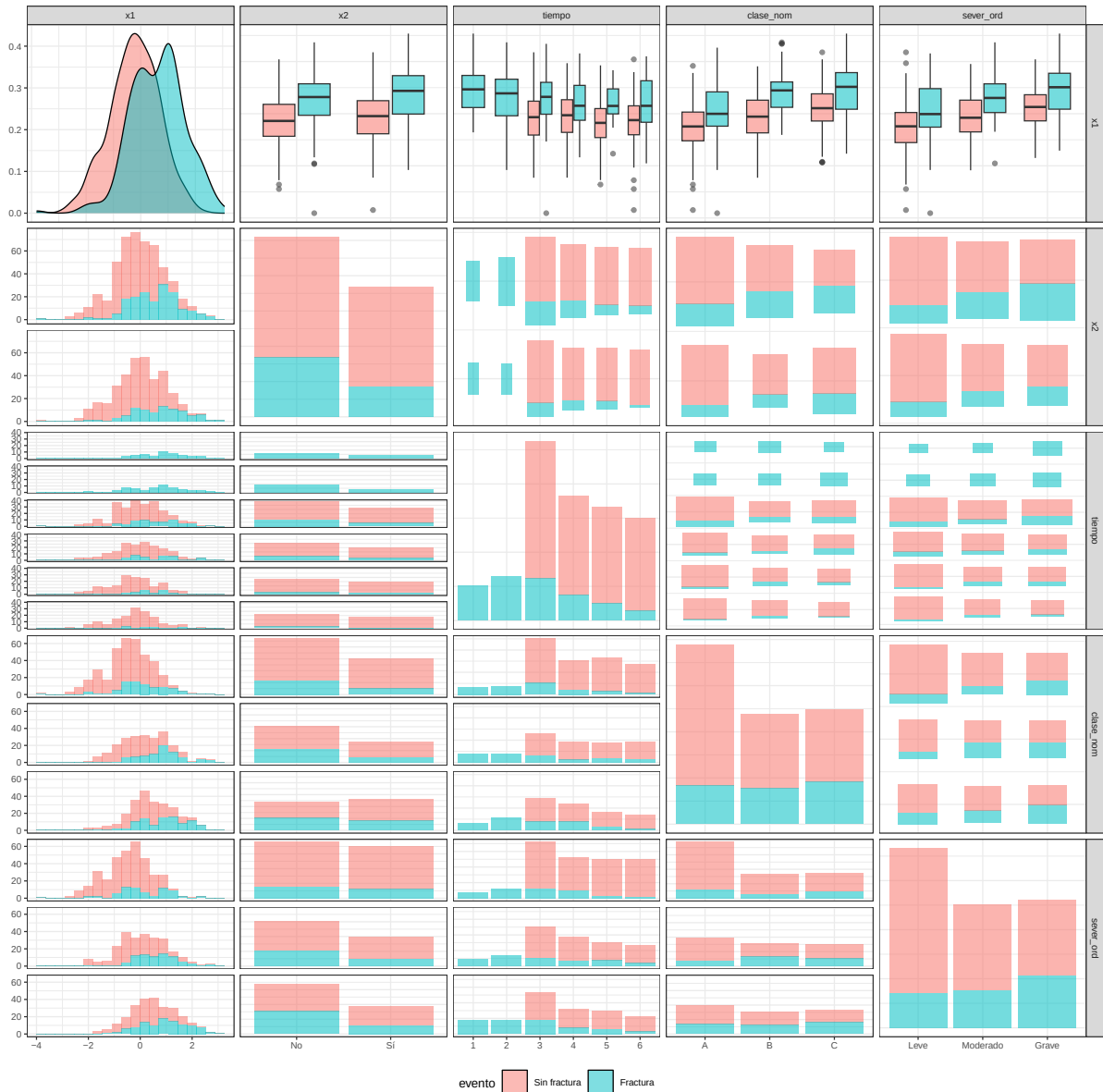


Figura 2: Descripción de cohorte. En la diagonal distribución de las variables, diferenciadas en color por evento. En los cruces la relación entre ellas.

1 · Cuando la recta no llega: del dato binario al marco GLM

Iniciamos nuestro curso descubriendo las limitaciones del modelo lineal cuando la respuesta ni es normal, ni hay modo de transformarla para hacerla lineal, como ocurre cuando tenemos una variable dicotómica por predecir. Nos apoyamos en la base de datos GLOW para ilustrar

dónde falla el modelo lineal, y mostrar las bondades que reporta un modelo lineal generalizado, en el que se tiene en cuenta la distribución adecuada para la respuesta, y se busca una transformación del predictor lineal que pueda generar predicciones viables sin salir de un modelo lineal. Comparamos en esta sección, los resultados de un modelo lineal y los de un modelo lineal generalizado, para fundamentar el desarrollo de este modelo en la siguiente sección.

i Sobre esta unidad

RA implicados:

- **RA1 · Marco GLM frente a OLS.** Reconocer por qué el modelo lineal (OLS) falla ante respuestas discretas y describir los tres componentes de un GLM (aleatorio, predictor lineal y enlace).
- **RA2 · Formulación y estimación** (primer contacto). Formular y estimar un GLM como generalización del modelo lineal.

1.1 El modelo lineal en GLOW

La primera tarea antes de proponer una modelización, pasa por observar el comportamiento de los datos. Nos interesan, de momento, las variables **fracture**, que actúa como respuesta e indica si la paciente ha sufrido una fractura, **age**, con la edad de la paciente, y **priorfrac**, que indica si tiene antecedentes de haber sufrido alguna fractura previa. Indagamos pues, cómo se relaciona la respuesta con la edad, distinguiendo por antecedente de fractura previa (en la Figura 3):

```
glow |>
  ggplot(aes(x = fracture, y = age, fill = fracture)) +
  geom_boxplot(alpha = 0.75, outlier.alpha = 0.3) +
  facet_wrap(~ priorfrac,
             labeller = labeller(priorfrac = c(No = "Sin fractura previa",
                                                Yes = "Con fractura previa")))) +
  labs(x = "Fractura en el primer año", y = "Edad (años)") +
  guides(fill = "none")
```

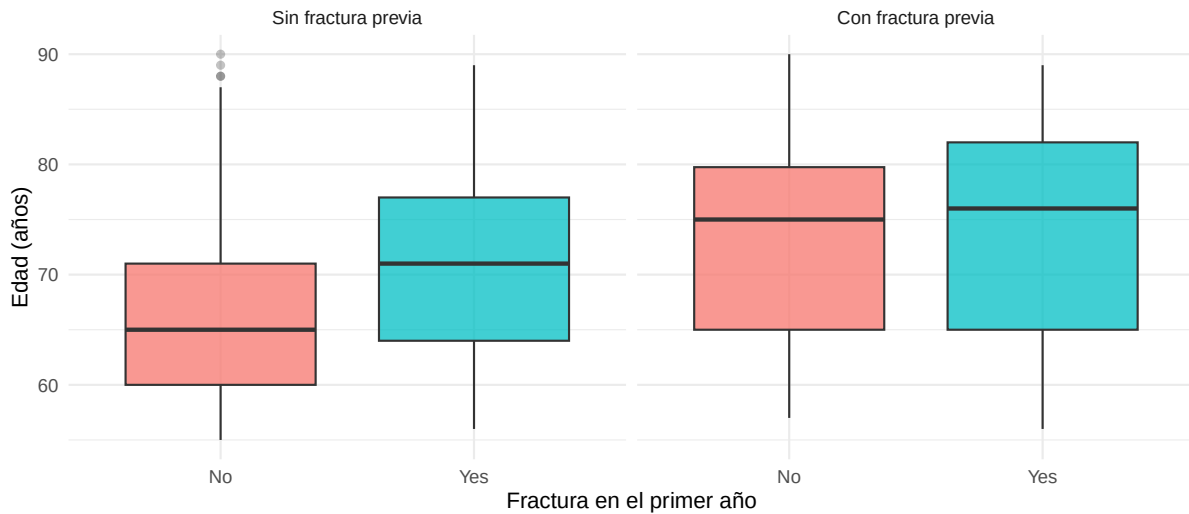


Figura 3: Distribución de la edad según haya habido o no fractura en el primer año, condicionada por el antecedente de fractura previa (priorfrac).

Observamos que las mujeres que **sí** fracturan son, en mediana, **algo mayores** que las que no, en los dos paneles: la edad es un **factor de riesgo**. Pero la separación es **modesta** y las cajas se **solapan** bastante —la edad por sí sola no discrimina bien, de ahí que el efecto lineal de la edad resultará débil, como veremos más tarde—. Además, el **antecedente de fractura previa** (panel derecho) eleva el riesgo por sí mismo —hay fracturas a cualquier edad—, así que conviene un modelo con **ambas** covariables, no una recta única sobre la edad.

Tiene sentido pues, plantear un modelo predictivo para **fracture** en función de **age** y **priorfrac**. Para intentar capturar el efecto de la edad (**age**) sobre la probabilidad de fractura (y valorar), agrupamos la edad por franjas y calculamos la proporción de fracturas en cada franja, diferenciando por el hecho de que la paciente hubiera sufrido o no, una fractura previa (**priorfrac**). En la Figura 4 tenemos esta representación, y superponemos sendas rectas, que transmiten una considerable variabilidad de los puntos en torno a ellas, especialmente en las pacientes que han sufrido fractura.

```
glow |>
  mutate(edad_grupo = cut(age, breaks = seq(55, 95, by = 5))) |>
  filter(!is.na(edad_grupo)) |>
  group_by(edad_grupo, priorfrac) |>
  summarise(age = mean(age), p_fractura = mean(fractura01), n = n(), .groups = "drop") |>
  ggplot(aes(age, p_fractura, color = priorfrac)) +
  #geom_line() +
  geom_point(aes(size = n)) +
  geom_smooth(aes(weight = n), method = "lm", se = FALSE, linewidth = 1) +
```

```
labs(x = "Edad (media del tramo)", y = "Proporción de fractura",
     color = "Fractura previa", size = "n")
```

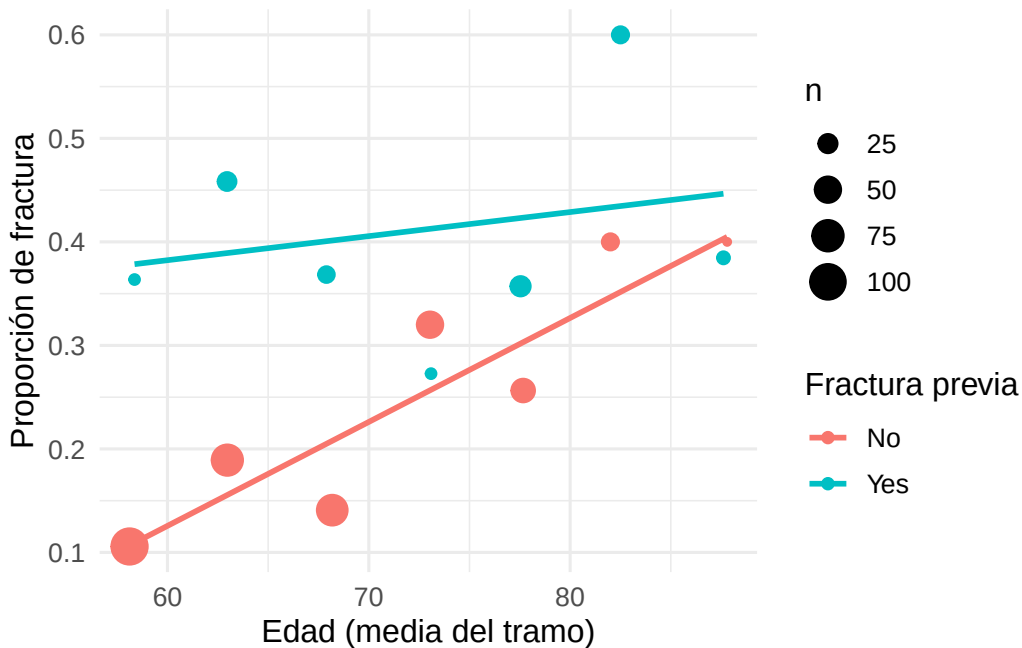


Figura 4: Proporción observada de fractura por tramos de edad y antecedente previo, con sendas rectas de regresión por grupo: el ajuste lineal es pobre y apenas captura tendencia.

Planteamos pues, una primera aproximación a través de un modelo lineal (sin interacción) en el que la respuesta ha de ser numérica, por lo que consideramos la recodificación de **fracture** con valores 0/1 -**fractura01**-, y los predictores son **age** y **priorfrac**.

```
fit_ols = lm(fractura01 ~ age + priorfrac, data = glow)
```

Y comenzamos a identificar los problemas que van apareciendo.

En primer lugar, un modelo lineal no garantiza una predicción de la respuesta dentro de un rango, y en particular como en este caso, dentro del rango $(0, 1)$ de probabilidades. Aunque para el tramo observado de edad el modelo lineal se comporta bien a este respecto, dando predicciones dentro del rango $(0, 1)$, vemos en la Figura 5 cómo la recta de ajuste da predicciones por debajo de 0 para edades inferiores a 40 años.


```

fit_ols <- lm(fractura01 ~ age + priorfrac, data = glow)
rango <- range(glow$age)

grid_ext <- expand_grid(
  age      = seq(35, 110, length.out = 200),
  priorfrac = factor(c("No", "Yes"), levels = levels(glow$priorfrac)))
grid_ext$p_hat <- predict(fit_ols, newdata = grid_ext)

ggplot(grid_ext, aes(age, p_hat, color = priorfrac)) +
  annotate("rect", xmin = rango[1], xmax = rango[2], ymin = -Inf, ymax = Inf,
    alpha = 0.12, fill = "grey40") +
  geom_line(linewidth = 1) +
  geom_hline(yintercept = c(0, 1), linetype = "dashed") +
  labs(x = "Edad (rango extrapolado; sombreado = rango observado)",
    y = "Probabilidad predicha (OLS)", color = "Fractura previa")

```

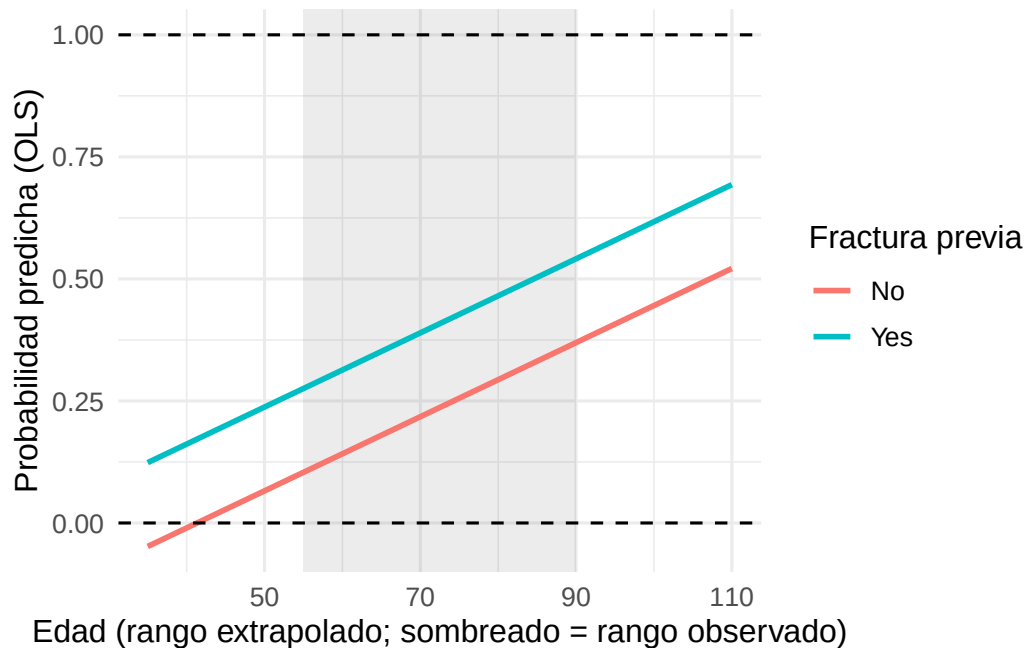


Figura 5: Predicción del modelo lineal extrapolada fuera del rango de edad observado (franja sombreada). Más allá de los datos, OLS predice probabilidades por debajo de 0: imposibles para una probabilidad.

Otro fallo se da en el diagnóstico de los residuos, como vemos en la Figura 6: a la izquierda apreciamos que los residuos, para un x dado, toman solo dos valores posibles, y se mueven con la media ajustada; se rompe la independencia de los residuos y la homocedasticidad. En el

gráfico a la derecha se muestran los residuos versus los percentiles de una distribución normal, y de nuevo se aprecia la ruptura de la hipótesis de normalidad. Esto ya era previsible desde el inicio, puesto que la respuesta es Bernoulli y no Normal.

```
fit_ols <- lm(fractura01 ~ age + priorfrac, data = glow)

g_res <- tibble(ajustado = fitted(fit_ols),
               residuo = glow$fractura01 - fitted(fit_ols),
               y = factor(glow$fractura01, labels = c("No fractura (y=0)", "Fractura (y=1)"))
ggplot(aes(ajustado, residuo, color = y)) +
  geom_point(alpha = 0.3) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  labs(x = "Valores ajustados", y = "Residuo de respuesta (y - ^)",
       title = "Residuos vs ajustados (OLS)", color = NULL) +
  theme(legend.position="bottom")

g_qplot <- tibble(residuo = residuals(fit_ols)) |>
  ggplot(aes(sample = residuo)) +
  stat_qq(alpha = 0.4, size = 0.6) +
  stat_qq_line(color = "firebrick") +
  labs(x = "Cuantiles teóricos (Normal)", y = "Cuantiles de los residuos de OLS")

patchwork::wrap_plots(g_res, g_qplot, ncol = 2)
```

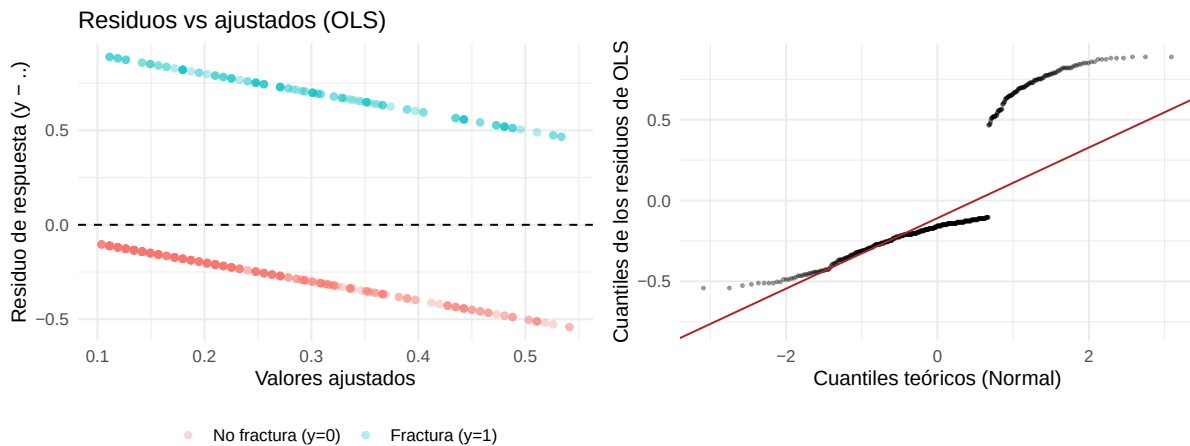


Figura 6: Diagnóstico del ajuste OLS sobre respuesta binaria. Izquierda: residuos frente a ajustados (no hay nube de puntos aleatoria en torno al cero). Derecha: distribución de los residuos frente a la Normal de igual media y desviación (fallo de normalidad).

1.2 Por qué OLS no sirve para una respuesta binaria

El modelo OLS postula $Y = \beta_0 + \beta_1 x_1 + \dots + \varepsilon$ con $\varepsilon \sim \mathcal{N}(0, \sigma^2)$. Como $Y \in \{0, 1\}$, su esperanza condicional es directamente la probabilidad de éxito, $\mathbb{E}[Y | x] = \Pr(Y = 1 | x) = \pi(x)$: OLS estaría modelando una **probabilidad como función lineal**. Cuatro problemas surgen todos ellos críticos (Dobson & Barnett, 2018, caps. 1–3; Fox, 2016, 14):

1. **Predicciones fuera de rango.** El predictor lineal recorre toda la recta real, pero una probabilidad vive en $[0, 1]$. Para valores extremos de las covariables, OLS predice probabilidades **negativas o mayores que 1**.
2. **Varianza no constante.** Si Y es Bernoulli con media $\pi(x)$, entonces $\text{Var}(Y | x) = \pi(x)[1 - \pi(x)]$: la varianza **depende de la media**. El supuesto de homocedasticidad se viola por construcción.
3. **Error no normal.** Fijado un x concreto, $\pi(x)$ es una **constante** e $Y \in \{0, 1\}$, así que $\varepsilon = Y - \pi(x)$ solo puede tomar **dos valores**: $-\pi(x)$ con probabilidad $1 - \pi(x)$, o $1 - \pi(x)$ con probabilidad $\pi(x)$. La distribución **condicional** del error (dado x) es de **dos puntos**, no una campana gaussiana. Al variar x esos dos valores se desplazan —de ahí las dos bandas de residuos del laboratorio—, pero el supuesto de normalidad de OLS recae sobre la distribución condicional, que es la que aquí es falsa.
4. **Linealidad implausible.** OLS impone que el efecto de una covariable sobre la probabilidad sea **constante**: la misma pendiente β tanto si π ronda 0,5 como si ya está pegada a 0 o a 1. Eso no es creíble, porque en los extremos apenas queda margen para moverse: si una paciente tiene una probabilidad de fractura casi nula (o casi segura), un año más de edad casi no puede cambiarla; en cambio, en la zona central ese mismo año podría desplazar bastante la probabilidad. Lo razonable es entonces una relación **en forma de S**: pronunciada en el centro y **aplanada (saturada) en los extremos**. La inversa del logit dibuja exactamente esa curva y, de paso, mantiene π dentro de $(0, 1)$.

1.3 Los tres componentes de un GLM

La alternativa que plantearon Nelder & Wedderburn (1972) unificó la regresión lineal, logística, de Poisson, etc., bajo una estructura de **tres piezas**:

1. **Componente aleatoria.** Y sigue una distribución de la **familia exponencial**.

Para GLOW, $Y | x \sim \text{Bernoulli}(\pi(x))$. Esta pieza fija la relación media-varianza y absorbe los problemas 2 y 3.

2. **Componente sistemática.** Mantiene un **predictor lineal** $\eta = \mathbf{x}^\top \beta$, lineal en los parámetros.
3. Introduce una **función de enlace** o *link* g , monótona y diferenciable, que liga la media con el predictor lineal, $g(\mu) = \eta$ y $\mu = g^{-1}(\eta)$.

Para binaria, el enlace por defecto es el **logit**:

$$g(\pi) = \log \frac{\pi}{1 - \pi}, \quad \pi = g^{-1}(\eta) = \frac{e^\eta}{1 + e^\eta},$$

que resuelve los problemas 1 y 4: el logit lleva $(0, 1)$ a toda la recta real (nunca fuera de rango).

En la Figura 7 se muestra cómo la función logit transforma una probabilidad cualquiera en el rango $[0, 1]$ en un valor sobre toda la recta real. Su inversa, la sigmoide, a la izquierda, permite recuperar un valor real en un rango de probabilidades, y se utilizará para estimar la probabilidad a partir del predictor lineal.

```
library(patchwork)
g1 <- tibble(p = seq(0.001, 0.999, by = 0.001)) |>
  mutate(y = qlogis(p)) |>
  ggplot(aes(p, y)) +
  geom_line(linewidth = 1, color = "steelblue") +
  geom_hline(yintercept = 0, linetype = "dashed") +
  labs(title = "logit:\n estira (0,1) sobre la recta real",
        x = expression(pi), y = expression(log(pi/(1 - pi))))

g2 <- tibble(eta = seq(-6, 6, by = 0.05)) |>
  mutate(p = plogis(eta)) |>
  ggplot(aes(eta, p)) +
  geom_line(linewidth = 1, color = "firebrick") +
  geom_hline(yintercept = c(0, 1), linetype = "dotted") +
  labs(title = "sigmoide:\n devuelve el predictor a (0,1)",
        x = expression(eta), y = expression(e^eta/(1 + e^eta)))

g1 + g2
```

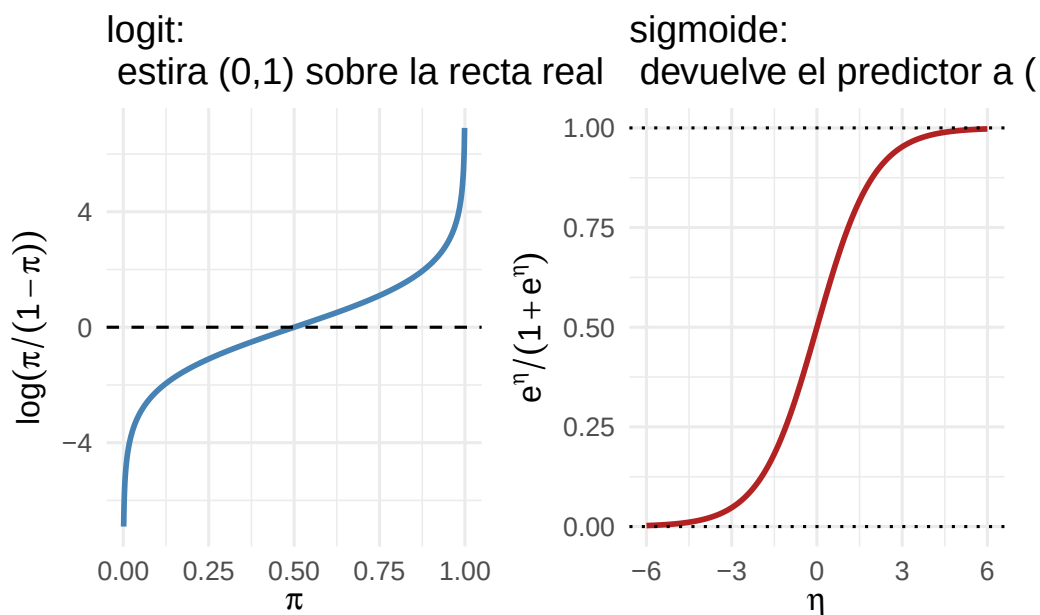


Figura 7: Función logit y sigmoide: las dos direcciones. El logit lleva (0,1) a toda la recta real; su inversa (la sigmoide) devuelve cualquier η a (0,1).

! OLS es un GLM

La regresión lineal es un modelo lineal generalizado con la familia Gaussiana y la función de enlace identidad. No abandonamos OLS: lo integramos en los GLM. Por eso `lm(y ~ x)` y `glm(y ~ x)` se parecen tanto.

En esta unidad trabajaremos con:

Tipo de respuesta	Ejemplo	Familia (por defecto)	Enlace canónico	Unidad
Binaria	fractura sí/no	Binomial	logit	Unidades 1 y 2
Binomial	nº fracturas por edad	Binomial	logit	Unidad 3
Politómica nominal	perfil A/B/C	Multinomial	logit (gen.)	Unidad 3
Politómica ordinal	severidad Leve/Mod/Grave	Multinomial ordinal	logit acumulado	Unidad 3

1.4 GLM en datos binarios

Para modelizar con un modelo lineal generalizado la predicción de **fracture**, mantenemos el carácter dicotómico de la respuesta, y su modelización con una variable Bernoulli, con p su valor esperado y la probabilidad de fractura. Buscamos pues, un modelo a través del que relacionar esta probabilidad con un predictor lineal construido en función de las variables **age** y **priorfrac**. Como hemos visto antes, la función **logit** nos permite manejar este cambio de escala de una probabilidad a toda la recta real, y por otro lado, la familia binomial integra a la distribución Bernoulli, por lo que planteamos lo que popularmente se conoce como **el modelo logit**, que aplicado a GLOW para predecir la fractura en función de la edad y el haber tenido una fractura previa, se formularía así:

$$Y = \begin{cases} 1 & \text{fractura, con probabilidad } \pi \\ 0 & \text{no fractura, con probabilidad } 1 - \pi \end{cases} \rightarrow \text{logit}(\pi) = \theta + \beta \cdot \text{age} + \gamma \cdot \mathbf{1}(\text{priorfrac} = \text{Yes})$$

💡 En R. El `glm()` con datos binarios

La función `glm(y ~ x1+x2+x1:x2+...,family=binomial(link = "logit"),data)` ajusta un modelo lineal generalizado en R, integrando los tres componentes del GLM:

- **Respuesta (componente aleatoria):** Si se trata de una respuesta binaria, esta puede venir etiquetada (No/Yes) o recodificada (0/1), siempre vinculada a la familia binomial $\rightarrow$ `family = binomial`. La categoría de “éxito” -y esto es relevante porque define la probabilidad que se estima- siempre es “1” en la codificación 0/1, y la que alfabéticamente esté posicionada la última (en No/Yes, sería Yes, pues se asume No=0/Yes=1).
- **Predictor lineal (componente sistemática):** Se especifica igual que en un `lm()` -lineal en los parámetros: `x1+x2+x1:x2+...`
- **Enlace:** `link = "logit"` conecta la media —la probabilidad— con el predictor y mantiene $\hat{\pi}$ en $(0, 1)$.

Los principales descriptivos los obtenemos con: - `summary(fit_glm)`, el resultado del ajuste - `tidy(fit_glm)` formatea el resultado - `glance()` da la **deviance** y el **AIC**, con las que evaluamos la calidad del ajuste. - `fitted(fit_glm)` da la predicción de probabilidades en los registros observados. - `broom::augment(fit_glm, type.predict = "response")` da los ajustes ordenados en $[0, 1]$ - `predict()` devuelve la predicción en la **escala del predictor** (log-odds) para cualquier valor de los predictores. Para obtenerla en **probabilidad** hay que pedir la escala de respuesta:

```
predict(fit_glm, newdata = ..., type = "response") # probabilidad en [0,1]
predict(fit_glm, newdata = ...)                   # por defecto: log-odds (type = "1
```

Ajustamos pues el modelo, y mostramos el resultado.

```
fit_glm <- glm(fracture ~ age + priorfrac, family = binomial, data = glow)
# Resultado del ajuste
summary(fit_glm)
```

Call:

```
glm(formula = fracture ~ age + priorfrac, family = binomial,
     data = glow)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-4.21429	0.84784	-4.971	6.67e-07	***
age	0.04119	0.01218	3.382	0.000719	***
priorfracYes	0.83884	0.23416	3.582	0.000340	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 562.34 on 499 degrees of freedom
Residual deviance: 528.52 on 497 degrees of freedom
AIC: 534.52

Number of Fisher Scoring iterations: 4

Este ajuste proporciona, como el `lm()`, coeficientes, medidas de bondad de ajuste, que podemos mostrar con formato con la librería `broom`.

```
tidy(fit_glm) # coeficientes (escala log-odds)
```

A tibble: 3 x 5

	term	estimate	std.error	statistic	p.value
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	(Intercept)	-4.21	0.848	-4.97	0.000000667
2	age	0.0412	0.0122	3.38	0.000719
3	priorfracYes	0.839	0.234	3.58	0.000340

```
glance(fit_glm) # deviance, AIC, BIC, gl...
```

```
# A tibble: 1 x 8
  null.deviance df.null logLik   AIC   BIC deviance df.residual  nobs
      <dbl>    <int>  <dbl> <dbl> <dbl>   <dbl>      <int> <int>
1      562.      499  -264.  535.  547.   529.      497   500

augment(fit_glm, type.predict = "response") |> # ajustes en escala probabilidad
  dplyr::select(fracture, age, priorfrac, .fitted) |>
  slice_head(n = 5)
```

```
# A tibble: 5 x 4
  fracture age priorfrac .fitted
  <fct>   <int> <fct>      <dbl>
1 No      62 No      0.160
2 No      65 No      0.177
3 No      88 Yes     0.562
4 No      82 No      0.302
5 No      61 No      0.154
```

La Figura 8 muestra las predicciones que se obtienen en los dos modelos considerados hasta el momento: el lineal y el lineal generalizado.

```
# Rejilla de predicción de los modelos
grid <- expand_grid(
  age      = seq(min(glow$age), max(glow$age), length.out = 100),
  priorfrac = factor(c("No", "Yes"), levels = levels(glow$priorfrac))
)
grid$OLS <- predict(fit_ols, newdata = grid) # escala probabilidad
grid$GLM <- predict(fit_glm, newdata = grid, type = "response") # escala probabilidad
grid_long <- pivot_longer(grid, c(OLS, GLM),
  names_to = "modelo", values_to = "p_hat") |>
  mutate(modelo = factor(modelo, levels = c("OLS", "GLM")))

# Proporciones observadas por grupo de edad y priorfrac (las de la EDA),
# situadas en la edad media de cada grupo
grupos <- glow |>
  mutate(edad_grupo = cut(age, breaks = seq(55, 95, by = 5))) |>
  filter(!is.na(edad_grupo)) |>
  group_by(edad_grupo, priorfrac) |>
  summarise(age = mean(age), p_fractura = mean(fractura01), n = n(), .groups = "drop")

ggplot() +
```



```
geom_point(data = grupos,
           aes(age, p_fractura, color = priorfrac, size = n)) +
geom_line(data = grid_long,
          aes(age, p_hat, color = priorfrac), linewidth = 1) +
geom_hline(yintercept = c(0, 1), linetype = "dashed") +
facet_wrap(~ modelo) +
labs(x = "Edad", y = "Proporción / probabilidad de fractura",
     color = "Fractura previa", size = "n")
```

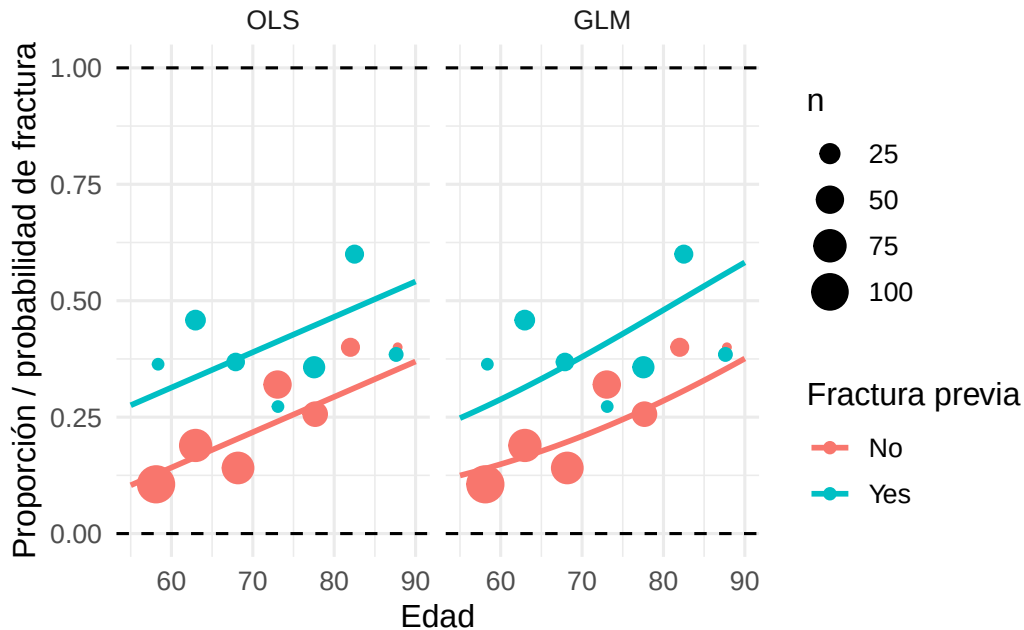


Figura 8: Predicciones de los modelos (`age + priorfrac`) frente a las proporciones de fractura observadas por grupo de edad (el tamaño de los puntos representa el n^o de casos). Ajustes OLS (izquierda) y logístico (derecha) deberían pasar cerca de los puntos del mismo color. En el rango observado ambos se mantienen en $[0,1]$.

Se aprecia en la Figura 8 la diferencia en la predicción de la probabilidad de fractura, asimilada en el OLS por sendas rectas paralelas y en el GLM con dos curvas que intentan adaptar mejor la forma de los datos.

Ya con el glm ajustado, al comparar los residuos medios por tramos (*binned residuals*, en la Unidad 2 para el ajuste OLS y el GLM (Figura 9), apreciamos mejor rendimiento en este último, con la mayoría de residuos dentro de la banda de confianza, a diferencia de los residuos OLS, que fluyen fuera de la banda.

```
p_ols <- plot(performance::binned_residuals(fit_ols)) +
  labs(title = "OLS (modelo lineal de probabilidad)")
p_glm <- plot(performance::binned_residuals(fit_glm)) +
  labs(title = "GLM logístico")
patchwork::wrap_plots( p_ols, p_glm, nrow = 2) +
  patchwork::plot_annotation(title = "Residuos medios por tramos: OLS vs GLM")
```

Residuos medios por tramos: OLS vs GLM

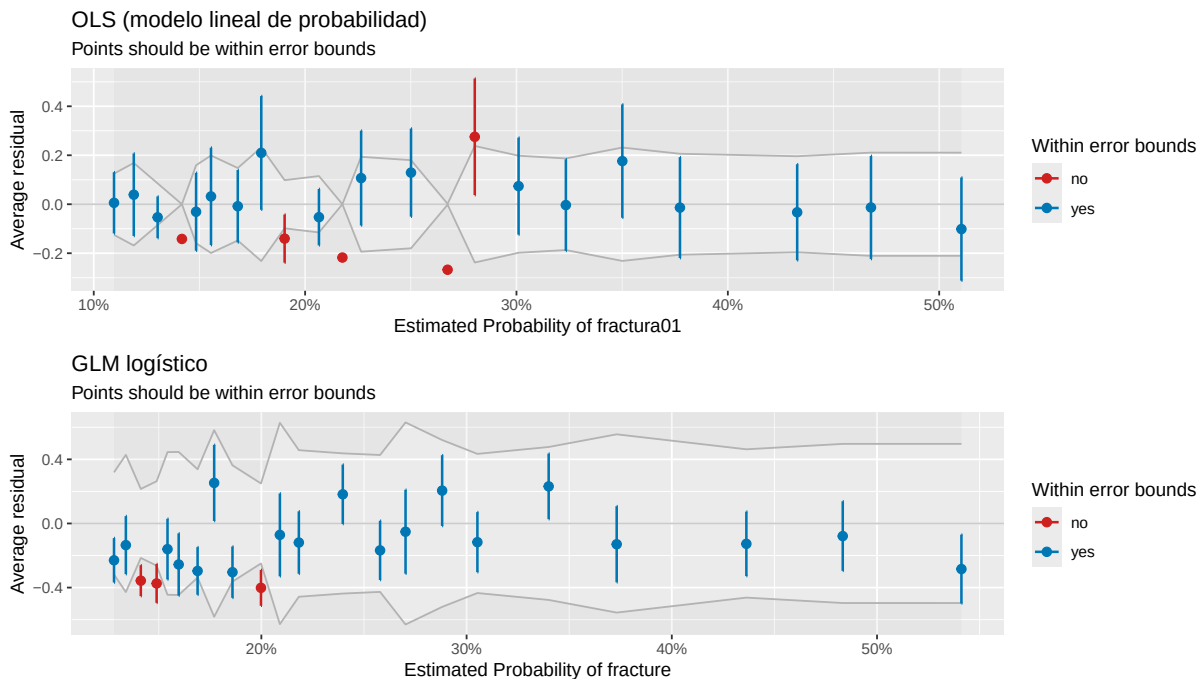


Figura 9: Residuos medios por tramos para los ajustes OLS y GLM.

! Tarea del estudiante (1.1) — arranca la Pregunta 1 del caso

Empiezas a responder la **Pregunta 1** del cliente ([Unidad 6](#)) —¿quién se fractura y cuánto influyen la fragilidad (x_1) y el tratamiento (x_2)?— sobre la **cohorte de tu equipo**. Aquí solo la dejas **planteada y ajustada**; interpretarla, decidirla y evaluarla es cosa de la Unidad 2.

Tareas:

1. **Identifica el objetivo y el tipo de respuesta.** Argumenta que «¿ocurre la fractura?» es una respuesta **binaria** (ever) y que el **modelo lineal (OLS) no sirve**: ajusta `lm(ever ~ x1)` y construye **un** gráfico que evidencie a la vez predicciones

fuera de $[0, 1]$ y los residuos en dos bandas.

2. **Del fallo al GLM.** En una tabla, asocia cada fallo de OLS con el **componente del GLM** (aleatoria / predictor lineal / enlace) que lo resuelve, con una frase de porqué.
3. **Primer ajuste.** Ajusta `glm(ever ~ x1 + x2, family = binomial)` y guárdalo en el `.qmd` del equipo: es el punto de partida de la Pregunta 1, que completarás en la Unidad 2.

Puente a la Unidad 2

Hoy dimos por buenos, sin justificarlos, varios ingredientes: que Bernoulli pertenece a la familia exponencial, que el logit es el enlace canónico, que `glm()` estima por máxima verosimilitud y que la deviance mide el ajuste. La [Unidad 2](#) abre esa caja negra para desarrollarla.

2 · Formulación de los GLM: estimación, inferencia, interpretación y evaluación

En la [Unidad 1](#) dimos por buenos, sin justificarlos, cuatro ingredientes: que la Bernoulli pertenece a la **familia exponencial**, que el **logit** es su enlace canónico, que `glm()` estima por **máxima verosimilitud** y que la **deviance** mide el ajuste. Esta unidad abre esa caja negra y la cierra con lo que de verdad se hace en la práctica: estimar, hacer inferencia, comparar modelos, interpretarlos en la escala correcta y evaluarlos como clasificadores.

Sobre esta unidad

RA implicados:

- **RA2 · Formulación y estimación.** Formular en la familia exponencial, estimar por MV (IWLS) y hacer inferencia (deviance, Wald, razón de verosimilitudes).
- **RA3 · Interpretación en la escala correcta.** Leer los efectos como odds ratios y efectos marginales.
- **RA4 · Diagnóstico y bondad de ajuste.** Separación, calibración, residuos y evaluación como clasificador.
- **RA5 · Selección y comparación de modelos.** Deviance, AIC/BIC y contrastes de razón de verosimilitudes.

(Agresti, 2015; Dobson & Barnett, 2018; Fox, 2016; Hosmer et al., 2013; McCullagh & Nelder, 1989; Nelder & Wedderburn, 1972)

2.1 Familia exponencial y relación media–varianza

Un GLM exige que la respuesta siga una distribución de la **familia exponencial de dispersión**, cuya densidad (o función de masa) se escribe en la forma canónica

$$f(y; \theta, \phi) = \exp \left\{ \frac{y\theta - b(\theta)}{a(\phi)} + c(y, \phi) \right\},$$

donde θ es el **parámetro natural** (o canónico), $b(\theta)$ la **función cumulante**, ϕ el **parámetro de dispersión** y $a(\phi)$, $c(y, \phi)$ funciones conocidas (Dobson & Barnett, 2018; McCullagh & Nelder, 1989). Esta forma no es un capricho notacional: de ella se derivan, derivando la cumulante, los dos momentos que gobiernan todo el marco.

i Quién vive en esta familia (y para qué sirve)

La misma forma canónica agrupa distribuciones muy distintas; cada una es la herramienta natural para un tipo de fenómeno del mundo real:

- **Bernoulli / Binomial** — sucesos sí/no: ¿hay fractura? (GLOW), abandono de cliente (*churn*), impago de un crédito, resultado de un test diagnóstico.
- **Poisson** — recuentos por unidad de tiempo o espacio: reclamaciones de un asegurado al año, visitas a urgencias, fallos de una máquina, llamadas por hora.
- **Exponencial** — tiempo continuo hasta un evento con riesgo constante (sin memoria): vida útil de un componente; abre el hilo de supervivencia (Caso 3).
- **Geométrica** — número de intentos hasta el primer éxito: envíos hasta la primera respuesta, ciclos hasta el primer fallo.
- **Gaussian** — magnitudes continuas sin restricción de signo: estatura, error de medida. Es el OLS que ya conoces, subsumido en el marco.
- **Gamma** — magnitudes positivas y asimétricas con variabilidad proporcional a la media: coste de un siniestro, gasto sanitario por paciente, duración de una sesión (Caso 3).
- **Inversa gaussiana / Tweedie** — duraciones de cola pesada, o datos con masa en cero y parte continua positiva a la vez (coste agregado de siniestros, precipitación diaria).

Partiendo de que $\int f dy = 1$ y derivando bajo el signo integral se obtiene el resultado central:

$$\mathbb{E}[Y] = \mu = b'(\theta), \quad \text{Var}(Y) = b''(\theta) a(\phi).$$

La media depende solo de θ y la varianza se **factoriza** en dos piezas: $a(\phi)$, común a la familia, y $b''(\theta)$, que reescrita en función de la media define la **función de varianza** $V(\mu) = b''(\theta(\mu))$. Esa $V(\mu)$ es la **huella digital** de cada familia: es lo que distingue una Bernoulli de una Poisson

o una Gamma, y lo que en 1.1 «absorbía los problemas 2 y 3» de OLS (varianza no constante y error no normal).

i El otro factor de la varianza: $a(\phi)$ y los pesos a priori

En los modelos GLM estándar el primer factor de $\text{Var}(Y_i) = a(\phi) V(\mu_i)$ toma **habitualmente** la forma

$$a(\phi) = \frac{\phi}{m_i},$$

con ϕ el **parámetro de dispersión** (común a la familia) y m_i un **peso a priori conocido** que registra cuántas unidades elementales hay detrás de cada fila. En general $a(\phi)$ es solo *alguna* función de ϕ ; la forma ϕ/m_i es la convención que surge cuando cada observación resume m_i unidades i.i.d.

Con datos **individuales** —una fila, una observación— se tiene $m_i = 1$ y por tanto $a(\phi) = \phi$. Es la situación de todo el Caso 1: cada mujer de GLOW aporta un único ensayo Bernoulli.

El peso $m_i \neq 1$ aparece cuando el dato llega **agregado**:

- **Binomial como proporción.** Si y_i es la fracción de éxitos sobre m_i ensayos, m_i es el denominador binomial y $\text{Var}(\bar{Y}_i) = \mu_i(1 - \mu_i)/m_i$. En R: respuesta `cbind(éxitos, fracasos)` o bien `weights = m`.
- **Gaussiana o Gamma sobre medias de grupo.** Si y_i es el promedio de m_i mediciones, m_i es el tamaño del grupo.

No conviene confundir las dos piezas: ϕ es la **escala libre** de la familia (recuadro siguiente), mientras que m_i refleja la **estructura de los datos** (réplicas o agregación), no la distribución.

i ¿Uno o dos parámetros? El papel de ϕ

En **Bernoulli, Poisson, exponencial y geométrica** el parámetro de dispersión es **conocido y fijo**, $\phi = 1$: no hay escala libre que estimar, y un **único parámetro** θ (equivalentemente, la media μ) gobierna a la vez la media y la varianza. Por eso $V(\mu)$ las liga de forma rígida — $\mu(1 - \mu)$, μ ...—. Son familias exponenciales **lineales** (un solo parámetro).

El resto de variantes —**Gaussiana, Gamma, inversa gaussiana, Tweedie**— añaden un parámetro de **dispersión propio** que se estima de los datos: el **escalamiento se modeliza con independencia de la media**, lo que permite describir situaciones más ricas (varianza constante en la Gaussiana, coeficiente de variación constante en la Gamma...). Son los **modelos de dispersión exponencial**.

Ese mando ϕ no es ornamental: reaparece como **sobredispersión** en los conteos del Caso 2 (quasi-Poisson, binomial negativa) y como las **continuas positivas** del Caso 3. En el

Caso 1, al ser todo Bernoulli/Binomial, $\phi = 1$ y no hay nada que estimar.

Tabla 6: Familias de la exponencial: parametrización e ingredientes canónicos.

	Bernoulli	Poisson	Gaussiana	Gamma
Parametrización	$\text{Br}(\mu)$	$\text{Pois}(\mu)$	$\mathcal{N}(\mu, \sigma^2)$	$\text{Ga}(\nu, \frac{\mu}{\nu})$
$\theta(\mu)$	$\log \frac{\mu}{1-\mu}$	$\log \mu$	μ	$-1/\mu$
$b(\theta)$	$\log(1 + e^\theta)$	e^θ	$\theta^2/2$	$-\log(-\theta)$
$b'(\theta)$	$\frac{e^\theta}{1 + e^\theta}$	e^θ	θ	$-1/\theta$
$b''(\theta)$	$\frac{e^\theta}{(1 + e^\theta)^2}$	e^θ	1	$1/\theta^2$
$a(\phi)$	1	1	$\phi = \sigma^2$	$\phi = \nu^{-1}$
$c(y, \phi)$	0	$-\log y!$	$-\frac{1}{2} \left(\frac{y^2}{\phi} + \log(2\pi\phi) \right)$	$\frac{1-\phi}{\phi} \log y - \frac{\log \phi}{\phi} - \log \Gamma\left(\frac{1}{\phi}\right)$
$V(\mu)$	$\mu(1 - \mu)$	μ	1	μ^2
Enlace canónico	logit	log	identidad	inversa

La fila $V(\mu)$ explica de un vistazo por qué OLS (Gaussiana, $V \equiv 1$) fracasa en binaria: para la Bernoulli, $V(\mu) = \mu(1 - \mu)$ **no** es constante. El objeto `family()` de R expone exactamente estas piezas, y cada familia permite recuperar sus ingredientes canónicos:

```
family(fit_glm)           # familia y enlace del ajuste de 1.1
```

```
Family: binomial
Link function: logit
```

```
binomial()$variance       # función de varianza V(mu) = mu(1-mu)
```

```
function (mu)
mu * (1 - mu)
<bytecode: 0x7fcfbf6611b28>
<environment: 0x7fcfbfbfadbd8>
```

```
binomial()$linkfun(0.25)  # logit(0.25): de la media al predictor
```

```
[1] -1.098612
```

```
binomial()$linkinv(-1.0986) # sigmoide: del predictor a la media
```

```
[1] 0.2500023
```

La varianza de una sola respuesta binaria no se observa: un 0/1 no tiene dispersión propia. Para *verla*, agrupamos GLOW en diez tramos (deciles) según la probabilidad ajustada $\hat{\mu}$; dentro de cada tramo el riesgo es parecido, así que podemos medir allí la varianza empírica de la respuesta. Confrontamos ese valor con la parábola $\mu(1 - \mu)$: si los puntos la siguen, confirmamos —solo con datos— que en la Bernoulli la varianza la **dicta la media** (máxima en $\mu = 0,5$, casi nula cerca de 0 o 1), no una constante como supondría OLS. La Figura 10 muestra que los puntos siguen la parábola.

```
glow |>
  mutate(p_hat = fitted(fit_glm),
         tramo = ntile(p_hat, 10)) |>
  group_by(tramo) |>
  summarise(mu = mean(p_hat),
            var_emp = var(fractura01), .groups = "drop") |>
  ggplot(aes(mu, var_emp)) +
  geom_function(fun = \(m) m * (1 - m), linewidth = 1, color = "firebrick") +
  geom_point(size = 2) +
  labs(x = expression(hat(mu)), y = "Varianza empírica de Y por tramo",
       caption = expression("Curva: V(" * mu * ") = " * mu * "(1 - " * mu * ")"))
```

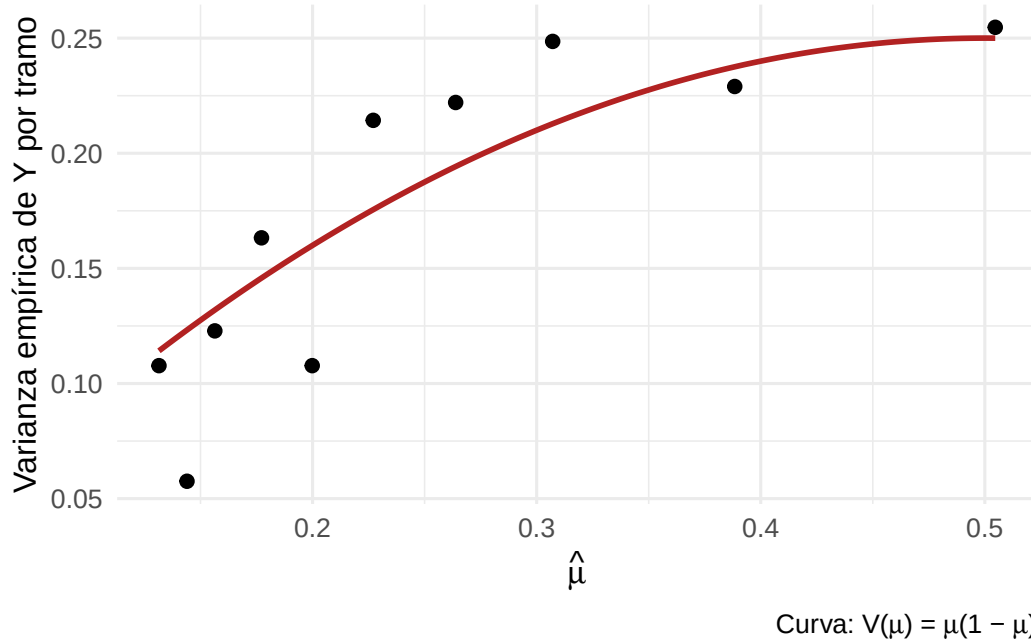


Figura 10: Relación media–varianza en GLOW. La varianza empírica de la respuesta por tramos de probabilidad ajustada (puntos) sigue la parábola teórica $V(\mu) = \mu(1 - \mu)$ (línea), no una constante: justo lo que OLS no puede reproducir.

2.2 El enlace: canónico y alternativas

El **enlace** o *función link* g conecta la media con el predictor lineal, $g(\mu) = \eta = \mathbf{x}^\top \beta$. Una misma familia puede utilizar cualquier enlaces g que sea una función monótona y diferenciable, pero el **enlace canónico** es aquel que iguala el parámetro natural al predictor lineal, esto es, $\theta = \eta$. Este enlace canónico simplemente aporta ventajas formales en las matemáticas, como estadísticos suficientes de dimensión mínima o simplificación de las ecuaciones de verosimilitud.

En el caso binario de la familia binomial, las tres alternativas que re-escalan un predictor lineal -teóricamente definido en toda la recta real- a la escala de una probabilidad -en el rango $[0,1]$ - son el enlace logit, el probit y el complementario log-log -cloglog-, que reproducen la forma de S invertida a la hora de relacionar el predictor lineal con la probabilidad, como se aprecia en la Figura 11. Logit y probit son **simétricos** (casi intercambiables tras reescalar: el coeficiente probit = logit/1.6); el **cloglog** (complementario log-log) es **asimétrico** y será el enlace natural de la supervivencia en tiempo discreto en la Unidad 5 (Agresti, 2015).

$$\text{logit: } \log \frac{\pi}{1-\pi} = \eta, \quad \text{probit: } \Phi^{-1}(\pi) = \eta, \quad \text{cloglog: } \log(-\log(1 - \pi)) = \eta.$$

💡 En R. Especificar el enlace (logit, probit, cloglog)

En R, la forma de especificar los distintos logit se hace a través del argumento `link={ 'logit', 'probit', 'cloglog' }`, o bien directamente como un argumento de la familia, `family={ binomial("logit"), binomial("probit"), binomial("cloglog") }`.

```
tibble(eta = seq(-5, 5, by = 0.02)) |>
  mutate(logit = plogis(eta),
         probit = pnorm(eta),
         cloglog = 1 - exp(-exp(eta))) |>
  pivot_longer(c(logit, probit, cloglog), names_to = "enlace", values_to = "p") |>
  ggplot(aes(eta, p, color = enlace)) +
  geom_line(linewidth = 1) +
  geom_hline(yintercept = c(0, 1), linetype = "dotted") +
  labs(x = expression(eta), y = expression(p == g^{-1}(eta)), color = "Enlace")
```

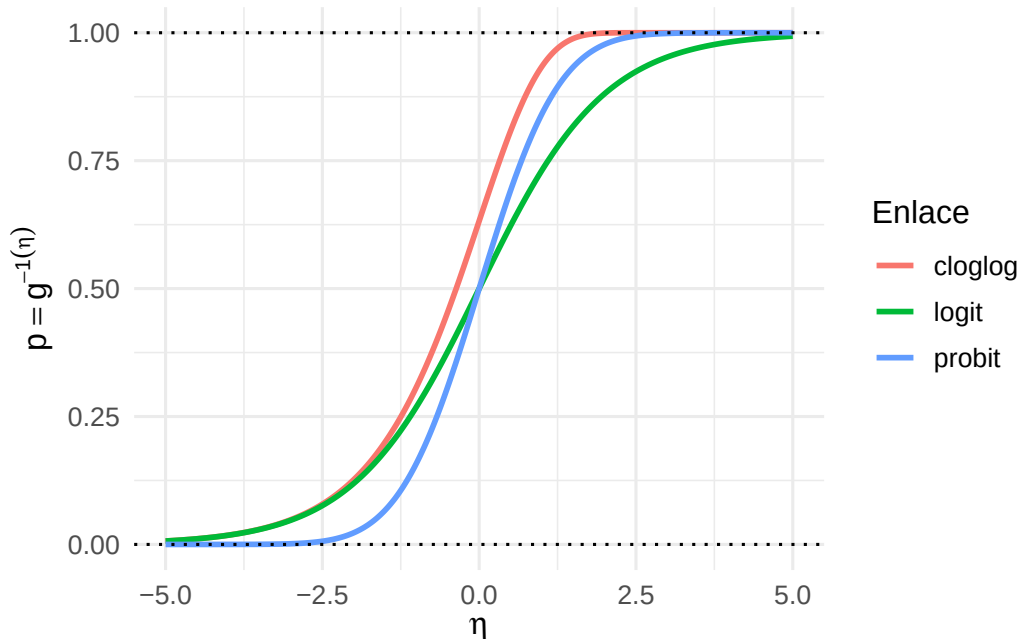


Figura 11: Funciones enlace en la familia binomial. Las tres inversas devuelven a (0,1). logit y probit son simétricas (casi intercambiables tras reescalar); el cloglog es asimétrico.

Para ver el efecto de utilizar diferentes enlaces, ajustamos GLOW con los tres y comparamos:

```

ajustes <- list(
  logit    = glm(fracture ~ age + priorfrac, binomial("logit"),  glow),
  probit   = glm(fracture ~ age + priorfrac, binomial("probit"), glow),
  cloglog  = glm(fracture ~ age + priorfrac, binomial("cloglog"), glow))

map_dfr(ajustes, \(m) glance(m) |> select(deviance, AIC), .id = "enlace")

# A tibble: 3 x 3
  enlace deviance  AIC
  <chr>    <dbl> <dbl>
1 logit      529.  535.
2 probit     528.  534.
3 cloglog    530.  536.

map_dfr(ajustes, tidy, .id = "enlace") |>
  select(enlace, term, estimate) |>
  pivot_wider(names_from = enlace, values_from = estimate)

# A tibble: 3 x 4
  term          logit  probit cloglog
  <chr>        <dbl>   <dbl>   <dbl>
1 (Intercept) -4.21   -2.54   -3.78
2 age          0.0412  0.0248  0.0332
3 priorfracYes 0.839   0.508   0.680

```

Los coeficientes estimados **no** son directamente comparables entre enlaces (viven en escalas distintas y como veremos, se interpretan de forma distinta), pero el ajuste global, calibrado en términos del AIC -que ya conocemos del modelo lineal- y la deviance -que presentaremos más tarde-, es muy parecido, con lo cual en este caso concluiríamos que **la elección de enlace apenas cambia las conclusiones**. Esto es lo habitual salvo en comportamientos extremos en las colas o en datos con mucha asimetría. Por interpretabilidad solemos utilizar el logit en la familia binomial.

Esta equivalencia entre enlaces se aprecia en la escala de la **probabilidad**, en la Figura 12, que superpone las curvas edad-probabilidad de los tres enlaces, separando en dos paneles las predicciones según el antecedente de fractura previa. Con el fin de ver la variación que se produce en las colas, y a pesar de que no es pertinente la extrapolación de la predicción fuera del rango observado, se han punteado las predicciones prolongadas fuera de dicho rango, para visualizar la forma de S invertida completa, y las diferencias en dichas colas.

```

rango_obs <- range(glow$age)
grid <- expand_grid(
  age      = seq(40, 105, length.out = 300),
  priorfrac = factor(c("No", "Yes"), levels = levels(glow$priorfrac)))

# Predicción en probabilidad de cada uno de los tres modelos de `ajustes`
pred_enlaces <- map_dfr(ajustes, \(m) {
  grid |> mutate(p_hat = predict(m, newdata = grid, type = "response"))
}, .id = "enlace") |>
  mutate(enlace = factor(enlace, levels = c("logit", "probit", "cloglog")))
pred_obs <- filter(pred_enlaces, age >= rango_obs[1], age <= rango_obs[2])

etiq_pf <- c(No = "Sin fractura previa", Yes = "Con fractura previa")

ggplot(pred_enlaces, aes(age, p_hat, color = enlace)) +
  annotate("rect", xmin = rango_obs[1], xmax = rango_obs[2],
    ymin = -Inf, ymax = Inf, alpha = 0.05) +
  geom_line(linewidth = 0.7, linetype = "22") +          # extrapolación
  geom_line(data = pred_obs, linewidth = 1.1) +          # rango observado
  facet_wrap(~ priorfrac, labeller = labeller(priorfrac = etiq_pf)) +
  scale_y_continuous(limits = c(0, 1), labels = scales::label_percent()) +
  labs(x = "Edad (años)", y = "P(fractura)", color = "Enlace") +
  theme(legend.position = "bottom")

```

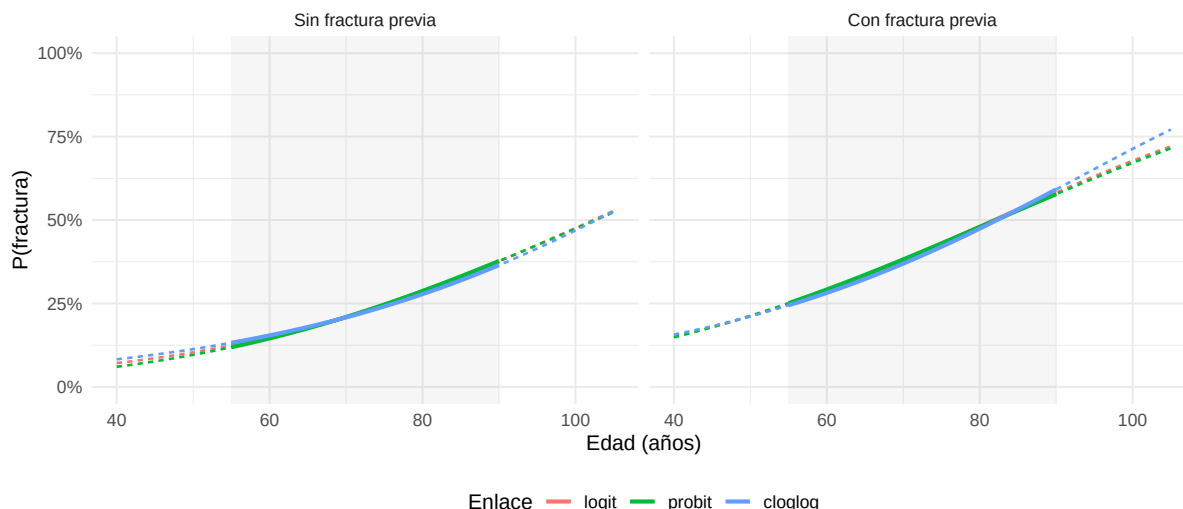


Figura 12: Probabilidad de fractura predicha por los tres enlaces (logit, probit, cloglog) sobre `age + priorfrac`, en paneles por antecedente de fractura previa. Trazo sólido y franja sombreada: rango observado en GLOW (55–90 años); punteado: extrapolación para apreciar la forma en S.

En el rango de edad **realmente observado** (zona sombreada, 55–90 años) las tres curvas son prácticamente **indistinguibles**: la elección de enlace no mueve ni las probabilidades predichas ni, por tanto, las conclusiones, en línea con la deviance y el AIC casi idénticos de la tabla anterior. Las diferencias solo asoman al **extrapolar** hacia las colas, donde el **cloglog** —asimétrico— se despega un poco de logit y probit (simétricos, casi superpuestos entre sí). Es el comportamiento típico: el enlace importa en las colas y bajo fuerte asimetría, no en el grueso de los datos, lo que respalda quedarnos con el logit por interpretabilidad.

2.3 Interpretación: del coeficiente a la probabilidad

El enlace rompió la línea recta. En OLS, β era el cambio en la respuesta por cada unidad de x : una lectura directa. Aquí ya no. El modelo es lineal en el **predictor** $\eta = \beta_0 + \beta x$, pero la respuesta es $\mu = g^{-1}(\eta)$, una transformación **no lineal** de η . Por eso β ya **no** mide el cambio en la probabilidad, sino el cambio en η : la misma diferencia en x mueve mucho la probabilidad en el centro de la curva y casi nada en las colas. Para interpretar, leeremos los coeficientes en la escala donde el modelo **sí** es lineal —y le pondremos nombre.

Lo que da el modelo: `summary()` (o `broom::tidy`) devuelve los coeficientes en la escala del predictor lineal:

Recordemos el modelo que estamos ajustando sobre GLOW:

$$\pi = P(Y = 1) \rightarrow \text{logit}(\pi) = \theta + \beta \cdot \text{age} + \gamma \cdot \mathbf{1}(\text{priorfrac} = \text{Yes})$$

```
tidy(fit_glm) # coeficientes en escala log-odds: lo que da summary() por defecto
```

```
# A tibble: 3 x 5
  term          estimate std.error statistic    p.value
  <chr>         <dbl>     <dbl>     <dbl>    <dbl>
1 (Intercept)  -4.21      0.848     -4.97 0.000000667
2 age          0.0412    0.0122     3.38 0.000719
3 priorfracYes 0.839      0.234     3.58 0.000340
```

Su **signo** ya informa —positivo sube η y con él la probabilidad; negativo la baja—, pero su **magnitud** no es un cambio en probabilidad. ¿Qué es, entonces, esa escala?

Interpretación en la escala de los odds y log-odds

Aparecen en la ecuación los **odds y log-odds**, que provienen del enlace canónico de la Bernoulli.

- El *odds* de un suceso es su **ventaja**, la probabilidad de que ocurra frente a la de que no: un odds de 2 dice que el evento es el doble de probable que su ausencia.

$$\text{odds}(x) = \frac{\pi(x)}{1 - \pi(x)}$$

- Su logaritmo —el **log-odds** o *logit*— es precisamente el enlace canónico, y por tanto **es** el predictor lineal:

$$\eta(x) = \log \frac{\pi(x)}{1 - \pi(x)} = \log .\text{odds}(x).$$

La escala de los log.odds es donde viven los coeficientes y donde el modelo es una recta.

$$\log .\text{odds}(x) = \eta(x) \rightarrow \text{odds}(x) = e^{\eta(x)}$$

Qué dice β ahí, y el odds ratio:

- Subir x una unidad **suma** β al log-odds: el efecto es **aditivo**

$$\beta = \eta(x + 1) - \eta(x) \rightarrow \log .\text{odds}(x + 1) = \beta + \log .\text{odds}(x)$$

- Al deshacer el logaritmo surge el **odds ratio** (OR), como un cociente de odds:

$$\beta = \log \frac{\text{odds}(x+1)}{\text{odds}(x)} = \log(OR) \rightarrow OR = e^\beta$$

de modo que el efecto en los **odds** es multiplicativo, y cada unidad de x multiplica los odds por el odds ratio:

$$\text{odds}(x+1) = e^{\eta(x)+\beta} = \text{odds}(x) \cdot e^\beta = \text{odds}(x) \cdot OR$$

Por eso, si bien el log-odds depende de x , el OR es un factor **constante** que aplica a toda la curva, y de ahí su interpretabilidad: cada movimiento de una unidad en x , sea cual sea el x , multiplica los odds por el mismo factor e^β . La relación entre el signo de los coeficientes, el OR y el efecto que genera en las probabilidades, es ahora más patente:

- $\beta > 0 \Leftrightarrow OR > 1$: aumenta la ventaja del evento;
- $\beta < 0 \Leftrightarrow OR < 1$: reduce la ventaja del evento.

La Figura 13 lo resume. Utiliza los valores $\beta = 1$ (en azul) y $\beta = -1$ (en rojo), y muestra el efecto de variar el predictor x (en el eje X) sobre el log-odds, el odds y la probabilidad del evento. A la izquierda, el log-odds = $\beta + x$ identifica una recta cuya pendiente es β (efecto aditivo, constante). En el centro, se aprecia la curva exponencial del odds = $e^{\beta + x}$: cada unidad de x multiplica las odds por e^β . A la derecha, la relación entre el predictor x y la probabilidad = $1/(1+e^{-(\beta + x)})$ viene representada en forma de S, creciente o decreciente, con efecto máximo en el centro y casi nulo en las colas. Esa no linealidad de la derecha es justo lo que impide leer como un cambio de probabilidad.

```
betas <- c(" = +1 (positivo)" = 1, " = -1 (negativo)" = -1)
demo <- expand_grid(x = seq(-3, 3, length.out = 300), signo = names(betas)) |>
  mutate(beta = betas[signo],
         `log-odds` = beta * x,          # 0 = 0
         odds       = exp(beta * x),
         prob       = plogis(beta * x))

col <- c(" = +1 (positivo)" = "#2c7fb8", " = -1 (negativo)" = "#d7301f")
escala <- function(y, ylab, titulo, y0)
  ggplot(demo, aes(x, .data[[y]], color = signo)) +
    geom_hline(yintercept = y0, linetype = "dotted", color = "grey60") +
    geom_line(linewidth = 1.1) +
    scale_color_manual(values = col) +
    labs(x = "predictor x", y = ylab, color = NULL, title = titulo)

p1 <- escala("log-odds", "log-odds ( )", " =  + x  ·  recta (pendiente )", 0)
p2 <- escala("odds", "odds = e^", "odds  ·  multiplica e^ por unidad", 1)
```

```
p3 <- escala("prob",      "P(evento)",      "probabilidad · forma en S",      0.5) +
  scale_y_continuous(limits = c(0, 1), labels = scales::label_percent())

(p1 + p2 + p3) + plot_layout(guides = "collect") &
  theme(legend.position = "bottom",
        plot.title = element_text(size = 9.5, face = "bold"))
```

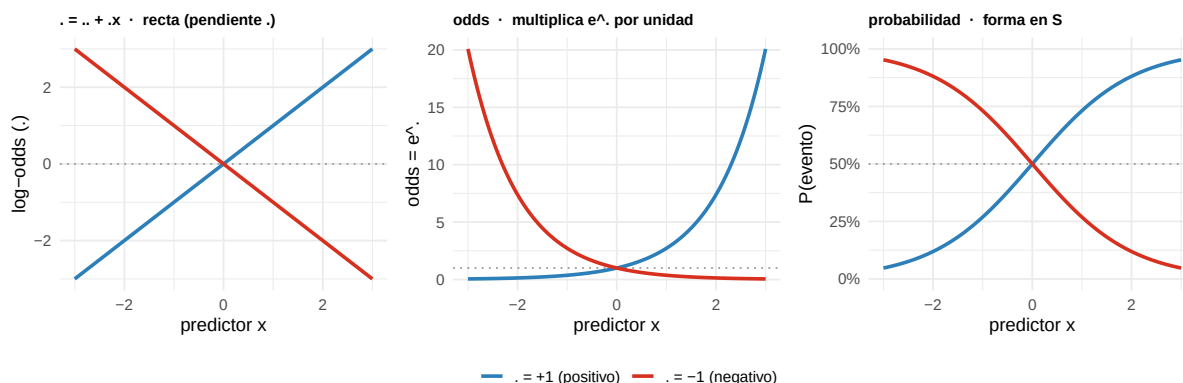


Figura 13: Efecto de la variación en un predictor genérico x ($x = 0$) sobre el log-odds, el odds y la probabilidad del evento. En azul un coeficiente positivo ($\beta = +1$); en rojo uno negativo ($\beta = -1$).

⚠ Odds ratio y riesgo relativo

Puntualizar que odds-ratio OR no es lo mismo que riesgo relativo RR. El riesgo relativo es útil para comparar probabilidades, luego en el contexto del modelo lineal cabría valorar el efecto, sobre la probabilidad, de añadir una unidad a la x

$$RR = \pi(x+1)/\pi(x)$$

En general $OR \neq RR$, si bien ambos se relacionan por

$$OR = RR \cdot \frac{1 - \pi(x)}{1 - \pi(x+1)},$$

de modo que el OR **se aleja de 1 más que el RR** y exagera el efecto cuando el evento es frecuente. Solo si el evento es **raro** (π pequeña) se cumple $1 - \pi \approx 1$ y entonces $OR \approx RR$. Por eso decir «el riesgo se triplica» al leer un $OR = 3$ es una aproximación cómoda pero informal: la lectura exacta es en términos de *odds* (ventaja), o bien en probabilidad mediante los efectos marginales (más abajo).

Intervalos de confianza. Como $OR = e^\beta$ y la exponencial es **monótona creciente**, el IC

del OR es la **exponencial de los extremos** del IC del coeficiente, sin recalcular nada:

$$\text{IC}_{95\%}(\text{OR}) = [e^{\beta_{\text{inf}}}, e^{\beta_{\text{sup}}}]$$

Y como $e^0 = 1$, que el IC de β **no contenga 0** equivale a que el del OR **no contenga 1**: la misma señal de efecto en las dos escalas.

En GLOW. Aplicado a las dos predictoras:

```
exp(cbind(OR = coef(fit_glm), confint(fit_glm))) # OR e IC: exponencial del IC de beta
```

	OR	2.5 %	97.5 %
(Intercept)	0.01478275	0.002737803	0.07645453
age	1.04205301	1.017574674	1.06743089
priorfracYes	2.31367006	1.458786254	3.65820301

La fractura previa multiplica las odds de fractura por $\text{OR} \approx 2,31$ (IC 95 %: 1,46–3,66) y cada año de edad por $\approx 1,04$ (IC 95 %: 1,02–1,07); ninguno cruza 1, de modo que ambos efectos son claros. (El IC de β procede del perfil de verosimilitud, [Sección 2.5](#).)

La misma idea en otros enlaces. El predictor lineal η es **idéntico** en los tres enlaces; cambia qué *representa* y, con ello, cómo se lee el coeficiente:

Tabla 7: Relación entre el predictor lineal η —donde actúan los coeficientes— y la media μ en los tres enlaces. En el logit, η es exactamente el log-odds; cada enlace «lee» η en una escala distinta.

Enlace	η es...	$\eta = g(\mu)$	$\mu = g^{-1}(\eta)$
logit	el log-odds	$\log \frac{\mu}{1-\mu}$	$\frac{e^\eta}{1+e^\eta}$
probit	el cuantil normal (z)	$\Phi^{-1}(\mu)$	$\Phi(\eta)$
cloglog	el log-riesgo (log del <i>hazard</i> acumulado)	$\log(-\log(1-\mu))$	$1 - e^{-e^\eta}$

De ahí que el mismo β signifique cosas distintas aunque el ajuste apenas varíe:

- **logit:** $e^\beta = \text{odds ratio}$ (multiplica las odds); la lectura habitual en clínica y epidemiología.
- **probit:** β **desplaza la escala latente normal** (z), sin lectura de odds; como guía, $\beta_{\text{probit}} \approx \beta_{\text{logit}}/1,6\text{--}1,8$.
- **cloglog:** $e^\beta = \text{razón de riesgos}$ (*hazard ratio*); lectura natural en supervivencia ([Unidad 5](#)).


```
# Mismos datos, distinta escala del coeficiente según el enlace
map_dfr(ajustes, \(m) tidy(m) |> select(term, estimate), .id = "enlace") |>
  pivot_wider(names_from = enlace, values_from = estimate) |>
  mutate(razon_logit_probit = logit / probit) # ~1,6-1,8 (salvo intercepto)
```

```
# A tibble: 3 x 5
  term          logit  probit cloglog razon_logit_probit
<chr>         <dbl>   <dbl>   <dbl>         <dbl>
1 (Intercept) -4.21    -2.54   -3.78          1.66
2 age          0.0412  0.0248  0.0332          1.66
3 priorfracYes 0.839    0.508   0.680          1.65
```

Interpretación en la escala de la probabilidad: efectos marginales

Cuando queremos interpretar un ajuste directamente en términos de las probabilidades, acudimos a los **efectos marginales**. El odds ratio es constante, pero el cambio en **probabilidad** no: un año más de **age** apenas mueve nada en las colas y mucho en el centro de la S. Para resumirlo en un número, el **efecto marginal medio (AME)** responde a la pregunta que de verdad importa:

“¿Cuántos puntos de probabilidad sube el evento, en promedio, por una unidad más de la covariable?”

En GLOW, el AME de **age** es **0,007**: unos **0,7 puntos porcentuales de riesgo de fractura por año**.

¿De dónde sale ese número?

Para calcular el AME o efecto medio marginal asociado a un predictor continuo x_k (con coeficiente β_k en el predictor lineal), se considera el efecto de cada individuo a través de la pendiente de su propia curva, y dado que la derivada del logit es $\mu(1 - \mu)$, esta pendiente es:

$$\frac{\partial \mu_i}{\partial x_k} = \hat{\pi}_i(1 - \hat{\pi}_i) \beta_k.$$

El AME es el promedio sobre las pendientes para las n personas observadas: literalmente, el coeficiente por la pendiente media de la sigmoide.

$$\text{AME}_k = \beta_k \cdot \frac{1}{n} \sum_i \hat{\pi}_i(1 - \hat{\pi}_i)$$

```
# AME de cada covariable, en puntos de probabilidad
avg_slopes(fit_glm)
```

	Term Contrast	Estimate	Std. Error	z	Pr(> z)	S	2.5 %	97.5 %
age	dY/dX	0.00717	0.00205	3.50	<0.001	11.1	0.00316	0.0112
priorfrac	Yes - No	0.16353	0.04900	3.34	<0.001	10.2	0.06749	0.2596

Type: response

```
# Reproducción "a mano" del cálculo del AME para 'age':
p <- fitted(fit_glm) # pi_hat de cada individuo
mean(p * (1 - p)) * coef(fit_glm)["age"] # = AME de age
```

```
age
0.007174101
```

Cuando se trata de evaluar el AME para un factor como **priorfrac**, el coeficiente no es una pendiente (derivada) sino una **diferencia contrafactual**. El AME entonces se calcula con el efecto diferencial sobre la categoría de referencia (en **priorfrac**, $\hat{\pi}$ con **Yes** menos con **No**), promediada entre individuos, es decir:

$$\text{AME}_{\text{priorfrac}} = \frac{1}{n} \sum_{i=1}^n \left[\hat{\pi}(\text{Yes}, x_i) - \hat{\pi}(\text{No}, x_i) \right],$$

donde $\hat{\pi}(\text{nivel}, x_i)$ se evalúa para todos los sujetos en la muestra en todos los niveles (en **priorfrac**, **Yes** y **No**).

$$\hat{\pi}(\text{nivel}, x_i) = \text{logit}^{-1}(\hat{\theta} + \hat{\beta} \text{age}_i + \hat{\delta}_{\text{pf}}^{\text{nivel}}), \quad \hat{\delta}_{\text{pf}}^{\text{ref}} = 0.$$

```
# Reproducción "a mano" del cálculo de AME para 'priorfrac'
g_yes <- transform(glow, priorfrac = factor("Yes", levels = levels(glow$priorfrac)))
g_no  <- transform(glow, priorfrac = factor("No", levels = levels(glow$priorfrac)))

mean(predict(fit_glm, g_yes, type = "response") -
      predict(fit_glm, g_no, type = "response")) # = AME de priorfrac
```

```
[1] 0.1635282
```

El AME en probabilidad es, además, **casi invariante al enlace** (los tres ajustes lo dan parecido pese a coeficientes distintos) y se prefiere frente a otros indicadores por promediar sobre la población observada (Arel-Bundock et al., 2024). En una frase: >El **OR** comunica la fuerza del efecto; el **AME**, su impacto absoluto en probabilidad.

En la Figura 14 se visualiza el efecto de la edad en dos escalas. A la izquierda, en escala log-odds, los dos perfiles son **rectas paralelas**: la edad entra de forma lineal y **priorfrac** solo desplaza la recta una cantidad **constante**, $\log(\text{OR}) \approx 0,84$ —ahí vive el odds ratio—. A la derecha, en probabilidad, esas mismas rectas se vuelven **curvas en S**: el efecto de **priorfrac** (la distancia entre curvas) es máximo en el centro y se desvanece en los extremos, **aunque el OR sea constante**. Claramente se aprecia la diferencia constante en los odds y variable en las probabilidades.

```
rango_obs <- range(glow$age)          # ~55-90: edad observada en GLOW
grid <- expand_grid(
  age      = seq(40, 105, length.out = 300),
  priorfrac = factor(c("No", "Yes"), levels = levels(glow$priorfrac))
pr <- predict(fit_glm, newdata = grid, type = "link", se.fit = TRUE)
grid <- grid |>
  mutate(eta = pr$fit, se = pr$se.fit,
         lo_eta = eta - 1.96 * se, hi_eta = eta + 1.96 * se,    # IC 95 % en log-odds
         p_hat  = plogis(eta),
         lo_p   = plogis(lo_eta), hi_p = plogis(hi_eta))      # IC 95 % en probabilidad
grid_obs <- filter(grid, age >= rango_obs[1], age <= rango_obs[2])

pA <- ggplot(grid, aes(age, eta, color = priorfrac, fill = priorfrac)) +
  annotate("rect", xmin = rango_obs[1], xmax = rango_obs[2],
         ymin = -Inf, ymax = Inf, alpha = 0.06) +
  geom_ribbon(aes(ymin = lo_eta, ymax = hi_eta), alpha = 0.15, color = NA) +
  geom_line(linewidth = 0.7, linetype = "22") +                # extrapolación
  geom_line(data = grid_obs, linewidth = 1.2) +                # rango observado
  labs(x = "Edad (años)", y = "log-odds ( )",
       color = "Fractura previa", fill = "Fractura previa",
       title = "Escala log-odds", subtitle = "rectas paralelas (separación 0,84)")

pB <- ggplot(grid, aes(age, p_hat, color = priorfrac, fill = priorfrac)) +
  annotate("rect", xmin = rango_obs[1], xmax = rango_obs[2],
         ymin = -Inf, ymax = Inf, alpha = 0.06) +
  geom_ribbon(aes(ymin = lo_p, ymax = hi_p), alpha = 0.15, color = NA) +
  geom_line(linewidth = 0.7, linetype = "22") +
  geom_line(data = grid_obs, linewidth = 1.2) +
  scale_y_continuous(limits = c(0, 1), labels = scales::label_percent()) +
```

```
labs(x = "Edad (años)", y = "P(fractura)",
     color = "Fractura previa", fill = "Fractura previa",
     title = "Escala probabilidad", subtitle = "curvas en S (separación variable)")

(pA + pB) +
  plot_layout(guides = "collect") &
  theme(legend.position = "bottom",
        plot.title = element_text(size = 11, face = "bold"),
        plot.subtitle = element_text(size = 8.5))
```

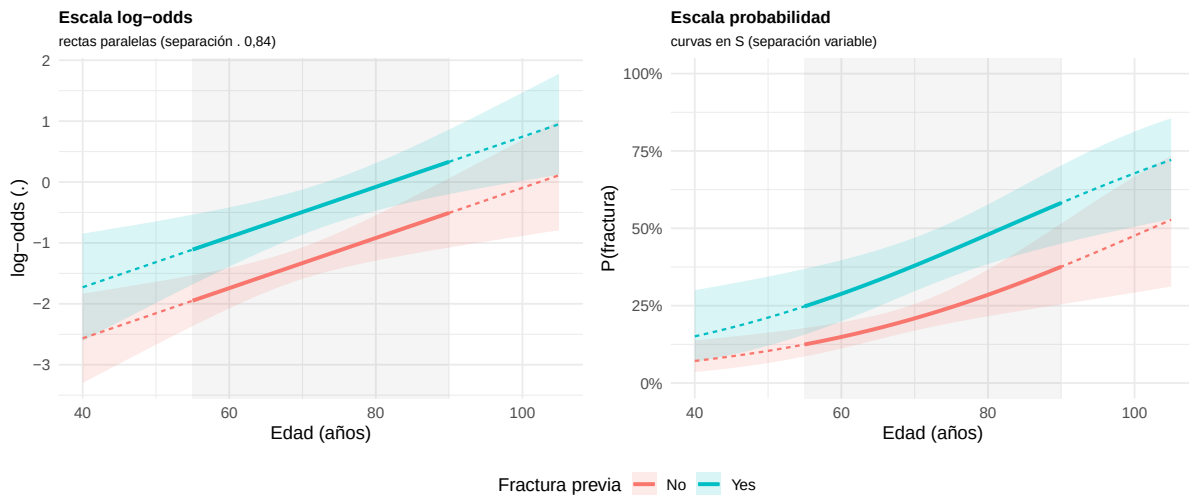


Figura 14: El mismo modelo en dos escalas. Izquierda (log-odds): rectas paralelas, separación constante 0,84 (logaritmo del OR). Derecha (probabilidad): curvas en S, con separación variable. Trazo sólido y franja sombreada: rango de edad observado en GLOW (55–90 años); fuera, extrapolación (punteado). Bandas: IC 95 %.



En R. Odds ratios, probabilidades y efectos marginales

Tres escalas de lectura, cada una con su herramienta:

- **Odds ratios con intervalo** — `broom::tidy(m, exponentiate = TRUE, conf.int = TRUE)`, o a mano `exp(cbind(OR = coef(m), confint(m)))`. `confint()` da el IC por **perfil de verosimilitud** (más fiable que Wald); que el IC del OR no contenga el 1 equivale a que el de β no contenga el 0.
- **Probabilidades predichas** — `predict(m, newdata, type = "response")` da $\hat{\pi}$; con `type = "link", se.fit = TRUE` se obtiene $\hat{\eta}$ y su error, y el IC se construye en la escala del enlace y **se pasa por la sigmoide (plogis)** —nunca sumando $\pm$ en probabilidad—.

- **Efecto marginal medio (AME)** — `marginaleffects::avg_slopes(m)`: el efecto en puntos de probabilidad, casi invariante al enlace. `marginaleffects::predictions()` / `emmeans` dan probabilidades por perfil. El **OR** comunica la fuerza del efecto; el **AME**, su impacto absoluto.

2.4 Estimación: máxima verosimilitud e IWLS

A diferencia de OLS, **no hay fórmula cerrada** para $\hat{\beta}$: se maximiza la log-verosimilitud numéricamente. Pero ese “numéricamente” no es una caja negra; al abrirlo aparece algo muy reconocible —una sucesión de regresiones lineales ponderadas—. Veámoslo en tres pasos: las ecuaciones que hay que resolver, el algoritmo que las resuelve y su reproducción a mano sobre GLOW.

Las ecuaciones de score: ¿dónde están los β ?

La estimación de los coeficientes β del predictor lineal pasa por maximizar la verosimilitud, o lo que es lo mismo, la log-verosimilitud $\ell(\beta)$. Maximizar $\ell(\beta) = \sum_i \ell_i$ exige anular su gradiente. Como los coeficientes entran en la log-verosimilitud a través del predictor lineal, $\eta_i = \mathbf{x}_i^\top \beta$, aplicando la regla de la cadena en la derivación, $\ell_i \rightarrow \mu_i \rightarrow \eta_i \rightarrow \beta$, y dado que $\partial \eta_i / \partial \beta = \mathbf{x}_i$, llegamos a:

$$\frac{\partial \ell}{\partial \beta} = \sum_{i=1}^n \underbrace{\frac{\partial \ell_i}{\partial \mu_i}}_{\frac{y_i - \mu_i}{a(\phi)V(\mu_i)}} \underbrace{\frac{\partial \mu_i}{\partial \eta_i}}_{\text{enlace}} \underbrace{\frac{\partial \eta_i}{\partial \beta}}_{\mathbf{x}_i} = \sum_{i=1}^n \frac{(y_i - \mu_i)}{a(\phi)V(\mu_i)} \frac{d\mu_i}{d\eta_i} \mathbf{x}_i = \mathbf{0}.$$

Resulta pues un sistema que iguala a cero una **suma ponderada de residuos** $(y_i - \mu_i)$, que no tiene solución cerrada y requiere de un procedimiento iterativo de optimización.

Del sistema al algoritmo: Fisher scoring = IWLS (*Iterative Weighted Least Squares*)

El método de **Newton–Raphson**, usando la **información esperada** (*Fisher scoring*), resuelve ese sistema, y tiene una lectura “interesante”: **cada iteración equivale a una regresión por mínimos cuadrados ponderados** (IWLS) (McCullagh & Nelder, 1989; Nelder & Wedderburn, 1972). En cada paso se fabrican: una **respuesta de trabajo** z_i (una linealización del enlace alrededor del ajuste actual) y unos **pesos** w_i (que dan más voz a las observaciones más precisas):

$$z_i = \eta_i + (y_i - \mu_i) \frac{d\eta_i}{d\mu_i}, \quad w_i = \frac{(d\mu_i/d\eta_i)^2}{V(\mu_i)},$$

y se actualiza $\beta \leftarrow (\mathbf{X}^\top \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{W} \mathbf{z}$ —la fórmula de WLS— hasta que deja de cambiar. Así, **estimar un GLM es resolver una sucesión de regresiones lineales ponderadas**.

i Recordatorio: Newton–Raphson y Fisher scoring

Para hallar la raíz de la ecuación de score $\partial \ell / \partial \beta = \mathbf{0}$, **Newton–Raphson** itera $\beta \leftarrow \beta - \mathbf{H}^{-1} \mathbf{s}$, con $\mathbf{s}$ el score (gradiente) y $\mathbf{H}$ el hessiano (segunda derivada). **Fisher scoring** sustituye $\mathbf{H}$ por su valor esperado, la **información esperada** $\mathcal{J} = \mathbb{E}[-\mathbf{H}]$: más estable y, en un GLM, lo que convierte cada paso en una regresión ponderada.

i Recordatorio: mínimos cuadrados ponderados (WLS)

OLS minimiza $\sum_i (z_i - \mathbf{x}_i^\top \beta)^2$ tratando todas las observaciones por igual. **WLS** asigna a cada una un peso w_i y minimiza $\sum_i w_i (z_i - \mathbf{x}_i^\top \beta)^2$, dando más influencia a las de menor varianza; su solución es $\hat{\beta} = (\mathbf{X}^\top \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{W} \mathbf{z}$, con $\mathbf{W} = \text{diag}(w_i)$. El IWLS no es más que aplicarla una y otra vez, recalculando z y w en cada vuelta.

Reproducción a mano sobre GLOW.

Reproducimos ahora el bucle con las piezas que el propio objeto `family()` proporciona (`linkinv`, `mu.eta`, `variance`) y comprobamos que coincide con la solución de `glm()`:

```
X  <- model.matrix(fit_glm)      # matriz de diseño: columnas (Intercept), age, prior
y  <- glow$fractura01            # respuesta binaria 0/1
fam <- binomial()                # la familia expone las piezas: linkinv, mu.eta, var

beta <- rep(0, ncol(X))          # arranque: todos los coeficientes a 0
traza <- list()                  # aquí guardaremos beta de cada iteración (para la f

for (it in 1:25) {
  eta <- as.vector(X %*% beta)    # predictor lineal      eta = X beta
  mu  <- fam$linkinv(eta)         # media ajustada      mu = g^{-1}(eta) = plogis(e
  mep <- fam$mu.eta(eta)         # derivada del enlace  dmu/deta = mu(1-mu)
  z   <- eta + (y - mu) / mep     # respuesta de trabajo (linealización de un paso)
  w   <- mep^2 / fam$variance(mu) # pesos IWLS          w = (dmu/deta)^2 / V(mu)
  WX  <- X * w                   # cada fila de X escalada por su peso
  # paso de mínimos cuadrados ponderados: beta = (X'WX)^{-1} X'W z
  beta_new <- as.vector(solve(crossprod(WX, X), crossprod(WX, z)))
  traza[[it]] <- beta_new        # registramos el beta de esta vuelta
  if (max(abs(beta_new - beta)) < 1e-10) { beta <- beta_new; break } # ¿convergió?
  beta <- beta_new               # si no, otra iteración partiendo del nuevo beta
}
```

```
cat("Convergió en", it, "iteraciones\n")
```

Convergió en 6 iteraciones

```
cbind(IWLS_a_mano = beta, glm = coef(fit_glm)) # deben coincidir con glm()
```

	IWLS_a_mano	glm
(Intercept)	-4.21429456	-4.21429456
age	0.04119282	0.04119282
priorfracYes	0.83883503	0.83883503

La trayectoria hasta la convergencia.

En cada vuelta hemos ido guardando el vector de coeficientes en la lista `traza`; ahora `do.call(rbind, traza)` **apila** esos vectores en una **matriz** —una fila por iteración, una columna por coeficiente—, que pasamos a formato largo para dibujar cómo evoluciona cada coeficiente. La Figura 15 lo muestra: partiendo de cero, los tres coeficientes alcanzan su valor de máxima verosimilitud —las líneas punteadas de `glm()`— en apenas un puñado de iteraciones. Esa convergencia casi inmediata es la marca del Fisher scoring cerca del óptimo.

```
do.call(rbind, traza) |> # apila la lista en matriz: filas = iteraciones, columna
as.data.frame() |>
setNames(colnames(X)) |> # nombra las columnas como los coeficientes
mutate(iter = row_number()) |> # índice de iteración
pivot_longer(-iter, names_to = "coef", values_to = "valor") |> # formato largo para gg
ggplot(aes(iter, valor, color = coef)) +
geom_line() + geom_point(size = 1.5) +
geom_hline(data = enframe(coef(fit_glm), "coef", "obj"), # valores de glm() como
aes(yintercept = obj, color = coef), linetype = "dashed") +
labs(x = "Iteración IWLS", y = "Coeficiente", color = NULL)
```

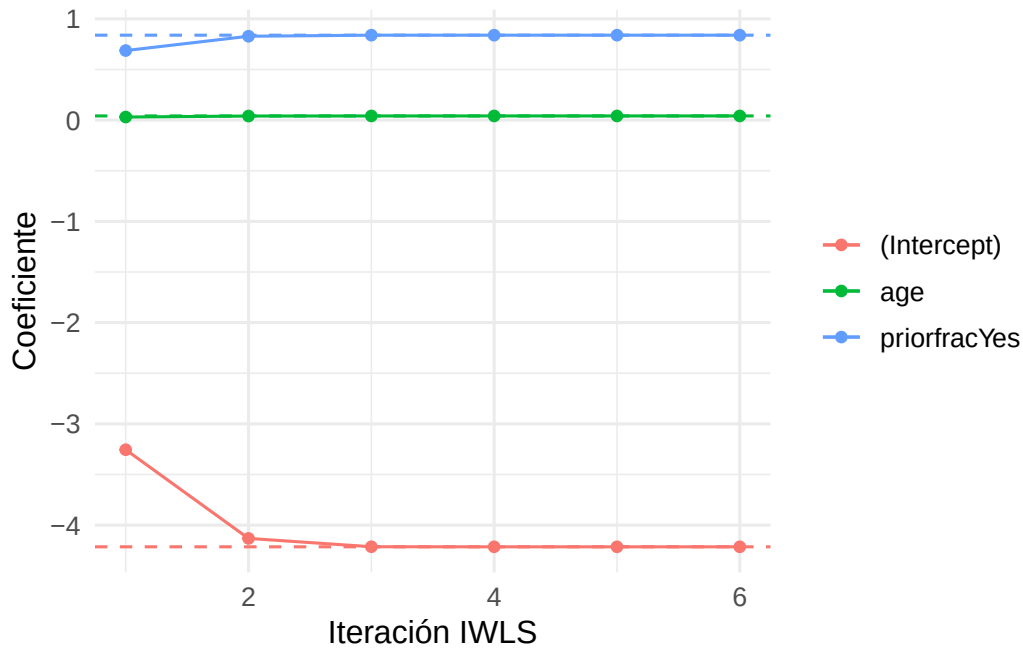


Figura 15: Convergencia del IWLS «a mano». Cada coeficiente se estabiliza en su valor de máxima verosimilitud (líneas punteadas, las de `glm()`) en muy pocas iteraciones.

2.5 Inferencia y selección de modelos

Una vez tenemos los coeficientes estimados, $\hat{\beta}$, toca interrogar al modelo en tres niveles, de lo particular a lo global: 1. ¿es real el efecto de cada coeficiente? 1. ¿aporta conocimiento un bloque de variables? y 1. ¿qué modelo elijo entre varias especificaciones?

Las tres preguntas se responden con la misma moneda —la verosimilitud— y su sombra operativa, la **deviance**.

La Deviance

Para medir cuánto se aleja un ajuste del mejor posible, se compara con el **modelo saturado**, que es el que tiene un parámetro por observación, y por lo tanto reproduce los datos exactamente ($\hat{\mu}_i = y_i$) y proporciona el valor de log-verosimilitud máxima ℓ_{sat} . La **deviance** es esa distancia, en unidades de log-verosimilitud, convertido en un valor positivo:

$$D = -2[\ell(\hat{\theta}) - \ell_{\text{sat}}].$$

La deviance viene a ser el análogo GLM de la suma de cuadrados residual: cuanto menor valor toma, mejor ajuste.

La **deviance nula** D_0 es la del modelo solo con interceptación (el más simple que no asume ningún efecto de los predictores); la **deviance residual** D , representa la deviance del modelo ajustado.

💡 En R. Deviances y grados de libertad de un glm

En un modelo `glm` ajustado, `fit_glm`, las deviances nula, del modelo saturado y del modelo ajustado, junto con sus grados de libertad, se obtienen así:

```
fit_glm$null.deviance      # deviance NULA (modelo solo con intercepto)
fit_glm$df.null            #   sus g.l.: n - 1

deviance(fit_glm)          # deviance del modelo AJUSTADO (= residual); igual a fit_glm$df.residual
df.residual(fit_glm)      #   sus g.l.: n - p; igual a fit_glm$df.residual
```

La deviance del modelo **saturado es 0 por definición**: es el modelo de referencia —un parámetro por observación, con g.l. residuales = 0— frente al que se mide *toda* deviance, de modo que no hay una función que la devuelva (la “residual” ya es la distancia del modelo ajustado al saturado).

Para mostrarlas todas conjuntamente, podemos utilizar `glance()`, de la librería `broom`, que devuelve en una sola fila la `null.deviance` con `df.null`, la `deviance` residual con `df.residual`, y además `logLik`, `AIC`, `BIC` y `nobs`:

```
glance(fit_glm)
```

💡 Tip

Detalle útil: con datos binarios **individuales**, $\ell_{\text{sat}} = 0$, de modo que $D = -2\ell(\hat{\beta})$.

Mostramos las deviances y otros indicadores como el AIC y BIC, que nos serán útiles más adelante para la comparación de modelos:

```
glance(fit_glm) |>
  select(null.deviance, df.null, deviance, df.residual, logLik, AIC, BIC)
```

```
# A tibble: 1 x 7
  null.deviance df.null deviance df.residual logLik   AIC   BIC
      <dbl>    <int>    <dbl>    <int>    <dbl> <dbl> <dbl>
1         562.     499     529.       497  -264.  535.  547.
```

En GLOW, la deviance baja de **562,3** (nula) a **528,5** (residual) al añadir `age` y `priorfrac`: esa reducción de 33,8 puntos es la materia prima de los contrastes que siguen.

La deviance residual **no** sirve como test *absoluto* de ajuste con datos individuales (lo veremos en la [Sección 2.6](#)). De momento la usaremos solo para **comparar** modelos.

Inferencia sobre los coeficientes

Para contestar la pregunta “¿es real el efecto de cada coeficiente?”, se utilizan los intervalos de confianza de Wald y de perfil.

Test de Wald

El test de Wald es el análogo directo de los tests t del modelo lineal. Se construye con el cociente entre el estimador del coeficiente β_k (asociado al predictor x_k) y su error estándar:

$$z_k = \frac{\hat{\beta}_k}{\widehat{\text{EE}}(\hat{\beta}_k)}.$$

Este estadístico de Wald se compara con la normal asintótica (de ahí z y no t), y es el que imprime `summary()/tidy()`. Es un test cómodo, pero **poco fiable** con muestras pequeñas o coeficientes grandes, y **engñoso bajo separación** (da por no significativo un efecto que en realidad es muy fuerte, debido a un inflado del error estándar -efecto (Hauck & Donner, 1977)).

Intervalos de confianza: Wald frente a perfil.

Por la misma razón, los **intervalos de confianza** se prefieren por **perfil de verosimilitud** antes que por Wald (Agresti, 2015). El de **Wald**, $\hat{\beta}_k \pm 1,96 \widehat{\text{EE}}$, aproxima la log-verosimilitud por una **parábola** y es siempre simétrico. El de **perfil** se obtiene **invirtiendo la razón de verosimilitudes**: es el conjunto de valores b que el LRT no rechazaría, es decir, aquellos en los que la **log-verosimilitud perfil**

$$\ell_p(b) = \max_{\beta_{-k}} \ell(b, \beta_{-k})$$

—la verosimilitud maximizada sobre el **resto** de parámetros con $\beta_k = b$ fijo— cae como mucho $1,92 = \frac{1}{2}\chi_{1,0,95}^2$ por debajo de su máximo:

$$\text{IC}_{95\%}(\beta_k) = \left\{ b : \ell(\hat{\beta}) - \ell_p(b) \leq 1,92 \right\}.$$

Geométricamente, se corta la curva de verosimilitud perfil a 1,92 del pico y se toman las dos abscisas del cruce. Como usa la **forma real** —a menudo **asimétrica**— de la verosimilitud, no asume normalidad de $\hat{\beta}$ y **no tiene por qué ser simétrico**: es más fiable justo cuando la parábola de Wald aproxima mal (muestras pequeñas, coeficientes grandes, cerca de la separación).

💡 En R. Intervalos de confianza sobre los coeficientes

En R,

- `confint()` devuelve el intervalo de confianza del perfil de verosimilitud —el método de perfilado implementado en MASS (Venables & Ripley, 2002)—
- `confint.default()`, el de Wald.

Vemos que en GLOW, que se comporta bien, casi coinciden ambos intervalos:

```
list(perfil = confint(fit_glm),  
     wald   = confint.default(fit_glm)) |>  
map(\(m) round(m, 4))
```

```
$perfil  
          2.5 %  97.5 %  
(Intercept) -5.9006 -2.5711  
age          0.0174  0.0653  
priorfracYes 0.3776  1.2970
```

```
$wald  
          2.5 %  97.5 %  
(Intercept) -5.8760 -2.5526  
age          0.0173  0.0651  
priorfracYes 0.3799  1.2978
```

Inferencia sobre bloques: el contraste de razón de verosimilitudes (LRT)

Toda la inferencia que **compara modelos** se apoya en un mismo motor: el **contraste de razón de verosimilitudes** (*likelihood ratio test*, LRT). Mide **cuánto mejora la verosimilitud** al pasar de un modelo **pequeño** a otro **más grande que lo contiene** (anidado). El estadístico es

$$\Lambda = 2(\ell_{\text{grande}} - \ell_{\text{pequeño}}) = D_{\text{pequeño}} - D_{\text{grande}} \sim \chi^2_{\Delta_{\text{gl}}},$$

es decir, **la diferencia de deviances** —recuerda que $D = 2(\ell_{\text{sat}} - \ell_{\text{modelo}})$, así que el término del saturado se cancela—. Bajo H_0 (“el modelo pequeño basta; los parámetros extra no aportan”), Λ sigue una χ^2 con tantos grados de libertad como **parámetros añadidos**. Un Λ **grande** (p bajo) indica que esos parámetros **sí** explican señal real: nos quedamos con el modelo grande.

Es **un único contraste** que, según qué par anidado compares, responde a preguntas distintas:

- **pequeño = nulo, grande = ajustado** → «¿aporta algo el modelo?» (el **ómnibus**, justo abajo).
- **pequeño = reducido, grande = completo** → «¿aporta este bloque de variables?» (la escalera de modelos que recorreremos enseguida).
- **grande = saturado** → *bondad de ajuste*: la **deviance residual** es el LRT contra el ajuste perfecto ([Sección 2.6](#)).

Dos requisitos: los modelos han de estar **anidados** y ajustados sobre los **mismos datos**. Para especificaciones **no anidadas** no hay LRT y se recurre a **AIC/BIC** (más abajo).

El ómnibus —“¿aporta algo el modelo?”— es el caso *nulo vs ajustado*, análogo al **test F global** del modelo lineal:

$$D_0 - D \sim \chi_p^2.$$

Para valorar el aporte de *cada término ajustando por el resto*, se echa mano del **Anova** de tipo II:

```
# Wald en escala log-odds: aporte de cada término independientemente
tidy(fit_glm) # proporcionado por summary(fit_glm)
```

```
# A tibble: 3 x 5
  term      estimate std.error statistic    p.value
<chr>      <dbl>      <dbl>      <dbl>    <dbl>
1 (Intercept) -4.21        0.848      -4.97 0.000000667
2 age          0.0412     0.0122       3.38 0.000719
3 priorfracYes  0.839        0.234       3.58 0.000340
```

```
# Anova tipo II: aporte de cada término ajustando por el resto
car::Anova(fit_glm, type = "II")
```

Analysis of Deviance Table (Type II tests)

Response: fracture

	LR	Chisq	Df	Pr(>Chisq)
age	11.551	1	0.0006772	***
priorfrac	12.544	1	0.0003975	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```
# LRT ómnibus: aporte del bloque completo frente al modelo nulo
anova(glm(fracture ~ 1, binomial, glow), fit_glm, test = "LRT")
```

Analysis of Deviance Table

```
Model 1: fracture ~ 1
Model 2: fracture ~ age + priorfrac
  Resid. Df Resid. Dev Df Deviance  Pr(>Chi)
1      499      562.34
2      497      528.52  2    33.818 4.535e-08 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

💡 En R. Contrastes de razón de verosimilitudes (LRT)

El LRT compara dos modelos **anidados** (ajustados sobre los **mismos datos**) por su diferencia de deviancias, con `anova(..., test = "LRT")` (equivale a `test = "Chisq"` en `glm`):

```
# Ómnibus: ¿aporta algo el modelo frente al nulo (solo intercepto)?
anova(glm(fracture ~ 1, family = binomial, data = glow), fit_glm, test = "LRT")

# LRT entre dos modelos anidados cualesquiera (reducido vs completo)
anova(mod_reducido, mod_completo, test = "LRT")

# Aporte de CADA término ajustando por el resto (LRT por término)
car::Anova(fit_glm, type = "II")      # type = "III" si hay interacciones
```

`summary(fit_glm)` / `tidy(fit_glm)` dan el contraste **de Wald** por coeficiente (aproximado); el **LRT** suele ser más fiable. Recuerda: el LRT **solo** vale para modelos **anidados**.

Comparar modelos no anidados: AIC y BIC

El LRT solo vale para comparar modelos anidados. Para comparar especificaciones cualesquiera se usan criterios que penalizan la verosimilitud por complejidad, como el AIC y el BIC (Akaike, 1974; Schwarz, 1978), que tienen en cuenta el número k parámetros y n observaciones:

$$\text{AIC} = -2\ell + 2k, \quad \text{BIC} = -2\ell + k \log n,$$

Menor es mejor, si bien el **BIC penaliza más** y por tanto **favorece la parsimonia**. Es importante remarcar que son medidas **relativas**, útiles para ordenar modelos, no para concluir si alguno ajusta bien en términos absolutos.

💡 En R. AIC y BIC de un modelo

Sobre un modelo ajustado `fit_glm`:

```
AIC(fit_glm)           # AIC
BIC(fit_glm)           # BIC
AIC(m_1, m_2, m_3)     # tabla con los AIC de varios modelos (BIC(...) para el BIC)
glance(fit_glm)        # broom: AIC, BIC, deviance, logLik, gl... en una fila
```

Menor es mejor; recuerda que son **relativos** (sirven para ordenar, no para juzgar el ajuste absoluto).

La escalera de modelos.

Seleccionar el mejor modelo no es elegir entre dos concretos, sino **recorrer una escalera o jerarquía** de modelos, de menos a más compleja (o viceversa), decidiendo en cada peldaño si la complejidad añadida (o excluida) se paga “de modo suficiente” con ajuste. Para realizar esta selección el analista propone un criterio y procede; los más habituales son LRT, AIC y BIC. La decisión final es **del analista**, y debe argumentarse.

A la hora de ajustar un mejor modelo sobre GLOW, consideramos diversos modelos de menos a más complejos, con las variables `age`, `priorfrac`, `momfrac` (fractura de cadera en la madre) y `bmi` (índice de masa corporal), teniendo en cuenta posibles interacciones entre factores o entre factores y covariables (los dos factores son `priorfrac` y `momfrac`).

```
mod_0 <- glm(fracture ~ age + priorfrac, family = binomial, data = glow)
mod_1 <- glm(fracture ~ age + priorfrac + momfrac + bmi, family = binomial, data = glow)
mod_2 <- glm(fracture ~ age * priorfrac + bmi + momfrac, family = binomial, data = glow)
mod_3 <- glm(fracture ~ (age + bmi) * priorfrac + momfrac, family = binomial, data = glow)
mod_4 <- glm(fracture ~ (age + bmi) * (priorfrac + momfrac), family = binomial, data = glow)
tibble(modelo = paste0("mod_", 1:4)) |>
  bind_cols(map_dfr(list(mod_1, mod_2, mod_3, mod_4),
    \ (m) glance(m) |> dplyr::select(df = df.residual, deviance, AIC, BIC))
```

```
# A tibble: 4 x 5
  modelo    df deviance   AIC   BIC
  <chr>  <int>    <dbl> <dbl> <dbl>
1 mod_1    495    522.  532.  553.
```

2	mod_2	494	516.	528.	553.
3	mod_3	493	514.	528.	558.
4	mod_4	491	514.	532.	570.

Los efectos principales **bajan la deviance con claridad**; las interacciones, menos. Lo contrastamos peldaño a peldaño con LRT:

```
anova(mod_4, mod_3, test = "LRT")
```

Analysis of Deviance Table

```
Model 1: fracture ~ (age + bmi) * (priorfrac + momfrac)
Model 2: fracture ~ (age + bmi) * priorfrac + momfrac
  Resid. Df Resid. Dev Df Deviance Pr(>Chi)
1      491      513.80
2      493      514.11 -2  -0.31228  0.8554
```

```
anova(mod_3, mod_2, test = "LRT")
```

Analysis of Deviance Table

```
Model 1: fracture ~ (age + bmi) * priorfrac + momfrac
Model 2: fracture ~ age * priorfrac + bmi + momfrac
  Resid. Df Resid. Dev Df Deviance Pr(>Chi)
1      493      514.11
2      494      516.09 -1  -1.9787  0.1595
```

```
anova(mod_2, mod_1, test = "LRT")
```

Analysis of Deviance Table

```
Model 1: fracture ~ age * priorfrac + bmi + momfrac
Model 2: fracture ~ age + priorfrac + momfrac + bmi
  Resid. Df Resid. Dev Df Deviance Pr(>Chi)
1      494      516.09
2      495      521.87 -1  -5.7753  0.01625 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
anova(mod_1, mod_0, test = "LRT")
```

Analysis of Deviance Table

```
Model 1: fracture ~ age + priorfrac + momfrac + bmi
Model 2: fracture ~ age + priorfrac
  Resid. Df Resid. Dev Df Deviance Pr(>Chi)
1      495      521.87
2      497      528.52 -2   -6.6485   0.036 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

El primer contraste contesta si son significativas (en bloque) las interacciones de edad y bmi con momfrac, y la respuesta es que no (p-valor 0.8554), por lo que cabe descartarlas para simplificar el modelo. La siguiente tabla Anova resuelve si tiene sentido la interacción de bmi con priorfrac, y la respuesta es no (p-valor 0.1595). A continuación se comparan los modelos mod_2 y mod_1, esto es, si es significativa la interacción age:priorfrac y el resultado es sí, con un p-valor de 0.0162. Llegado a este punto, pararíamos y nos quedaríamos con el mod_2. La última comparación valora si merece la pena incluir las variables momfrac y bmi en el modelo, y la respuesta es nuevamente que sí (p-valor 0.036), pues amplían la información que dan exclusivamente age y priorfrac.

Interpretar una interacción exige cuidado: el coeficiente de priorfrac ya **no** es “el efecto de la fractura previa”, sino su efecto **a edad 0** (sin sentido sin centrar). Por eso, con interacciones, la lectura se hace en **efectos marginales o predicciones** (ya vista en la [Sección 2.3](#)), no en coeficientes crudos:

```
tidy(mod_2)
```

```
# A tibble: 6 x 5
  term          estimate std.error statistic    p.value
<chr>          <dbl>     <dbl>     <dbl>    <dbl>
1 (Intercept)   -6.84        1.33      -5.15 0.000000263
2 age           0.0678       0.0159      4.25 0.0000214
3 priorfracYes   5.21        1.84       2.83 0.00464
4 bmi           0.0248       0.0189      1.31 0.189
5 momfracYes     0.748       0.299      2.50 0.0125
6 age:priorfracYes -0.0612     0.0254     -2.40 0.0162
```


💡 En R. Automatizar la escalera: **dredge** (exhaustiva) y **stepAIC** (paso a paso)

Con varias variables e interacciones el número de modelos crece deprisa. Dos funciones lo automatizan, con filosofías opuestas; ambas parten de un **modelo global** ajustado con `na.action = na.fail` (requisito de **dredge**):

```
global <- glm(fracture ~ (age + bmi) * (priorfrac + momfrac),
              family = binomial, data = glow, na.action = na.fail)
```

MuMIn::dredge — **exhaustiva** (Bartoń, 2024): ajusta **todos** los submodelos (respetando la marginalidad) y los ordena por el criterio elegido:

```
sel <- dredge(global, rank = "AIC")           # rank = "BIC" para favorecer la parsimonia
head(sel, 3)                                # mejores modelos
mejor <- get.models(sel, subset = 1)[[1]]     # el modelo ganador, ya ajustado
formula(mejor)                              # solo su fórmula
```

MASS::stepAIC — **voraz / paso a paso** (Venables & Ripley, 2002): añade o quita un término cada vez siguiendo el AIC, sin recorrer todo el espacio (más rápida, pero puede quedarse en un óptimo local):

```
m_step <- MASS::stepAIC(global, direction = "both", trace = FALSE) # 'both'/'forward'/'backward'
formula(m_step)                                                    # solo su fórmula
```

dredge recorre **todo** el espacio; **stepAIC**, **un solo camino**: si coinciden, refuerza la elección; si discrepan, hay varios modelos casi indistinguibles.

Las comparamos entre sí:

```
#library(MuMIn)
global <- glm(fracture ~ (age + bmi) * (priorfrac + momfrac),
              family = binomial, data = glow, na.action = na.fail) # na.fail: requisito d

sel <- dredge(global, rank = "AIC")           # todos los submodelos, ordenados por AIC (respe
head(sel, 3)                                # los cinco mejores
```

```
Global model call: glm(formula = fracture ~ (age + bmi) * (priorfrac + momfrac),
                      family = binomial, data = glow, na.action = na.fail)
---
```

Model selection table

(Int)	age	bmi	mmf	prr	age:prr	bmi:prr	df	logLik	AIC	delta
-------	-----	-----	-----	-----	---------	---------	----	--------	-----	-------

```

46 -5.896 0.06400          + + +          5 -258.900 527.8 0.00
48 -6.845 0.06777 0.024850 + + +          6 -258.047 528.1 0.29
176 -6.079 0.06475 0.004593 + + +          + 7 -257.057 528.1 0.31
weight
46 0.368
48 0.318
176 0.314
Models ranked by AIC(x)

```

```
dredge(global, rank = "BIC") |> head(3) # el BIC, más parsimonioso, suele elegir otro
```

```

Global model call: glm(formula = fracture ~ (age + bmi) * (priorfrac + momfrac),
  family = binomial, data = glow, na.action = na.fail)
---
Model selection table
      (Int)      age mmf prr age:prr df   logLik   BIC delta weight
10 -4.214 0.04119      +          3 -264.259 547.2  0.00  0.485
42 -5.689 0.06251      +          + 4 -261.636 548.1  0.97  0.299
14 -4.277 0.04075  +   +          4 -261.963 548.8  1.62  0.216
Models ranked by BIC(x)

```

```

# Mejor por AIC (exhaustivo) frente a la búsqueda paso a paso:
formula(get.models(sel, subset = 1)[[1]]) # ganador de dredge

```

```
fracture ~ age + momfrac + priorfrac + age:priorfrac + 1
```

```
formula(MASS::stepAIC(global, direction = "both", trace = FALSE)) # ganador de stepAIC
```

```
fracture ~ age + priorfrac + momfrac + age:priorfrac
```

En este caso, ambos métodos han llegado al mismo óptimo.

💡 En R. Generar modelos anidados con MuMIn (dredge)

En cualquier caso, cabe tener como referencia que a partir de un modelo más complejo, podemos extraer todos los posibles modelos (anidados) viables con la librería MuMIn, para compararlos por cualquier criterio.

```
library(MuMIn)
# Definimos el modelo más complejo de partida
global <- glm(fracture ~ (age + bmi) * (priorfrac + momfrac) + priorfrac:momfrac,
              family = binomial, data = glow, na.action = na.fail) # na.fail: requisito de

calls <- dredge(global, evaluate = FALSE)      # NO ajusta: solo genera las llamadas/fórmulas
length(calls)                                # cuántos modelos hay
calls[[10]]                                  # una llamada concreta
formulas = lapply(calls, function(cl) formula(eval(cl))) # extrae las fórmulas, si las quiere
formulas[[10]]
```

Conviene leer las tres herramientas **juntas**, no como árbitros que se sustituyan: el **LRT** dice qué términos aportan señal real (aquí, **age:priorfrac** sí; **priorfrac:momfrac** no); **dredge** y **stepAIC**, al optimizar AIC, tienden a un modelo de buena predicción, pero pueden discrepar entre sí —**dredge** ve todo el espacio; **step**, solo un camino— y, sobre todo, del **BIC**, más parsimonioso. Que dos algoritmos converjan o no es ya información: si discrepan, varios modelos son casi indistinguibles y la elección no debe delegarse al criterio.

⚠ Sobre la selección automática (**step**, **dredge**)

La búsqueda automática —paso a paso (**MASS::stepAIC**) o exhaustiva (**MuMIn::dredge**)— es útil como **exploración**, pero no como árbitro. Ambas inválidan los *p*-valores posteriores (inferencia **post-selección**) y favorecen el sobreajuste; la exhaustiva, además, **agrava la multiplicidad** al comparar muchísimos modelos. En este curso las usamos para mapear el espacio, y la decisión final se justifica con LRT/AIC/BIC, validación y conocimiento del problema (Harrell, 2015).

💡 Protocolo de inferencia y selección

1. **¿Aporta algo el modelo?** Contraste ómnibus (LRT nulo vs completo).
2. **¿Aporta cada término?** LRT / Anova tipo II por término, con Wald e IC (mejor por perfil) como lectura individual.
3. **¿Qué especificación?** LRT entre anidados; AIC/BIC entre cualesquiera (AIC → predicción, BIC → parsimonia).
4. **Selección de variables.** Recorre la escalera **guiada por el problema**, no a ciegas; combina LRT/AIC/BIC; evita la búsqueda automática como árbitro; **valida y argumenta** la elección.

2.6 Bondad de ajuste y diagnóstico

En 2.5 preguntamos si el modelo **aporta** y **cuál** elegir. Ahora la pregunta es otra: el modelo elegido, **¿ajusta bien los datos?** Es la **bondad de ajuste**, y la bibliografía la trata de la mano del **análisis de residuos** (Agresti, 2015; Faraway, 2016; Hosmer et al., 2013), porque ambos responden a lo mismo desde dos ángulos: un **contraste global** y una **mirada caso a caso**.

El contraste global: ¿aportar o ajustar? (y por qué falla en binaria)

Conviene situar tres modelos en una **escalera**, del peor al mejor ajuste posible: el **nulo** (solo intercepto, deviance D_0); el **ajustado**, el nuestro (deviance residual D); y el **saturado**, con un parámetro por observación, que **ajusta los datos a la perfección** (deviance = 0). Con ellos se responden **dos preguntas distintas que no hay que mezclar**:

- **¿Aporta el modelo?** (utilidad, la de la [Sección 2.5](#)). Se compara **ajustado vs nulo**: $D_0 - D \sim \chi_p^2$. Un p bajo **rechaza el nulo** a favor del nuestro —las covariables explican algo—. Es la pregunta «¿mejor que nada?».
- **¿Ajusta bien?** (bondad de ajuste, la de aquí). Se compara **ajustado vs saturado**: la propia **deviance residual** $D = 2(\ell_{\text{sat}} - \ell_{\text{ajustado}})$ es el LRT frente al mejor ajuste posible y, bajo un modelo correcto, se distribuiría χ_{n-p}^2 . Mide «¿cuánto queda sin explicar respecto a un ajuste perfecto?»; un p bajo señala **falta de ajuste**.

¿Por qué el **saturado** y no el nulo es la referencia de la *bondad de ajuste*? Porque “ajustar bien” es **acercarse al mejor ajuste alcanzable**, no solo superar a “no ajustar nada”: un modelo puede **batir al nulo** y, a la vez, quedar **lejos del saturado** (forma funcional pobre, enlace equivocado, no linealidad ignorada). Aportaría, pero no ajustaría. Por eso son contrastes **complementarios**, no intercambiables.

Y aquí está el problema en binaria. El χ_{n-p}^2 exige un régimen asintótico concreto: que el **número de celdas** —los parámetros del saturado— quede **fijo** mientras la **información dentro de cada celda crece**. Con datos **agrupados** (binomial, m_i ensayos por celda) se cumple: hay un número fijo de celdas y, al aumentar m_i , cada proporción $\hat{\pi}_i$ se estima con precisión. Con respuesta **binaria individual** ($m_i = 1$, una fila por sujeto) ocurre lo contrario: cada “celda” es **una sola** observación, así que al crecer n **crece también el saturado** —un parámetro por dato, estimado siempre con un único 0/1—; nunca se acumula información por parámetro, D **no** sigue χ_{n-p}^2 y el contraste **carece de valor**. De hecho, en GLOW $\ell_{\text{sat}} = 0$ y $D = -2\ell(\hat{\beta})$ depende **solo** de las probabilidades ajustadas, sin informar del ajuste absoluto (Agresti, 2015; Hosmer et al., 2013).

Cómo se resuelve, entonces. Dos vías: (i) **agrupar** —si las covariables son categóricas, se pasa a binomial: tanto D como el **estadístico de Pearson** X^2 **recuperan su sentido** como contraste χ^2 , y allí los definimos con detalle ([Sección 3.2](#))—; (ii) cuando no se puede

(covariables continuas, como en GLOW), se sustituye el contraste global por herramientas que **agrupan implícitamente** o **simulan**: el **test de Hosmer–Lemeshow** (deciles de riesgo), la **curva de calibración** y los **residuos por tramos**, que vemos a continuación.

Residuos

Para diagnosticar un GLM **observación a observación** se examinan los **residuos**. Los estándares de un GLM son los de **Pearson** y los de **deviance** (McCullagh & Nelder, 1989); son informativos cuando hay **suficiente información por observación** —datos agrupados, conteos altos—, y los desarrollamos en propiedad (fórmulas, gráfico y variantes —estandarizados, leverage, distancia de Cook—) en la [Sección 3.2](#), donde **sí** funcionan.

El problema en binaria. Con respuesta **binaria individual**, $y_i \in \{0, 1\}$: para cada $\hat{\mu}_i$ el residuo crudo solo toma dos valores y se ordena en **dos bandas**, y ni Pearson ni deviance llegan a ser $\approx N(0, 1)$. Hacen falta herramientas que **rescaten** el diagnóstico, y conviene elegir las **según para qué**:

- **Diagnóstico general** (mala especificación de cualquier tipo: forma funcional, dispersión, *outliers*) → los **residuos simulados / de cuantil** (DHARMa) (Dunn & Smyth, 1996; Hartig, 2024). Se simula muchas veces del modelo ajustado y se sitúa cada observación en su distribución simulada; bajo un modelo correcto son **uniformes** $(0, 1)$, comparables con un Q–Q y un test de Kolmogorov–Smirnov. No son “de binaria”: valen para **cualquier** GLM —son, simplemente, el rescate cuando los estándares fallan—.
- **Diagnóstico de calibración** (¿son fiables las probabilidades, y en qué tramo fallan?) → los **residuos por tramos** (*binned*), que promedian observado – predicho por tramos de riesgo. Como responden a la pregunta de la calibración, los tratamos **donde se usan**: en la subsección siguiente.



En R. Residuos simulados (DHARMa)

```
sim <- DHARMa::simulateResiduals(fit_glm) # plot = TRUE por defecto
plot(sim)                                # Q-Q (con test KS) + residuos vs predicción
DHARMa::testResiduals(sim)                # tests formales: uniformidad, dispersión, o
```

Los de Pearson y deviance, con `residuals(fit_glm, type = "pearson" | "deviance")`.

```
sim <- DHARMa::simulateResiduals(fit_glm, plot = FALSE)
plot(sim)
```

DHARMa residual

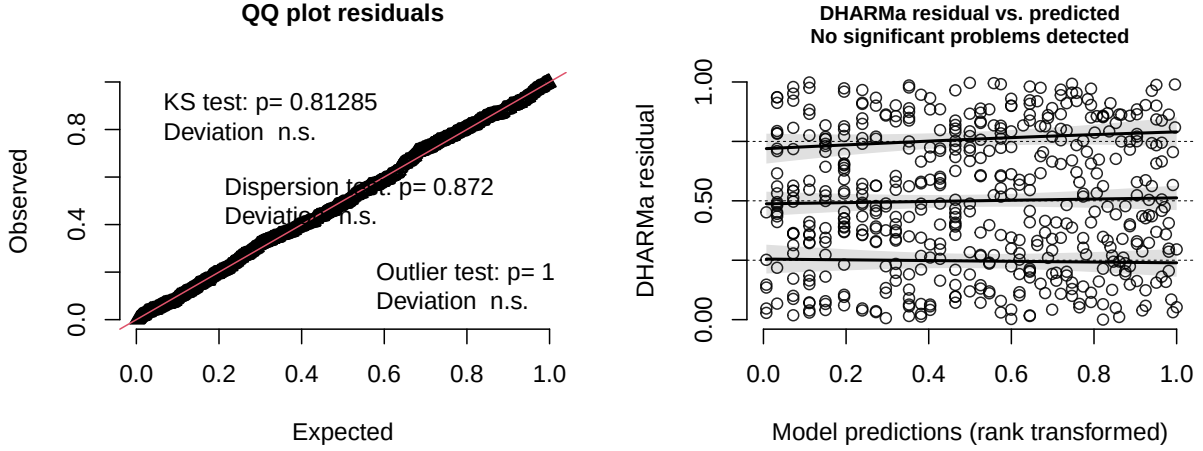


Figura 16: Residuos simulados (DHARMa) del modelo logístico en GLOW. Izquierda: Q–Q de los residuos, que deberían seguir la diagonal uniforme. Derecha: residuos frente a predicción, sin patrón si el ajuste es adecuado.

En GLOW, los residuos simulados **no muestran ningún problema**: el Q–Q sigue la diagonal uniforme (test de Kolmogorov–Smirnov $p \approx 0,81$) y no hay señal de **sobredispersión** ($p \approx 0,87$) ni de **outliers** ($p \approx 1$); a la derecha, los residuos se reparten **sin patrón** frente a la predicción. Ninguna evidencia, por tanto, de mala especificación —ni en la forma, ni en la dispersión, ni en casos atípicos—.

Calibración: ¿son fiables las probabilidades?

Confronta la probabilidad **predicha** con la frecuencia **observada**. El contraste de referencia es el **test de Hosmer–Lemeshow** (HL) (Hosmer & Lemeshow, 1980): se ordenan las n observaciones por su probabilidad predicha $\hat{\pi}_i$ y se reparten en G grupos de igual tamaño (habitualmente $G = 10$, deciles de riesgo). En cada grupo g se cuentan los eventos **observados** $O_g = \sum_{i \in g} y_i$ y los **esperados** $E_g = \sum_{i \in g} \hat{\pi}_i = n_g \bar{\pi}_g$ (con $\bar{\pi}_g$ la probabilidad media predicha del grupo), y el estadístico los confronta:

$$\hat{C} = \sum_{g=1}^G \frac{(O_g - E_g)^2}{n_g \bar{\pi}_g (1 - \bar{\pi}_g)}.$$

Bajo H_0 («el modelo calibra bien»), $\hat{C}$ se distribuye aproximadamente χ^2_{G-2} —los $G-2$ grados de libertad, establecidos por simulación, descuentan dos por el agrupamiento y la estimación—. Un $\hat{C}$ **grande** (p bajo) indica que observado y esperado **discrepan** → mala calibración; un p **alto** señala **ausencia de evidencia** de mala calibración (no «prueba» que calibre bien).

```
performance::performance_hosmer(fit_glm, n_bins = 10)
```

```
# Hosmer-Lemeshow Goodness-of-Fit Test
```

```
Chi-squared: 10.816
      df:    8
p-value:    0.212
```

En GLOW, $p \approx 0,21$: sin evidencia de mala calibración. Pero el HL tiene **limitaciones conocidas** —sensible al número de grupos y poco potente—, así que la práctica moderna lo **acompaña de un gráfico de calibración** y de los residuos por tramos (Agresti, 2015; Harrell, 2015):

```
diag <- tibble(p_hat = fitted(fit_glm), y = glow$fractura01)

# --- Panel A: residuos por tramos (estilo binnedplot, en ggplot) ---
binned <- diag |>
  mutate(tramo = ntile(p_hat, 20)) |>
  group_by(tramo) |>
  summarise(p_med = mean(p_hat),
            r_med = mean(y - p_hat),
            se      = sqrt(mean(p_hat * (1 - p_hat)) / n()), .groups = "drop")

pA <- ggplot(binned, aes(p_med, r_med)) +
  geom_ribbon(aes(ymin = -2 * se, ymax = 2 * se), alpha = 0.15) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_point(size = 1.8) +
  labs(x = "Probabilidad ajustada", y = "Residuo medio por tramo",
       title = "Residuos por tramos")

# --- Panel B: calibración por deciles ---
calib <- diag |>
  mutate(decil = ntile(p_hat, 10)) |>
  group_by(decil) |>
  summarise(pred = mean(p_hat), obs = mean(y), .groups = "drop")

pB <- ggplot(calib, aes(pred, obs)) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed") + # calibración perfecta
  geom_line() + geom_point(size = 1.8) +
  coord_equal(xlim = c(0, 1), ylim = c(0, 1)) +
  labs(x = "Riesgo predicho (media por decil)", y = "Frecuencia observada",
```

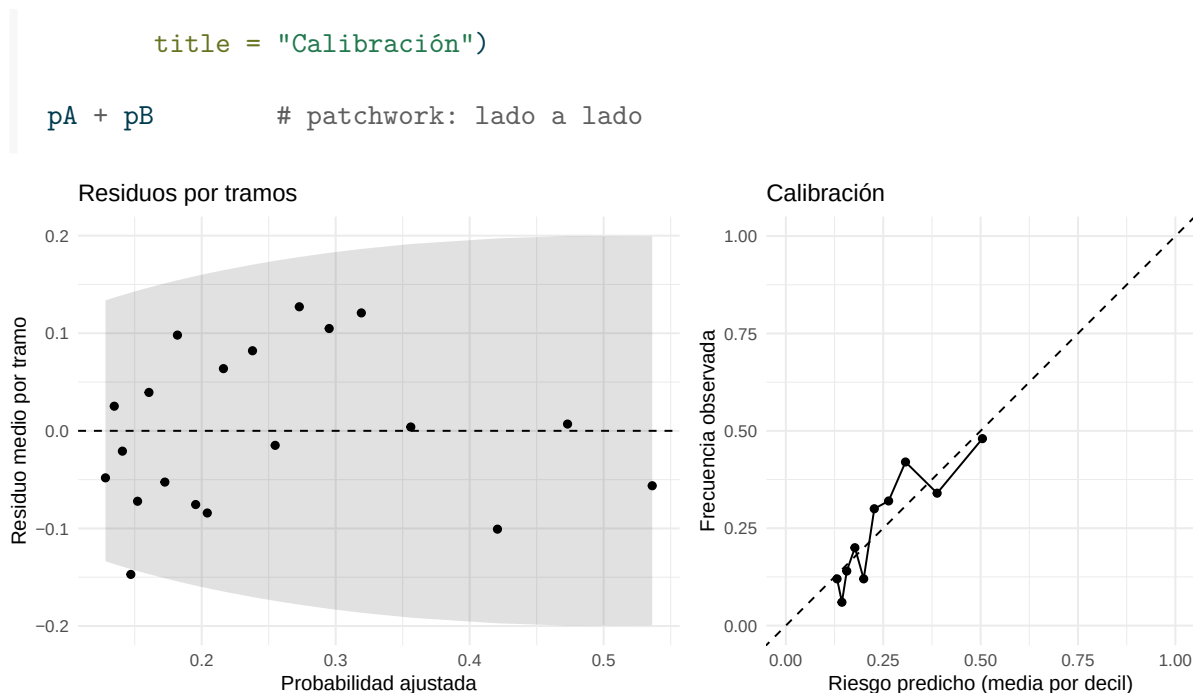


Figura 17: Diagnóstico visual. Izquierda: residuos por tramos (deberían caer dentro de las bandas de ± 2 EE). Derecha: curva de calibración por deciles de riesgo (debería seguir la diagonal).

Ambos paneles convierten el “pasa / no pasa” del HL en un **diagnóstico localizado**: dicen no solo *si* hay problema, sino *dónde*, *en qué sentido* y *qué* podría corregirlo.

💡 Cómo leer los dos gráficos

Curva de calibración (probabilidad predicha en X, frecuencia observada en Y; la referencia es la diagonal de 45°):

- **Sobre la diagonal** $\rightarrow$ bien calibrado: entre los casos con riesgo predicho p , una fracción $\approx p$ tiene el evento.
- **Por debajo de la diagonal** (sobre todo en riesgo alto) $\rightarrow$ el modelo **sobreestima** el riesgo en esa zona.
- **Por encima de la diagonal** (sobre todo en riesgo bajo) $\rightarrow$ **subestima**.
- **Forma en S** alrededor de la diagonal $\rightarrow$ exceso o defecto de confianza en los extremos.
- Lo informativo es **en qué tramo** de riesgo aparece la desviación, no solo que exista.

Residuos por tramos (residuo medio por tramo en Y, probabilidad media en X, con bandas de ± 2 EE $\approx 95\%$):

- **Dispersos en torno a 0 y dentro de las bandas** → sin falta de ajuste sistemática.
- **Patrón sistemático** (p. ej. negativos en riesgo bajo y positivos en alto, o una curva) → el modelo **yerra la media** en ese rango.
- **Más de ~5 % de puntos fuera de las bandas** → señal de mal ajuste.
- Si se representa el residuo medio por tramos frente a una **covariable** (la edad, por ejemplo) en lugar de frente a la probabilidad ajustada, un patrón indica que **esa variable pide un término no lineal** (un cuadrático) o una interacción: orienta sobre **qué** cambiar.

En GLOW (Figura 17), coherente con el HL ($p \approx 0,21$): los residuos por tramos quedan dentro de las bandas y la calibración se ajusta a la diagonal, sin señales de descalibración ni de falta de ajuste en el rango observado.

💡 En R. Hosmer–Lemeshow y curva de calibración

Test de Hosmer–Lemeshow:

```
performance::performance_hosmer(fit_glm, n_bins = 10)
ResourceSelection::hoslem.test(glow$fractura01, fitted(fit_glm), g = 10) # alternativa
```

Residuos por tramos (binned):

```
plot(performance::binned_residuals(fit_glm)) # observado - predicho promediado por tra
```

Curva de calibración (riesgo predicho vs frecuencia observada, por deciles):

```
tibble(p = fitted(fit_glm), y = glow$fractura01) |>
  mutate(grupo = ntile(p, 10)) |>
  group_by(grupo) |> summarise(pred = mean(p), obs = mean(y)) |>
  ggplot(aes(pred, obs)) +
  geom_abline(linetype = "dashed") + geom_point() + geom_line()
```

(Curva suavizada e índices: `rms::val.prob()`, `performance::check_model()`.)

Separación: cuando el ajuste “perfecto” rompe la inferencia.

Es el patológico que vuelve inútil al Wald. Hablamos de “separación” cuando una combinación de covariables **predice perfectamente** el resultado en algún subgrupo: la verosimilitud sigue creciendo al alejar el coeficiente hacia $\pm\infty$, así que `glm()` devuelve estimaciones **enormes con errores estándar gigantescos** (Albert & Anderson, 1984).

GLOW lo exhibe de forma natural: entre las fumadoras que autoperciben su riesgo como «menor» (smoke = Yes, raterisk = Less) **no hay ninguna fractura** (0 de 12), lo que separa ese grupo.

```
glow |> count(smoke, raterisk, fracture) |>
  filter(smoke == "Yes", raterisk == "Less") # celda con 0 fracturas
```

```
# A tibble: 1 x 4
  smoke raterisk fracture      n
  <fct> <fct>    <fct>    <int>
1 Yes   Less     No         12
```

```
fit_sep <- glm(fracture ~ smoke * raterisk, binomial, glow)
tidy(fit_sep) |> filter(str_detect(term, "smoke")) # coef enorme, EE descomunal
```

```
# A tibble: 3 x 5
  term                estimate std.error statistic p.value
  <chr>                <dbl>    <dbl>    <dbl>    <dbl>
1 smokeYes            -15.1      693.    -0.0217  0.983
2 smokeYes:rateriskSame  14.5      693.     0.0209  0.983
3 smokeYes:rateriskGreater 15.6      693.     0.0225  0.982
```

El coeficiente de smokeYes ronda **−15 con un EE de varios cientos**: síntoma inequívoco de cuasi-separación, no un efecto «protector» real. El Wald carece de sentido aquí.

La solución es detectarla (detectseparation::detect_separation) y, si aparece, estimar por **verosimilitud penalizada de Firth** (Firth, 1993; Heinze & Schemper, 2002), que produce estimaciones finitas:

```
# install.packages("logistf") # o brglm2::brglm_fit
# La verosimilitud penalizada de Firth devuelve estimaciones FINITAS e inferencia con sent
logistf::logistf(fracture ~ smoke * raterisk, data = glow)
```

```
logistf::logistf(formula = fracture ~ smoke * raterisk, data = glow)
```

Model fitted by Penalized ML

Confidence intervals and p-values by Profile Likelihood

Coefficients:

(Intercept)	smokeYes	rateriskSame
-------------	----------	--------------

-1.4982123	-1.7206635	0.4817357
rateriskGreater	smokeYes:rateriskSame	smokeYes:rateriskGreater
0.7664928	1.3020556	2.2853289

Likelihood ratio test=15.12048 on 5 df, p=0.009859754, n=500

Se ve inmediatamente la corrección: el término de **smoke** pasa de ≈ -15 con EE de cientos y $p \approx 1$ (Wald, roto) a una estimación **finita**, con IC y p razonables (Firth).



En R. Detectar la separación y estimar por verosimilitud penalizada (Firth)

Detección (antes de fiarte de un **glm** con coeficientes enormes y EE descomunales; también avisa el warning “*fitted probabilities numerically 0 or 1 occurred*”):

```
detectseparation::detect_separation(fracture ~ smoke * raterisk,
                                   family = binomial, data = glow) # marca los términos
```

Estimación penalizada de Firth —penaliza la verosimilitud (corrección de Jeffreys), de modo que los coeficientes quedan **finitos** incluso bajo separación, con inferencia por verosimilitud de perfil—:

```
logistf::logistf(fracture ~ smoke * raterisk, data = glow) # coef, IC y p fin
glm(fracture ~ smoke * raterisk, family = binomial, data = glow,
    method = brglm2::brglm_fit) # alternativa (brglm2)
```

2.7 Evaluación como clasificador

El modelo entrega **probabilidades**; convertirlo en un **clasificador** (predecir sí/no) exige un **umbral** c y abre un tercer eje de evaluación del modelo: la **discriminación** —¿separa eventos de no-eventos?—. Este enfoque de evaluación es distinta al que se realiza sobre la bondad del ajuste (en la [Sección 2.6](#) y sobre la utilidad inferencial (en la [Sección 2.5](#)).

Del umbral a la matriz de confusión.

Fijado un umbral $c \in [0, 1]$, por defecto $c = 0,5$, clasificamos con una observación en función de su predicción $\hat{\pi}_i$ con:

$$\hat{y}_i = \mathbf{1}[\hat{\pi}_i \geq c]$$

Para evaluar la calidad de la clasificación, tenemos fácil la comparación entre lo observado (los datos iniciales 0/1) y lo predicho al clasificar. El cruce de lo predicho con lo observado define la **matriz de confusión**, punto de partida de todas las métricas.

Las cuatro celdas.

Con un umbral dado c , cada caso cae en una de cuatro celdas según coincidan predicción y realidad: **verdadero positivo** (TP, *true positive*), **falso positivo** (FP, *false positive*), **falso negativo** (FN, *false negative*) y **verdadero negativo** (TN, *true negative*). De ahí salen las métricas más habituales (nombradas en inglés porque es así como las nombra casi todo el software):

- **Exactitud** — *accuracy*: $\frac{TP + TN}{\text{total}}$. Proporción global de aciertos.
- **Sensibilidad** o *recall* — *sensitivity / true positive rate*: $\frac{TP}{TP + FN}$. De los eventos reales, cuántos detecta.
- **Especificidad** — *specificity / true negative rate*: $\frac{TN}{TN + FP}$. De los no-eventos reales, cuántos descarta.
- **Precisión** o valor predictivo positivo — *precision / positive predictive value (PPV)*: $\frac{TP}{TP + FP}$. De los que predice como eventos, cuántos lo son de verdad.

Sensibilidad y precisión suelen estar **en tensión** (bajar el umbral detecta más eventos pero genera más falsas alarmas); cuando ambas importan se utiliza el

- **F1** (*F1-score*), la media armónica de sensibilidad y precisión, $2 \frac{\text{precisión} \cdot \text{recall}}{\text{precisión} + \text{recall}}$.

Cuál mirar.

A la hora de valorar la calidad de la clasificación, cabe tener en cuenta estas consideraciones:

- La **exactitud engaña con clases desbalanceadas**: si una clase es rara, “clasificar todo como la mayoritaria” logra una exactitud alta y es inútil. Conviene entonces mirar **sensibilidad, especificidad y precisión por separado** y elegir según el **coste de cada error**: la sensibilidad prima cuando un falso negativo es caro (no detectar un evento); la precisión, cuando lo caro es la falsa alarma.
- Además, sensibilidad y especificidad son **invariantes a la prevalencia** (son propiedades del clasificador dada la clase real), mientras que exactitud y precisión **sí dependen** de ella; por eso, y para resumir todos los umbrales a la vez, conviene complementarlas con el **AUC**, también invariante a la prevalencia.

Calculamos a mano la matriz de confusión para GLOW.

```
# --- A mano (para ver qué es cada cosa) ---
umbral <- 0.5
pred <- factor(ifelse(fitted(fit_glm) >= umbral, "Yes", "No"),
               levels = c("No", "Yes"))
cm <- table(Predicho = pred, Observado = glow$fracture)
cm
```

	Observado	
Predicho	No	Yes
No	360	111
Yes	15	14

```
TP <- cm["Yes", "Yes"]; FP <- cm["Yes", "No"]
FN <- cm["No", "Yes"]; TN <- cm["No", "No"]
c(exactitud      = (TP + TN) / sum(cm),      # accuracy
  sensibilidad    = TP / (TP + FN),          # sensitivity / recall
  especificidad   = TN / (TN + FP),          # specificity
  precision       = TP / (TP + FP)) |> round(3) # precision / PPV
```

exactitud	sensibilidad	especificidad	precision
0.748	0.112	0.960	0.483

En la práctica no se calcula a mano: `yardstick` (Kuhn et al., 2026) devuelve la matriz y todas las métricas en una llamada. El único cuidado es declarar la **clase positiva**; como aquí **Yes** es el **segundo** nivel del factor, se indica con `event_level = "second"` (si no, sensibilidad y especificidad saldrían intercambiadas):

```
library(yardstick)
eval_df <- tibble(obs = glow$fracture, pred = pred)

conf_mat(eval_df, truth = obs, estimate = pred) # misma matriz de confusión
```

	Truth	
Prediction	No	Yes
No	360	111
Yes	15	14

```
metric_set(accuracy, sensitivity, specificity, precision, recall, f_meas)(
  eval_df, truth = obs, estimate = pred, event_level = "second")
```

```
# A tibble: 6 x 3
  .metric      .estimator .estimate
  <chr>        <chr>      <dbl>
1 accuracy    binary      0.748
2 sensitivity  binary      0.112
3 specificity  binary      0.96
4 precision   binary      0.483
5 recall      binary      0.112
6 f_meas      binary      0.182
```

💡 En R. Matriz de confusión y métricas

A partir de la respuesta observada y la predicha (factores con los **mismos niveles**), `yardstick` da todo en una llamada; el único cuidado es declarar la **clase positiva** con `event_level`:

```
library(yardstick)
pred    <- factor(ifelse(fitted(fit_glm) >= 0.5, "Yes", "No"), levels = c("No", "Yes"))
eval_df <- tibble(obs = glow$fracture, pred = pred)

conf_mat(eval_df, truth = obs, estimate = pred) # matriz de confus
metric_set(accuracy, sensitivity, specificity, precision, recall, f_meas)(
  eval_df, truth = obs, estimate = pred, event_level = "second") # 'second' = posit
```

Alternativa: `caret::confusionMatrix(pred, obs, positive = "Yes")`.

ROC y AUC

Todas las métricas anteriores dependen de un **umbral fijo** para clasificar a los sujetos en Yes/No ($c = 0,5$ por defecto): cambiar el corte cambia sensibilidad, especificidad y precisión. Para evaluar el clasificador **con independencia del umbral** —y, como acabamos de ver, de la prevalencia— se recorre el corte de 0 a 1 y se resume el compromiso entre acertar eventos y evitar falsas alarmas. Eso es la curva **ROC** y su área, el **AUC** (Robin et al., 2011).

La **curva ROC** (*receiver operating characteristic*) recorre **todos los umbrales** $c \in [0, 1]$ y, para cada uno, dibuja la **sensibilidad** (tasa de verdaderos positivos, TPR) frente a **1 — especificidad** (tasa de falsos positivos, FPR):

$$\text{TPR}(c) = \frac{TP(c)}{TP(c) + FN(c)} = \Pr(\hat{\pi} \geq c \mid Y = 1), \quad (1)$$

$$\text{FPR}(c) = \frac{FP(c)}{FP(c) + TN(c)} = \Pr(\hat{\pi} \geq c \mid Y = 0). \quad (2)$$

La curva es el conjunto de puntos $\{(\text{FPR}(c), \text{TPR}(c)) : c \in [0, 1]\}$; cada punto es un clasificador posible. La **diagonal** es el azar ($\text{TPR} = \text{FPR}$), y cuanto más se arrima la curva a la **esquina superior izquierda**, mejor separa (ver Figura 18).

El **AUC** (*area under the curve*) es el área bajo esa curva,

$$\text{AUC} = \int_0^1 \text{TPR} d(\text{FPR}),$$

con 0,5 = azar y 1 = perfecto. Tiene una lectura **probabilística** elegante (equivale al estadístico de Mann–Whitney): es la **probabilidad de que el modelo asigne más riesgo a un evento que a un no-evento** tomados al azar,

$$\text{AUC} = \Pr(\hat{\pi}_i > \hat{\pi}_j \mid Y_i = 1, Y_j = 0),$$

que se estima como la **proporción de pares (evento, no-evento) bien ordenados** (los empates cuentan media). Al promediar sobre todos los umbrales, mide la **discriminación** con independencia del corte y de la prevalencia.

```
roc_obj <- pROC::roc(glow$fracture, fitted(fit_glm), quiet = TRUE)
pROC::auc(roc_obj)
```

Area under the curve: 0.6807

```
pROC::ggroc(roc_obj) +
  geom_abline(slope = 1, intercept = 1, linetype = "dashed") +
  labs(x = "Especificidad", y = "Sensibilidad")
```

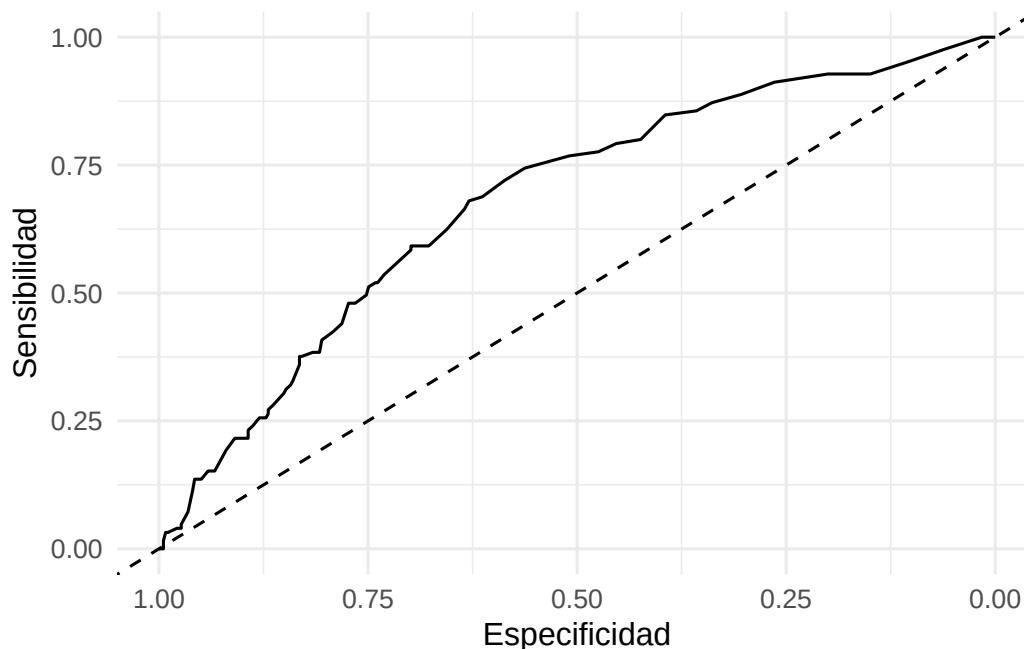


Figura 18: Curva ROC del modelo logístico en GLOW. El AUC resume la capacidad de discriminación agregada sobre todos los umbrales.

El AUC **0,68**: discriminación **modesta**, coherente con que la fractura depende de muchos factores no incluidos. Un AUC alto no garantiza buenas probabilidades: eso es **calibración**, ya vista en la [Sección 2.6](#).

Elegir el umbral es una decisión, no un dato.

El 0,5 por defecto rara vez es óptimo: depende del **coste relativo** de un falso positivo frente a un falso negativo. Un criterio habitual y **simétrico** (trata ambos errores por igual) es el **índice de Youden**,

$$J(c) = \text{sensibilidad}(c) + \text{especificidad}(c) - 1 = \text{TPR}(c) - \text{FPR}(c),$$

con $J \in [0, 1]$ (0 = no discrimina, sobre la diagonal; 1 = perfecto). El umbral elegido es el que **maximiza** J ,

$$c^* = \arg \max_c [\text{TPR}(c) - \text{FPR}(c)],$$

que geométricamente es el punto de la curva ROC **más alejado verticalmente de la diagonal**. Cuando los costes de los dos errores **no** son simétricos, Youden deja de ser apropiado y se elige el umbral que **minimiza el coste esperado** (ponderando FP y FN por su coste).

```
pROC::coords(roc_obj, "best", best.method = "youden",
  ret = c("threshold", "sensitivity", "specificity"))
```



```
threshold sensitivity specificity
1 0.2265844          0.68    0.6293333
```

💡 En R. Curva ROC, AUC y umbral de Youden

Con `pROC`, a partir de la respuesta observada y las probabilidades predichas:

```
library(pROC)
roc_obj <- roc(glow$fracture, fitted(fit_glm)) # (respuesta, predictor)
auc(roc_obj)                                # AUC ; ci.auc(roc_obj) para su IC 95%
ggroc(roc_obj)                               # la curva ROC (ggplot)

coords(roc_obj, "best", best.method = "youden", # umbral óptimo de Youden
       ret = c("threshold", "sensitivity", "specificity"))
```

Alternativa con `yardstick`: `roc_auc(eval_df, truth, prob, event_level = "second")` y `roc_curve(...)`.

📖 Tres preguntas, tres herramientas

Con el fin de resumir lo trabajado en esta unidad al respecto de cómo evaluar y analizar qué nos proporciona un ajuste, hemos respondido una serie de preguntas, y proporcionado una serie de herramientas con las que contestarlas, y son:

- **¿Aporta? ¿qué modelo?** (utilidad e inferencia) → Wald, LRT ómnibus, AIC/BIC (2.5).
- **¿Ajusta bien?** (bondad de ajuste) → residuos, calibración, Hosmer–Lemeshow (2.6).
- **¿Discrimina bien?** (clasificación) → matriz de confusión, ROC/AUC, umbral (2.7).

Un modelo puede ser **significativo** y aun así **calibrar mal** o **discriminar poco**: son preguntas independientes.

❗ Tarea del estudiante (1.2) — resuelve la Pregunta 1 del caso

Completa la **Pregunta 1** del cliente ([Unidad 6](#)) sobre la cohorte de tu equipo: partiendo del `glm(ever ~ x1 + x2)` de la Unidad 1, interprétalo, decídelo y evalúalo.

Tareas:

1. **Interpreta (RA3)**. Da el efecto de la fragilidad y del tratamiento en **dos escalas** —odds ratio (con IC) y efecto marginal medio (AME)—, en lenguaje clínico; explica

por qué el coeficiente crudo no es un efecto sobre la probabilidad. Estima el **riesgo de una paciente nueva** con un perfil concreto.

2. **¿Protege por igual a todas? (RA5).** El cliente pregunta si el efecto del tratamiento depende de la fragilidad: añade la **interacción $x_1:x_2$** , contrástala (**LRT**) dentro de una escalera de modelos y compáralos por **AIC y BIC**; documenta dónde discrepan y **justifica tu modelo final** (no basta el AIC mínimo).
3. **Evalúa (RA4).** Reporta **AUC**, evalúa la **calibración** (gráfico + Hosmer–Lemeshow, con su salvedad) y comprueba si hay **separación** en algún subgrupo; si la hubiera, explica cómo la tratarías.
4. **Responde al cliente.** Con lo anterior, redacta la respuesta a la Pregunta 1 y una recomendación accionable.

Formato: fragmento reproducible en el `.qmd` del equipo; es el borrador de la Pregunta 1 del informe (ver [Unidad 6](#)).

Puente a la Unidad 3

Hasta aquí, una respuesta **binaria** modelada con **efectos fijos**. Las próximas unidades generalizan en dos frentes. La [Unidad 3](#) amplía el **tipo de respuesta**: datos **agrupados** (binomial) y **más de dos categorías** (politómica nominal y ordinal). La [Unidad 4](#) introduce los **efectos aleatorios** (GLMM) para el agrupamiento por centro —GLOW tiene pacientes anidadas en 6 centros y la cohorte simulada, en 24—, donde las observaciones de un mismo centro se parecen más de lo que el azar explicaría. Y por último la [Unidad 5](#) aborda la supervivencia desde la perspectiva de los datos binarios.

3 · Extensión de la respuesta binaria: binomial y politómica

La respuesta binaria de las unidades anteriores admite **dos extensiones naturales**, y esta unidad recorre ambas.

La primera es **agrupar**. Cuando las predictoras son **categóricas**, dos sujetos con el mismo perfil de covariables son **intercambiables** —para el modelo, ensayos idénticos—, así que podemos **colapsar** cada perfil en una fila que cuente cuántos casos hay en el grupo. Con respuesta **binaria**, agrupar convierte la Bernoulli individual ($y_i \in \{0, 1\}$) en una **binomial**, $Y_i \sim \text{Bin}(m_i, \pi_i)$ con m_i el tamaño del grupo: es **el mismo modelo logístico**, solo cambia la disposición del dato. Lo vemos en las Secciones [3.1](#) y [3.2](#), donde agrupar además **recupera los diagnósticos de ajuste**.

La segunda es admitir **más de dos categorías**. Si la respuesta es **politómica** —categórica con tres o más niveles—, el logit binario se generaliza al **logit de categoría base** cuando no

hay orden (Sección 3.3) y al de **odds proporcionales** cuando sí lo hay (Sección 3.4). Aquí las tratamos sobre datos **individuales** de GLOW (`raterisk`).

💡 Tip

Las respuestas politómicas también podrían agruparse en recuentos, generando una **multinomial**, $Y_i \sim \text{Mult}(m_i, \pi_i)$, pero su ajuste se apoya en la equivalencia con un modelo **log-lineal de Poisson**, que desarrollamos en el Caso 2, donde se introduce la Poisson, de modo que lo estudiaremos allí.

i Sobre esta unidad

RA implicados (el marco de 1.1–1.2 aplicado a nuevos tipos de respuesta):

- **RA2 · Formulación y estimación.** Ajustar el binomial agrupado y los modelos politómicos (nominal, ordinal).
- **RA3 · Interpretación en la escala correcta.** Odds ratios relativos (nominal) y acumulados (ordinal).
- **RA4 · Diagnóstico y bondad de ajuste.** Contraste ², calibración y sobredispersión.
- **RA5 · Selección y comparación de modelos.** Nominal frente a ordinal (parsimonia).

3.1 Del dato individual al agrupado: respuesta binomial

Para ejemplificar la implementación de esta versión del modelo binomial, apliquemos una agrupación a GLOW, considerando ahora como predictoras las variables `priorfrac`, `momfrac` y `raterisk`. Como las tres predictoras son **factores**, cada combinación de niveles reúne a mujeres intercambiables. Al colapsarlas, la respuesta pasa de un 0/1 por sujeto a un **recuento de fracturas sobre el total del grupo** —una binomial—. Lo llamativo es que **el ajuste no cambia** respecto a la logística individual; lo que sí cambia —y es la ganancia de agrupar— lo veremos en la bondad de ajuste (Sección 3.2).

Agrupados los datos en k celdas (perfiles de covariables), en la celda i

$$Y_i \sim \text{Bi}(m_i, \pi_i), \quad i = 1, \dots, k, \quad \sum_i m_i = n,$$

donde Y_i es el número de fracturas entre las m_i mujeres del perfil y π_i su probabilidad de fractura.

La parte sistemática es común a todos los perfiles, así que la escribimos sin índice —con la respuesta (éxitos, fracasos) = $(Y, m - Y)$ —, y así el modelo binomial plantea estimar:

$$\text{logit}(\pi) = \theta + \beta \text{priorfrac} + \gamma \text{momfrac} + \delta_{\text{Same}} \mathbf{1}(\text{raterisk} = \text{Same}) + \delta_{\text{Greater}} \mathbf{1}(\text{raterisk} = \text{Greater}),$$

con `raterisk = Less` como referencia.

💡 En R. El `glm()` con datos binomiales

El ajuste de datos binomiales es similar al de datos binarios, con la salvedad de cómo se especifica la respuesta aquí, a partir de una matriz con dos columnas: éxitos y fracasos:

```
glm(cind(exitos,fracasos) ~ x1+x2+x1:x2+...,family=binomial(link = "logit"),data)
```

Agrupamos entonces GLOW por tres factores (`priorfrac`, `momfrac`, `raterisk`) y contamos fracturas:

```
glow_agg <- glow |>
  count(priorfrac, momfrac, raterisk, fracture) |>
  tidyr::pivot_wider(names_from = fracture, values_from = n, values_fill = 0) |>
  mutate(total = No + Yes)          # ensayos por celda; Yes = n° de fracturas
glow_agg
```

A tibble: 12 x 6

	priorfrac	momfrac	raterisk	No	Yes	total
	<fct>	<fct>	<fct>	<int>	<int>	<int>
1	No	No	Less	109	18	127
2	No	No	Same	101	23	124
3	No	No	Greater	60	16	76
4	No	Yes	Less	9	2	11
5	No	Yes	Same	11	8	19
6	No	Yes	Greater	11	6	17
7	Yes	No	Less	19	8	27
8	Yes	No	Same	23	14	37
9	Yes	No	Greater	22	22	44
10	Yes	Yes	Less	2	0	2
11	Yes	Yes	Same	3	3	6
12	Yes	Yes	Greater	5	5	10

Antes de modelar conviene **ver** los datos agrupados. La Figura 19 muestra, para cada perfil de covariables, la **proporción observada de fractura** (y_i/m_i); la cifra sobre cada barra es el número de mujeres de la celda (m_i), que recuerda cuánta información sostiene cada proporción.

```

glow_agg |>
  mutate(prop = Yes / total) |>
  ggplot(aes(x = priorfrac, y = prop, fill = momfrac)) +
  geom_col(position = position_dodge(width = 0.8), width = 0.7) +
  geom_text(aes(label = total), position = position_dodge(width = 0.8),
            vjust = -0.3, size = 3) +
  facet_wrap(~ raterisk, labeller = label_both) +
  labs(x = "Fractura previa (priorfrac)", y = "Proporción de fractura (Yes/total)",
       fill = "Antecedente materno (momfrac)") +
  ylim(0, NA)

```

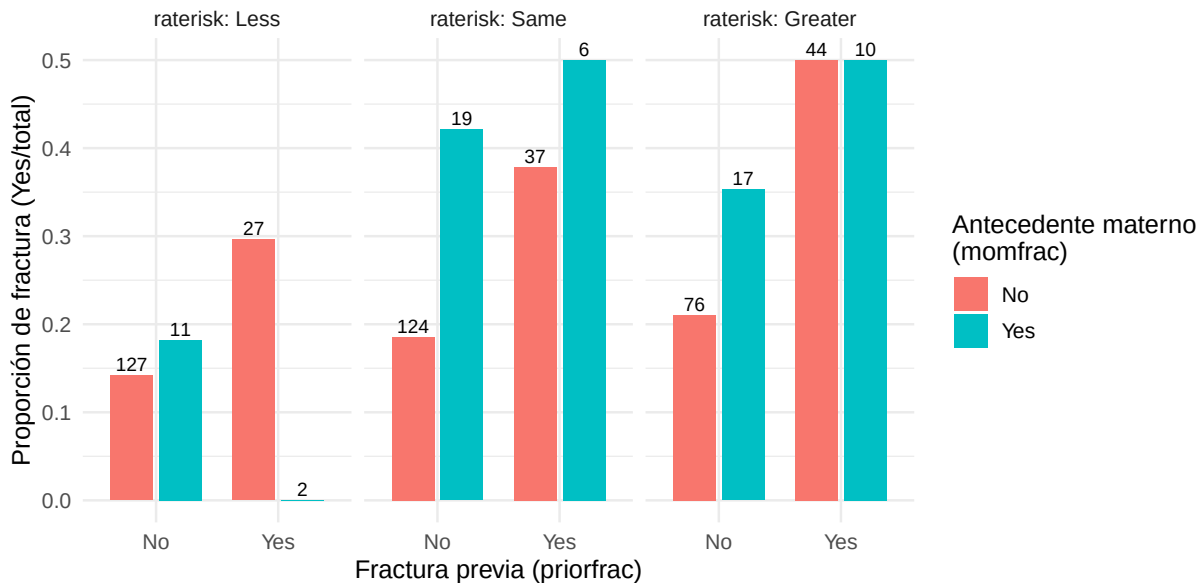


Figura 19: Proporción observada de fractura por perfil de covariables en GLOW. Cada panel es un nivel de riesgo autopercibido (raterisk); dentro de cada panel, las barras cruzan fractura previa (priorfrac) y antecedente materno (momfrac). La cifra sobre la barra es el n° de mujeres de la celda.

En la Figura 19 se aprecian ya las tendencias que el modelo recogerá: la proporción de fractura **sube con la fractura previa** (priorfrac), **con el antecedente materno** (momfrac) y al pasar de riesgo autopercibido **Less** a **Same** y **Greater** —el orden esperado—, lo que respalda incluir las tres covariables. Conviene, eso sí, leer cada barra junto al **tamaño de su celda** (la cifra encima): las tres celdas basales (priorfrac = No, momfrac = No) concentran la mayor parte de los datos ($n = 127, 124, 76$), mientras que las que acumulan ambos antecedentes son **diminutas** ($n = 2, 6, 10$). Sus proporciones son muy inestables —la celda **Less** con ambos antecedentes y $n = 2$ cae a 0, y su análoga en **Same** con $n = 6$ salta a 0,5—: es **ruido de muestreo** que no conviene sobreinterpretar y que, además, es justo el que degrada la

aproximación χ^2 del contraste de bondad de ajuste cuando hay celdas pequeñas (la cautela de la [Sección 3.2](#)).

Ahora ajustamos el binomial con la sintaxis `cbind(éxitos, fracasos)` y comprobamos que **recupera exactamente** los mismos coeficientes que la logística individual con esas predictoras —misma verosimilitud, distinta presentación—:

```
fit_bin <- glm(cbind(Yes, No) ~ priorfrac + momfrac + raterisk,
              family = binomial, data = glow_agg)
fit_ind <- glm(fracture ~ priorfrac + momfrac + raterisk,
              family = binomial, data = glow)

cbind(agrupado = coef(fit_bin), individual = coef(fit_ind)) # idénticos
```

	agrupado	individual
(Intercept)	-1.8711967	-1.8711967
priorfracYes	0.9797192	0.9797192
momfracYes	0.5750477	0.5750477
rateriskSame	0.4647529	0.4647529
rateriskGreater	0.6717294	0.6717294

La interpretación de los coeficientes (odds ratios, [Sección 2.3](#)) y la selección de modelos (LRT/AIC/BIC, [Sección 2.5](#)) son **idénticas** a las del caso individual. Lo que cambia —y es la ganancia de agrupar— es la bondad de ajuste.

3.2 Bondad de ajuste y diagnóstico en datos binomiales

En la [Sección 2.6](#) vimos que, con datos **individuales**, ni la deviance ni el estadístico de Pearson sirven como test absoluto de ajuste: el saturado crece con n y la asintótica χ^2 no se sostiene. Con datos **agrupados** (y m_i suficientemente grandes) esa asintótica **se recupera** (Agresti, 2015; Faraway, 2016). También cobran relevancia ciertos residuos comunes en los GLM que no funcionaban en el caso binario. Presentamos en primer lugar estos residuos y construimos, a partir de ellos y con su **suma de cuadrados**, los estadísticos globales de bondad de ajuste.

Residuos

Al agrupar los datos, cada celda reúne m_i ensayos, de modo que los residuos **estándar** ya son informativos —aproximadamente $N(0, 1)$ con m_i grande (McCullagh & Nelder, 1989)—. Estos residuos, genéricos en los GLM, son:

- **Pearson**:, la diferencia entre lo observado y lo esperado en la celda, tipificada por su desviación estándar,

$$r_{P,i} = \frac{y_i - m_i \hat{\pi}_i}{\sqrt{m_i \hat{\pi}_i (1 - \hat{\pi}_i)}}$$

- **Deviance**:, la raíz con signo de la contribución d_i de cada celda a la deviance,

$$r_{D,i} = \text{sign}(y_i - m_i \hat{\pi}_i) \sqrt{d_i}$$

- Para compararlos en igualdad de condiciones se usan sus versiones **estandarizadas** (o *estudentizadas*), que dividen por $\sqrt{1 - h_{ii}}$,

$$r_{P,i}^s = \frac{r_{P,i}}{\sqrt{1 - h_{ii}}}, \quad r_{D,i}^s = \frac{r_{D,i}}{\sqrt{1 - h_{ii}}}$$

donde h_{ii} es el **leverage** (apalancamiento) de la celda —su influencia *potencial*, dada por el elemento diagonal de la matriz gorro $\hat{H}$, con $\sum_i h_{ii} = p$ —: un h_{ii} alto marca una celda con un perfil de covariables extremo, esto es, sujetos con características alejadas de la mayoría de los datos.

i La matriz sombrero en un GLM

En regresión lineal $\hat{H} = X(X^\top X)^{-1}X^\top$ proyecta $\mathbf{y}$ sobre los valores ajustados. En un GLM el ajuste es por mínimos cuadrados **ponderados** (IWLS), así que la matriz sombrero incorpora los pesos de la última iteración $W = \text{diag}(w_i)$:

$$\hat{H} = W^{1/2} X (X^\top W X)^{-1} X^\top W^{1/2}.$$

Es **simétrica e idempotente**, sus diagonales son los **leverages** h_{ii} y suman el número de parámetros, $\sum_i h_{ii} = \text{tr}(\hat{H}) = p$. En el binomial/logístico los pesos de trabajo son $w_i = m_i \hat{\pi}_i (1 - \hat{\pi}_i)$, de modo que h_{ii} combina lo extremo del perfil de covariables con su varianza. Es el h_{ii} que aparece en $r_{P,i}^s = r_{P,i} / \sqrt{1 - h_{ii}}$ y en la distancia de Cook.

- La **distancia de Cook** combina residuo y leverage, y mide **cuánto cambiarían los coeficientes** si se eliminara esa celda; valores grandes señalan **celdas influyentes** que conviene revisar.

$$D_i = \frac{(r_{P,i}^s)^2}{p} \frac{h_{ii}}{1 - h_{ii}}$$

Calculamos y representamos los residuos estandarizados y las distancias de Cook para GLOW en la Figura 20.

```
#library(patchwork)

diag_bin <- glow_agg |>
  mutate(pred = fitted(fit_bin),
         r_std = rstandard(fit_bin, type = "pearson"),
         cook = cooks.distance(fit_bin),
         cell = row_number())

p_res <- ggplot(diag_bin, aes(pred, r_std, size = total)) +
  geom_hline(yintercept = c(-2, 0, 2), linetype = c("dotted", "dashed", "dotted")) +
  geom_point(alpha = 0.7) +
  labs(x = "Probabilidad ajustada", y = "Residuo de Pearson estandarizado",
       size = "n por celda")

p_cook <- ggplot(diag_bin, aes(cell, cook, size = total)) +
  geom_hline(yintercept = 4 / nrow(diag_bin), linetype = "dotted") +
  geom_segment(aes(xend = cell, yend = 0), linewidth = 0.3) +
  geom_point(alpha = 0.7) +
  scale_x_continuous(breaks = diag_bin$cell) +
  labs(x = "Celda", y = "Distancia de Cook", size = "n por celda")

p_res + p_cook + plot_layout(guides = "collect")
```

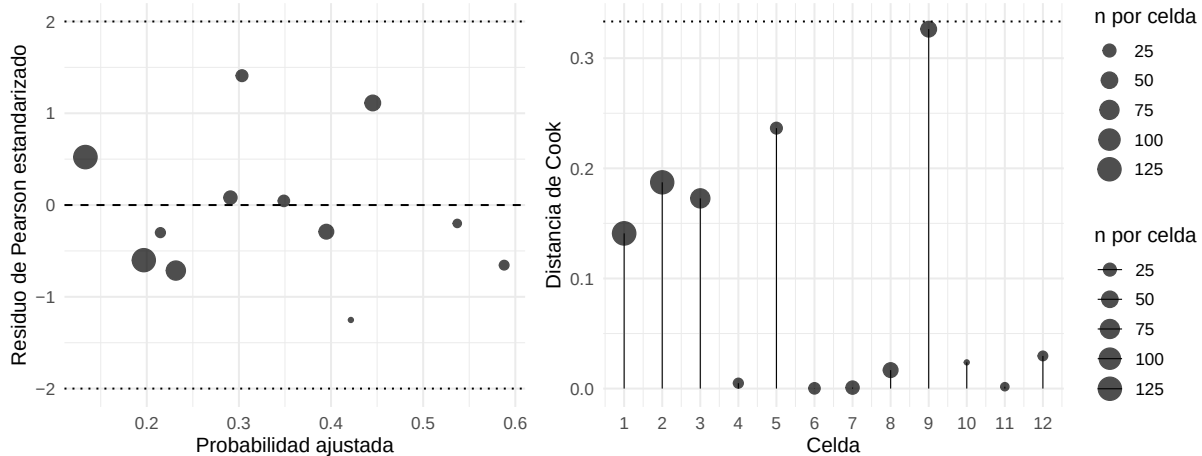


Figura 20: Diagnósticos por celda (el tamaño del punto es el n° de ensayos). Izquierda: residuos de Pearson estandarizados frente a la probabilidad ajustada; con un buen ajuste se reparten en torno a 0 dentro de la banda de ± 2 . Derecha: distancia de Cook por celda, con línea de referencia orientativa en $4/n$.

En GLOW, vemos, a la izquierda en la Figura 20, que los **12 perfiles** (celdas) dejan todos los residuos estandarizados (de Pearson) dentro de la banda de ± 2 , y se reparten sin patrón alrededor de 0 (rango $-1,3$ a $+1,4$); las celdas con más ensayos —las de menor riesgo— caen muy cerca de 0. No hay, pues, **celdas mal ajustadas ni influyentes**, en línea con el χ^2 de bondad de ajuste y la calibración. En el panel derecho, las distancias de Cook son pequeñas en términos absolutos —la mayor (celda 9) ronda 0,33 y apenas roza la referencia orientativa de $4/n$ —, muy por debajo del umbral clásico $D \gtrsim 1$. Las celdas con más ensayos (1–3) presentan un Cook moderado por su peso en el ajuste, no por desajuste. Junto con la ausencia de residuos fuera de la banda de ± 2 , se confirma que ninguna celda resulta influyente.



En R. Residuos e influencia

```
residuals(fit_bin, type = "deviance")      # o "deviance", "response"
rstandard(fit_bin, type = "pearson")      # estandarizados (ajustan por el leverage)
hatvalues(fit_bin)                        # leverage h_ii de cada celda
cooks.distance(fit_bin)                   # influencia de cada celda
```

Bondad de ajuste

Sumando los cuadrados de los residuos deviance y Pearson se obtienen los **dos estadísticos globales**, que comparan el modelo **ajustado con el saturado** —el mejor ajuste posible—:

- **Pearson:** $X^2 = \sum_i r_{P,i}^2$, la suma directa de los residuos de Pearson al cuadrado.
- **Deviance:** $D = \sum_i r_{D,i}^2 = 2(\ell_{\text{sat}} - \ell_{\text{ajustado}})$, que es además el **LRT frente al saturado**.

Bajo un modelo correcto y con celdas no demasiado pequeñas, ambos se distribuyen χ^2_{n-p} (Agresti, 2015; Faraway, 2016), con lo que **ya podemos contrastar el ajuste de forma absoluta** contra el modelo saturado y responder a la pregunta **¿ajusta bien nuestro modelo?**.

```
gl <- df.residual(fit_bin)                # n - p = n° de celdas - n° de parámetros
c(deviance = deviance(fit_bin),
  pearson   = sum(residuals(fit_bin, type = "pearson")^2),
  gl        = gl)
```

```
deviance  pearson      gl
4.732757  4.072651  7.000000
```

```
# p-valores del contraste de bondad de ajuste (H0: el modelo ajusta)
c(p_deviance = pchisq(deviance(fit_bin), gl, lower.tail = FALSE),
  p_pearson = pchisq(sum(residuals(fit_bin, "pearson")^2), gl, lower.tail = FALSE))
```

```
p_deviance  p_pearson
0.6925382  0.7713725
```

Al conseguir un p-valor **alto** (muy superior a la significatividad 0.05) no evidenciamos falta de ajuste (no podemos rechazar el modelo ajustado frente al saturado), y así concluimos que el modelo reproduce razonablemente las proporciones por celda. Es exactamente el contraste que en la [Sección 2.6](#) no podíamos usar y que ahora, al agrupar, recupera su sentido.

! Importante

Cautela: la aproximación exige celdas no demasiado pequeñas; con muchos m_i pequeños vuelve a degradarse.

💡 En R. El contraste χ^2 de bondad de ajuste (deviance y Pearson)

Con el modelo agrupado ajustado, los grados de libertad son **celdas menos parámetros** (`df.residual`) y cada estadístico se compara con una χ^2_{n-p} mediante `pchisq(..., lower.tail = FALSE)`:

```
gl <- df.residual(fit_bin)                                # n - p = n° de celdas - n° de parámetros

# Deviance
pchisq(deviance(fit_bin), gl, lower.tail = FALSE)

# Pearson
X2 <- sum(residuals(fit_bin, type = "pearson")^2)
pchisq(X2, gl, lower.tail = FALSE)
```

Un p **alto** = no hay evidencia de falta de ajuste. ¿Cuál usar?

- **Pearson** (X^2) para el **test absoluto** de bondad de ajuste: aproxima mejor la χ^2 con recuentos moderados y es, además, la base de la **sobredispersión** ($\hat{\phi} = X^2/\text{gl}$).
- **Deviance** para **comparar modelos anidados** (su diferencia es el LRT) y como medida ligada a la verosimilitud; como test **absoluto** es algo menos fiable que Pearson cuando los recuentos son pequeños.
- **Ambos** exigen celdas **no pequeñas** (regla práctica: frecuencias esperadas $\gtrsim 5$); con muchos m_i pequeños la aproximación χ^2 se degrada —y la deviance, antes—.

Calibración

Al agrupar, la **calibración** es inmediata —algo que el caso individual no permitía sin recurrir a deciles—: cada celda aporta su **probabilidad predicha** y su **proporción observada**, y basta enfrentarlas. Si el modelo está bien calibrado, los puntos se alinean sobre la diagonal:

```
# Calibración DIRECTA: cada celda = (probabilidad predicha, proporción observada)
calib_bin <- glow_agg |>
  mutate(pred = fitted(fit_bin), obs = Yes / total)

ggplot(calib_bin, aes(pred, obs, size = total)) +
  geom_abline(linetype = "dashed") + # calibración perfecta
  geom_point(alpha = 0.7) +
  labs(x = "Probabilidad predicha", y = "Proporción observada (Yes/total)",
       size = "n por celda")
```

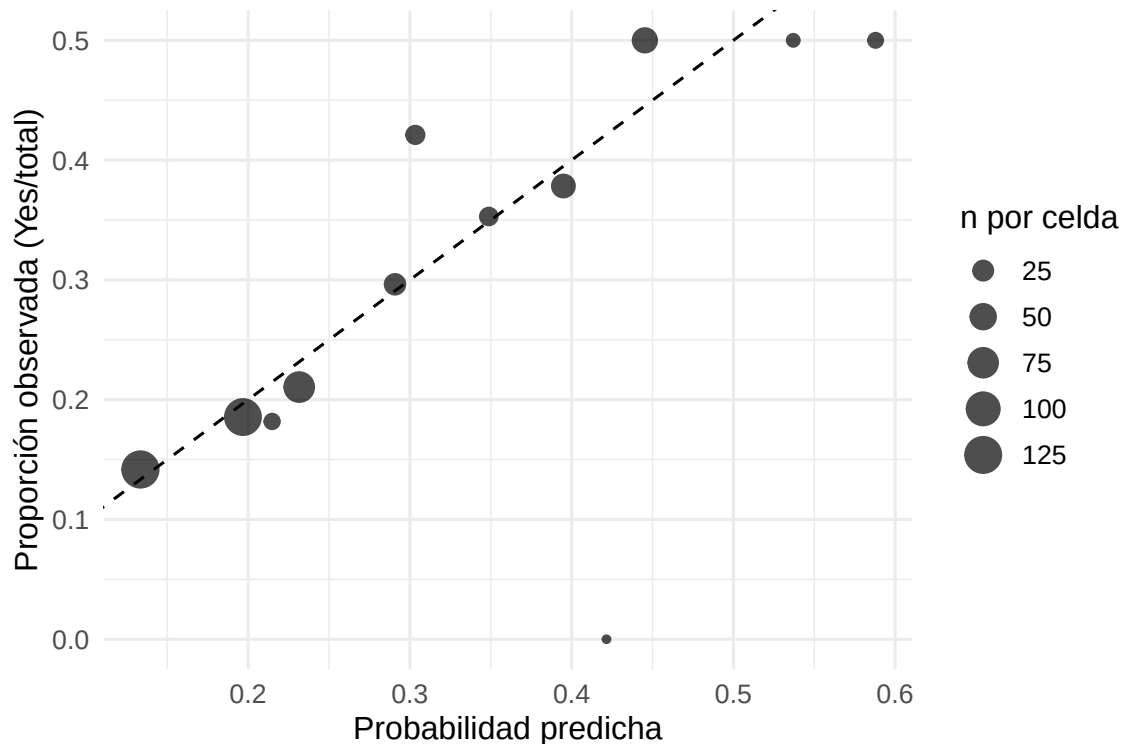


Figura 21: Curva de calibración por celda en GLOW: probabilidad predicha frente a proporción observada de fractura (el tamaño del punto es el n° de mujeres de la celda). La diagonal discontinua marca la calibración perfecta; los puntos próximos a ella, sin sesgo sistemático, indican un modelo bien calibrado.

Como se aprecia en Figura 21, en GLOW los 12 puntos caen cerca de la diagonal, sin sesgo sistemático por tramos de riesgo: el modelo está **bien calibrado**, en línea con el χ^2 de bondad de ajuste y los residuos.



En R. El data frame de observados vs predichos para la calibración

La calibración solo necesita, por celda, la probabilidad **predicha** y la proporción **observada**. La forma directa es añadirlas con `mutate()`:

```
calib_bin <- glow_agg |>
  mutate(pred = fitted(fit_bin),      # probabilidad predicha por el modelo
         obs  = Yes / total)         # proporción observada en la celda
```

Con `broom` se obtiene lo mismo de forma compacta —`type.predict = "response"` devuelve la predicción en escala de probabilidad en la columna `.fitted`—:

```
library(broom)
calib_bin <- augment(fit_bin, data = glow_agg, type.predict = "response") |>
  mutate(obs = Yes / total)
```

Sobre ese data frame, `geom_point(aes(pred, obs))` dibuja la calibración por celda y `geom_abline()` la diagonal de referencia; un `geom_smooth(aes(pred, obs))` traza además la **curva de calibración** suavizada.

Sobredispersión

El modelo binomial $y_i \sim Bi(m_i, \pi_i)$, impone que la varianza de cada celda sea exactamente $m_i \hat{\pi}_i (1 - \hat{\pi}_i) = Var(y_i)$. Cuando la varianza real en los datos **excede** ese valor hay **sobredispersión**, y al agrupar pasa a ser **estimable** mediante el cociente de Pearson:

$$\hat{\phi} = \frac{X^2}{n - p} = \frac{1}{n - p} \sum_i r_{P,i}^2.$$

Un $\hat{\phi} \approx 1$ indica que el binomial basta; $\hat{\phi} > 1$ indica sobredispersión (variación *extra-binomial*); $\hat{\phi} < 1$ indica infradispersión (es menos frecuente).

La sobredispersión puede ser debida a cierta **correlación entre los ensayos de una misma celda** (sujetos agrupados que no son del todo independientes), a **heterogeneidad no modelada** o por una **covariable omitida**. Lo que importa es su efecto: los coeficientes $\hat{\beta}$ siguen siendo **consistentes**, pero los **errores estándar se subestiman**, de modo que los contrastes de Wald y LRT resultan **demasiado liberales**, y los intervalos **demasiado estrechos**.

El remedio inmediato es el modelo **quasi-binomial**: no cambia las estimaciones puntuales, pero **estima ϕ de los datos** e **infla los errores estándar** por $\sqrt{\hat{\phi}}$ (y sustituye los contrastes χ^2 por contrastes **F**). Cuando la sobredispersión procede de un efecto de agrupamiento identificable, la alternativa estructural pasa por incluir **efectos aleatorios** (Unidad 4).

```
# Índice de sobredispersión (cociente de Pearson)
sum(residuals(fit_bin, type = "pearson")^2) / df.residual(fit_bin)
```

```
[1] 0.5818072
```

En GLOW, $\hat{\phi} \approx 0,58$: **por debajo de 1**, así que **no hay rastro de sobredispersión** —el binomial es de sobra suficiente— y el quasi-binomial no haría falta. Lo mostramos igualmente como herramienta para cuando $\hat{\phi}$ supere con claridad la unidad.

💡 En R. Sobredispersión y quasi-binomial

```
# Índice de sobredispersión (cociente de Pearson): ~1 si basta el binomial
sum(residuals(fit_bin, type = "pearson")^2) / df.residual(fit_bin)

# Ajuste robusto a sobredispersión: mismas estimaciones, SE inflados por sqrt(phi)
fit_q <- glm(cbind(Yes, No) ~ priorfrac + momfrac + raterisk,
            family = quasibinomial, data = glow_agg)
summary(fit_q)                # 'Dispersion parameter' = phi estimado; contrastes tipo t/F
drop1(fit_q, test = "F")      # comparación de modelos con test F (no chi^2)
```

❗ ¿Y tratar la sobredispersión binomial con quasi-Poisson?

Hay una excepción a tratar la sobredispersión en el modelo binomial con un **quasi-Poisson**, y es el régimen de **sucesos raros** (m_i grande, π_i pequeña y $m_i\pi_i$ moderado), donde la binomial se aproxima a una Poisson. Ahí sí tiene sentido modelar el **conteo** con una Poisson de enlace log y un *offset* $\log(m_i)$ —es decir, modelar una **tasa**—, y un quasi-Poisson o una binomial negativa absorberían la variación extra. Pero eso es porque has entrado **deliberadamente** en el dominio Poisson, no porque el quasi-Poisson arregle una binomial. Estos modelos los veremos en el Caso 2.

En **todo lo demás**, el binomial agrupado se ajusta e interpreta **exactamente igual** que el logístico individual de la Unidad 2: misma estimación (máxima verosimilitud / IWLS), misma inferencia (Wald, LRT, IC por perfil) y misma selección de modelos (deviance, AIC/BIC, la escalera de la Sección 2.5). Y la **lectura de los resultados no cambia**: los coeficientes siguen

siendo log-odds, e^β es un odds ratio y las predicciones se interpretan en probabilidad igual que en la [Sección 2.3](#). La única diferencia es la que acabamos de explotar: al agrupar, recuperamos los **diagnósticos de ajuste** (el χ^2 de bondad de ajuste, los residuos, la calibración y la sobredispersión por celda) que en el caso individual no eran válidos.

3.3 Más de dos categorías sin orden: politómica nominal

Cuando seguimos teniendo respuestas individualizadas y la respuesta tiene **más de dos categorías sin orden**, el logit binario se generaliza al **logit de categoría base**: se fija una categoría de referencia y se modela, frente a ella, el log-odds de cada una de las demás.

Para ilustrar este modelo con GLOW, usamos **raterisk** —el riesgo de fractura **autopercibido** como variable respuesta Y , observada en tres niveles **Less/Same/Greater**—, y usamos la categoría **Same** como referencia.

$$\log \frac{\Pr(Y = k)}{\Pr(Y = \text{Same})} = \eta_k = \theta_k + \beta_k \cdot \text{age} + \gamma_k \cdot \mathbf{1}(\text{priorfrac} = \text{Yes}), \quad k \in \{\text{Less}, \text{Greater}\}$$

Conviene leerla como **dos rectas, una por cada categoría no-referencia**. Fijada **Same** como base, a cada nivel $k \in \{\text{Less}, \text{Greater}\}$ le asignamos su **propio predictor lineal** η_k , con su **propia** interceptación y **sus propias** pendientes: una recta describe cómo varían, con la edad y la fractura previa, los log-odds de percibirse en **menor** riesgo frente a «igual» (**Less** vs **Same**); la otra, los de percibirse en **mayor** riesgo frente a «igual» (**Greater** vs **Same**). La categoría base no necesita recta propia: hace de **vara de medir**, con $\eta_{\text{Same}} = 0$ por convención.

Ajustar el modelo nominal equivale, pues, a estimar **a la vez** $K - 1 = 2$ regresiones —**Less|Same** y **Greater|Same**—, cada una con su intercepto $\theta_k + \gamma_k \mathbf{1}(\text{priorfrac} = \text{Yes})$ y su pendiente β_k . Que cada categoría tenga **su propia recta** es lo que da al modelo nominal su flexibilidad —cada nivel puede responder a la edad o a la fractura previa de manera distinta, incluso en **direcciones opuestas**—, frente al ordinal de la [Sección 3.4](#), que impondrá una **pendiente común** a todos los cortes.

Al exponenciar, e^{β_k} es un **odds ratio relativo**: el factor por el que se multiplican las **odds de pertenecer a la categoría k en lugar de a Same** cuando **age** aumenta una unidad, manteniendo el resto constante. Por ejemplo, $e^{\beta_{\text{Greater}}}$ es cuánto cambia la ventaja de percibirse en **mayor** riesgo (**Greater**) frente a **igual** (**Same**) por cada año más de edad. Cuando se estiman los coeficientes del modelo, su **signo** ya orienta: $\beta_k > 0$ aumenta el riesgo de k sobre **Same** al aumentar la edad, y $\beta_k < 0$, disminuye el riesgo en k respecto de **Same**.

Es siempre una comparación **por pares contra la referencia**: el odds de **Less** frente a **Greater** no aparece directamente —se obtiene combinando ambos juegos ($e^{\eta_{\text{Less}} - \eta_{\text{Greater}}}$) o reeligiendo la categoría base—. Cambiar la referencia cambia los coeficientes, pero **no** las probabilidades ajustadas.

i Individual o agrupado: qué cambia y qué no

Tanto si la respuesta categórica es **individual** (una fila por sujeto, como aquí con `raterisk`) como **agrupada** (recuentos por perfil, multinomial), el **modelo y sus estimaciones son los mismos**: misma verosimilitud, misma interpretación de coeficientes, y la **selección de modelos** (LRT, AIC/BIC) funciona igual. Lo que **no** vale con datos individuales es el contraste χ^2 **de bondad de ajuste** ni el análisis de **residuos**: como en el paso Bernoulli→binomial (Sección 3.2), solo recuperan su sentido al **agrupar**. Por eso aquí valoramos el ajuste mirando las **probabilidades predichas**, no un χ^2 .

Un cambio de motor: esto ya no se ajusta con `glm()`. La función `glm()` está programada para respuestas de la familia exponencial **univariante** —un predictor lineal para una respuesta escalar—; el multinomial tiene respuesta **vectorial** (K probabilidades que suman 1) y necesita $K - 1$ **predictores lineales a la vez**, de modo que se sale de lo que cubre `glm()` (aunque sigue siendo un GLM en sentido amplio). En R se ajusta con `nnet::multinom()` —lo que usamos aquí— o con `VGAM::vglm(..., family = multinomial)`; ambas estiman por máxima verosimilitud el **mismo** logit de categoría base.

? En R. Ajustar e inspeccionar el multinomial nominal

La respuesta debe ser un **factor** en el que hay que identificar la categoría de referencia.

Ajuste (dos motores equivalentes).

```
library(nnet)
m_nom <- multinom(relevel(raterisk, ref = "Same") ~ age + priorfrac, data = glow, trace = FALSE)

library(VGAM)
m_vglm <- vglm(raterisk ~ age + priorfrac,
               family = multinomial(refLevel = "Same"), data = glow)
```

Resúmenes.

En `nnet`:

- `summary(m_nom)` da un bloque de coeficientes por categoría no-referencia (`Less|Same`, `Greater|Same`) con sus errores estándar;
- `multinom` no imprime p -valores, así que se sacan con `broom::tidy(m_nom)` (añade z y p), y
- `exp(coef(m_nom))` da los **odds ratios relativos**.

En `vglm`:

- `summary(m_vglm)` y

- `coef(m_vglm, matrix = TRUE)`.

Bondad de ajuste. Con datos individuales no hay χ^2 de bondad de ajuste (Sección 2.6); se usan la verosimilitud y los criterios de información: `logLik()`, `AIC()`, `BIC()`, `deviance()`.

Comparación de modelos. Modelos **anidados** con el LRT —`lmtest::lrtest(m0, m_nom)` (o `anova()` en `vglm`)—; **no anidados**, con AIC/BIC.

Ajustes y predicciones.

En `nnet`:

```
fitted(m_nom) # matriz de probabilidades** (una columna por nivel)
predict(m_nom, type = "probs") # predice probabilidades para datos nuevos
predict(m_nom, type = "class") # da la categoría más probable.
ggeffects::ggpredict(m_nom, terms = "age [all]") # para visualizarlas frente a una covariable
```

En `vglm`:

```
fitted(m_vglm)` # equivale a `predict(m_vglm, type = "response")`
nd <- data.frame(age = 60:90, priorfrac = "No") # datos nuevos
predict(m_vglm, newdata = nd, type = "response") # predicción de probabilidades para datos nuevos
predict(m_vglm, newdata = nd, type = "link") # predicción de logit frente a referencia
colnames(p)[max.col(p, ties.method = "first")] # da la categoría más probable
```

Ajustamos a continuación este modelo sobre GLOW, utilizando `nnet`.

```
library(nnet)
m_nom <- multinom(relevel(raterisk, ref = "Same") ~ age + priorfrac,
                  data = glow, trace = FALSE)
summary(m_nom) # dos bloques de coeficientes: Less|Same y Greater|Same
```

Call:

```
multinom(formula = relevel(raterisk, ref = "Same") ~ age + priorfrac,
          data = glow, trace = FALSE)
```

Coefficients:

	(Intercept)	age	priorfracYes
Less	-0.7074585	0.009952521	-0.4240676
Greater	1.0758421	-0.022686702	0.8022770

Std. Errors:

	(Intercept)	age	priorfracYes
--	-------------	-----	--------------


```
Less      0.8471448 0.01242217    0.2809105
Greater   0.8876437 0.01325035    0.2598469
```

```
Residual Deviance: 1072.149
AIC: 1084.149
```

```
tidy(m_nom) # visualización ordenada
```

```
# A tibble: 6 x 6
  y.level term          estimate std.error statistic p.value
  <chr>   <chr>          <dbl>    <dbl>    <dbl>   <dbl>
1 Less   (Intercept)    -0.707     0.847    -0.835  0.404
2 Less   age             0.00995    0.0124     0.801  0.423
3 Less   priorfracYes   -0.424     0.281    -1.51   0.131
4 Greater (Intercept)     1.08      0.888     1.21   0.226
5 Greater age      -0.0227    0.0133    -1.71   0.0869
6 Greater priorfracYes  0.802     0.260     3.09   0.00202
```

```
exp(coef(m_nom)) # odds ratios relativos
```

```
      (Intercept)      age priorfracYes
Less      0.4928953 1.0100022    0.6543797
Greater    2.9324614 0.9775687    2.2306142
```

multinom devuelve **dos bloques** de odds ratios relativos, **Less|Same** y **Greater|Same**. El motor es la **fractura previa**: tener una multiplica por **2,23** las odds de percibirse en **mayor** riesgo (**Greater**) frente a «igual» (**Same**) y, a la vez, las reduce a **0,65** (−35 %) para percibirse en **menor** riesgo (**Less**); es decir, empuja la percepción fuera de «Same» hacia «Greater» —clínicamente coherente y buena señal de que el modelo capta algo real—. La **edad**, en cambio, apenas influye (OR 1: 1,01 para **Less**, 0,98 para **Greater**). Dos matices: son odds **relativas a Same**, no frente a todas las categorías; y los interceptos (OR de **Less** 0,49 y **Greater** 2,93) corresponden a **age = 0**, una extrapolación sin sentido sustantivo —para leer en probabilidad por edades, la Figura 22—.

Con los coeficientes estimados, la **probabilidad predicha** sale de combinar los dos predictores lineales (con **1**(priorfrac = Yes) y age en años):

$$\eta_{\text{Less}} = -0,71 + 0,010 \text{ age} - 0,42 \cdot \mathbf{1}(\text{priorfrac} = \text{Yes}), \quad \eta_{\text{Greater}} = 1,08 - 0,023 \text{ age} + 0,80 \cdot \mathbf{1}(\text{priorfrac} = \text{Yes}).$$

Como **Same** es la referencia y las tres probabilidades suman 1,

$$\Pr(Y = \text{Less}) = \frac{e^{\eta_{\text{Less}}}}{1 + e^{\eta_{\text{Less}}} + e^{\eta_{\text{Greater}}}}, \quad \Pr(Y = \text{Greater}) = \frac{e^{\eta_{\text{Greater}}}}{1 + e^{\eta_{\text{Less}}} + e^{\eta_{\text{Greater}}}},$$

y $\Pr(Y = \text{Same}) = 1 / (1 + e^{\eta_{\text{Less}}} + e^{\eta_{\text{Greater}}})$.

Lo más comunicable es la **probabilidad predicha** de cada nivel. La Figura 22 la traza frente a la edad:

```
library(ggeffects)
plot(ggpredict(m_nom, terms = "age [all]"))
```

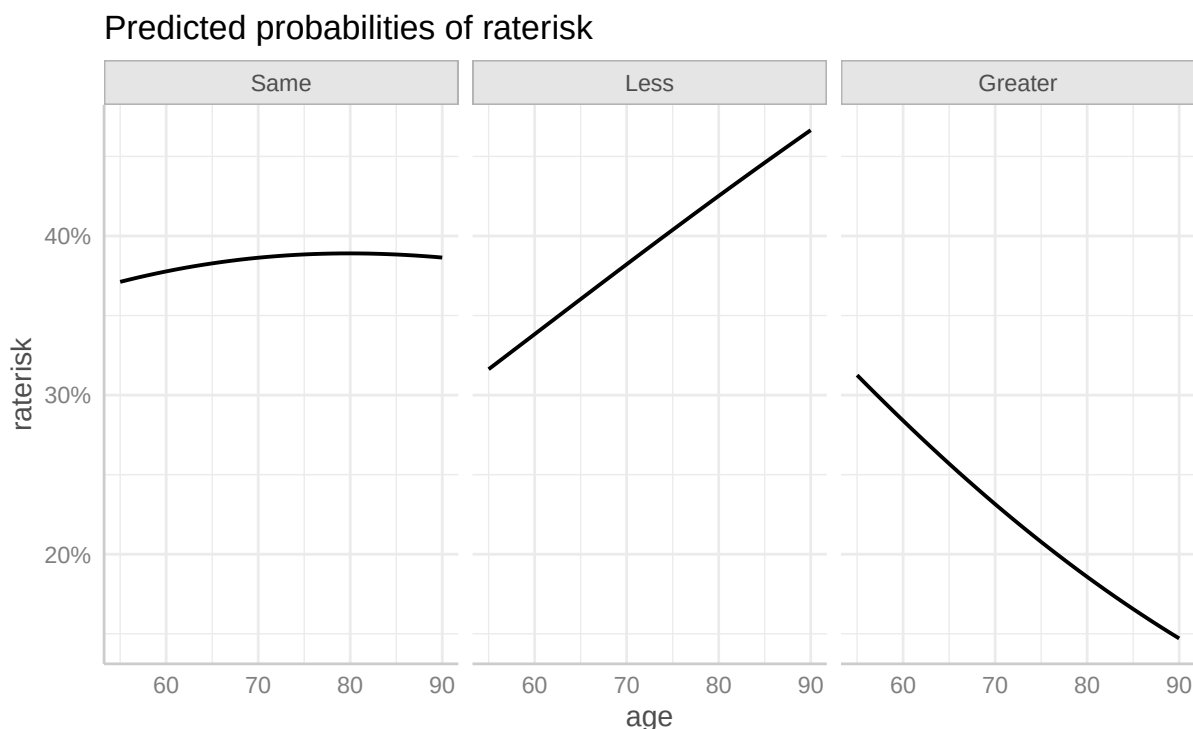


Figura 22: Probabilidad predicha de cada nivel de raterisk según la edad (multinomial nominal, priorfrac = No). Cada categoría sigue su propia curva, sin restricción entre ellas.

La Figura 22 traduce los coeficientes a **probabilidad** (con **priorfrac** = No) y revela el patrón con la edad: la de percibirse en **menor** riesgo (**Less**) **crece** de 32 % a los 55 años a 47 % a los 90, mientras que la de **mayor** riesgo (**Greater**) **cae** de 31 % a 14 %; la referencia **Same** se mantiene casi plana en torno al 38 % (con una leve curvatura: en la escala de probabilidad las curvas no tienen por qué ser monótonas aunque los logit sean lineales). Las tres se mueven **con independencia** —sin restricción de orden, propio del modelo nominal— pero **suman**

1 a cada edad. El desplazamiento concuerda con los OR por año (**Less** 1,01; **Greater** 0,98): minúsculos uno a uno, pero apreciables **acumulados** sobre 35 años. Sustantivamente llama la atención —y es clínicamente relevante— que las mujeres mayores tiendan a **autopercibir menos riesgo** justo cuando el objetivo es mayor, una brecha de percepción que el modelo capta. (Con `priorfrac = Yes` el perfil se desplazaría hacia **Greater**, acorde con su OR de 2,23.)

Bondad de ajuste, diagnóstico y comparación.

- Con la respuesta **individual** no disponemos del χ^2 de bondad de ajuste ni de los residuos estándar ([Sección 3.2](#)): el ajuste se valora con la **verosimilitud**.
- El contraste **global** —¿aporta el modelo frente al nulo?— es un LRT, $D_0 - D \sim \chi^2$ con $(K - 1)p$ grados de libertad (aquí, los dos bloques de pendientes a la vez).
- El **diagnóstico** se apoya en las **probabilidades predichas**
- La **calibración** se examina **por categoría** —probabilidad media predicha frente a frecuencia observada en grupos de riesgo, una *reliability* por nivel—.
- El acierto clasificatorio se concluye con la **matriz de confusión** (`predict(m_nom, type = "class")`) o el *Brier score* multiclase.
- Para residuos formales hay que recurrir a residuos **simulados/surrogate**, no a los clásicos.
- La **comparación de modelos** sigue la lógica habitual:
 - **anidados** por LRT (`lmtest::lrtest, anova`) —por ejemplo, contrastar si `priorfrac` aporta— y
 - **no anidados** por **AIC/BIC**.

Una comparación natural es con el modelo **ordinal** de la [Sección 3.4](#), que aprovecha el orden de las categorías y gana parsimonia: un solo β por covariable en vez de uno por categoría.

El *Brier score* multiclase, que acabamos de mencionar, merece su definición. Es el **error cuadrático medio** entre el vector de probabilidades predichas y la clase observada codificada como *one-hot*:

$$\text{BS} = \frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K (\hat{p}_{ik} - y_{ik})^2, \quad y_{ik} = \mathbf{1}\{\text{la clase observada de } i \text{ es } k\},$$

donde $\hat{p}_{ik}$ es la probabilidad predicha de que la observación i pertenezca a la categoría k . Es un *scoring rule* **propio** —penaliza a la vez la mala **calibración** y la falta de **nitidez**—: vale **0** con predicción perfecta y, sumando sobre las K clases, como máximo **2** (el caso binario es el conocido $\frac{1}{n} \sum_i (\hat{p}_i - y_i)^2 \in [0, 1]$). Cuanto **menor**, mejor; sirve para **comparar modelos** sobre las mismas observaciones.

💡 En R. Brier score multiclase

A partir de la matriz de probabilidades predichas (`fitted`) y la clase observada en *one-hot*:

```
P <- fitted(m_nom) # n x K; columnas = niveles de la respuesta
obs <- factor(glow$raterisk, levels = colnames(P))
Y <- model.matrix(~ 0 + obs) # one-hot
colnames(Y) <- levels(obs) # alinea columnas con las de P
brier <- mean(rowSums((P - Y)^2)) # 0 = perfecto; máximo 2
brier
```

Con `yardstick` se calcula directamente —la respuesta y las columnas de probabilidad deben llevar el nombre de cada clase—:

```
library(yardstick); library(tibble)
prob <- as_tibble(fitted(m_nom)) # columnas = niveles de la respuesta
prob$obs <- factor(glow$raterisk, levels = names(prob))
brier_class(prob, truth = obs, Same, Less, Greater)
```

3.4 Categorías ordenadas: el modelo de odds proporcionales

Consideramos ahora la respuesta individualizada `raterisk`, que **tiene orden** (`Less < Same < Greater`), y el modelo politómico anterior lo ignoraba. Cuando el orden existe, conviene aprovecharlo con el modelo de **odds proporcionales** (*cumulative logit*), que modela los **logits acumulados** con umbrales ordenados y **un único** vector de efectos:

$$\text{logit } \Pr(Y \leq k) = \log \frac{\Pr(Y \leq k)}{\Pr(Y > k)} = \theta_k - (\beta \cdot \text{age} + \gamma \cdot \mathbf{1}(\text{priorfrac} = \text{Yes})),$$

con $k \in \{\text{Less}, \text{Same}\}$, $\theta_{\text{Less}} < \theta_{\text{Same}}$.

Frente al nominal de la [Sección 3.3](#) —que estimaba **un vector de efectos por categoría**— aquí hay dos piezas bien diferenciadas:

- los **umbrales** (interceptos ordenados), uno por cada corte entre categorías contiguas. Con tres niveles hay **dos**,

- corte `Less|Same`, es decir $\Pr(Y \leq \text{Less})$: θ_{Less} y
- corte `Same|Greater`, $\Pr(Y \leq \text{Same})$: θ_{Same} ,

ordenados $\theta_{\text{Less}} < \theta_{\text{Same}}$ para garantizar que las probabilidades acumuladas crezcan y, con ellas, que las de cada categoría sean no negativas—;

- un **único vector de efectos** (β, γ) **compartido por los dos cortes** —de ahí «odds proporcionales»—: las covariables **desplazan en bloque** toda la distribución sobre la escala ordenada, sin alterar la separación entre umbrales.

El signo y su lectura.

El predictor lineal entra **restado**, $\theta_k - \eta$, con $\eta = \beta \cdot \text{age} + \gamma \cdot \mathbf{1}(\text{priorfrac} = \text{Yes})$ -que es la convención de `MASS::polr`-. No es un capricho: hace que un coeficiente **positivo** empuje hacia las categorías **altas**. Si $\beta > 0$, sube η , baja $\theta_k - \eta$ y baja $\Pr(Y \leq k)$ en **todos** los cortes —es decir, sube $\Pr(Y > k)$ —.

Conviene, pues, leer el efecto sobre las **odds de superar un corte**: - una unidad más de x multiplica $\text{odds}(Y > k)$ por e^β —el **mismo** factor en los dos cortes— y, equivalentemente, $\text{odds}(Y \leq k)$ por $e^{-\beta}$ - el hecho de que una paciente haya tenido fractura previa frente a que no la haya tenido, resta γ a $\text{logit } \Pr(Y \leq k)$, de modo que multiplica por $e^{-\gamma}$ las odds acumuladas $\text{odds}(Y \leq k)$ y por $e^{+\gamma}$ las odds de superar el corte $\text{odds}(Y > k)$, y es el mismo factor en los dos cortes.

En GLOW, los umbrales estimados son $\theta_{\text{Less}} \approx -2,06$ y $\theta_{\text{Same}} \approx -0,44$, y el vector de efectos $(\beta_{\text{age}}, \gamma_{\text{priorfrac}}) \approx (-0,023, 0,90)$. Se lee así: la **fractura previa** ($\gamma > 0$, $e^\gamma \approx 2,46$) multiplica por $\approx 2,5$ las odds de percibirse en una categoría **más alta** (más **Greater**, menos **Less**), por igual en ambos cortes; la **edad** ($\beta < 0$, $e^\beta \approx 0,98$ por año) empuja levemente hacia categorías **más bajas** (**Less**). Ambos efectos coinciden en dirección con los del nominal, señal de que la proporcionalidad de odds es razonable aquí.

Se ajusta con `MASS::polr`, que **exige** una respuesta de tipo **factor ordenado**:

```
glow_ord <- glow |>
  mutate(raterisk = ordered(raterisk, levels = c("Less", "Same", "Greater")))
m_ord <- MASS::polr(raterisk ~ age + priorfrac, data = glow_ord, Hess = TRUE)
tidy(m_ord)           # dos umbrales (theta) y los efectos (beta), comunes a ambos cortes
```

A tibble: 4 x 5

	term	estimate	std.error	statistic	coef.type
	<chr>	<dbl>	<dbl>	<dbl>	<chr>
1	age	-0.0228	0.00974	-2.34	coefficient
2	priorfracYes	0.900	0.204	4.41	coefficient
3	Less Same	-2.06	0.665	-3.10	scale
4	Same Greater	-0.439	0.658	-0.667	scale

```
exp(coef(m_ord))      # odds ratios acumulados
```

```

      age priorfracYes
0.9774424    2.4588903

```

Frente al nominal de 3.3, el ordinal **gana parsimonia**: un solo β por covariable en vez de uno por categoría, a cambio del **supuesto de proporcionalidad de odds** (el mismo efecto en todos los cortes), que conviene contrastar (un test de líneas paralelas).

💡 En R. Ajustar e inspeccionar el ordinal (odds proporcionales)

Respuesta. Debe ser un **factor ordenado** (`ordered()` con los niveles en su orden natural):

```

glow$raterisk <- ordered(glow$raterisk, levels = c("Less", "Same", "Greater"))

```

Ajuste (motores equivalentes).

```

m_ord <- MASS::polr(raterisk ~ age + priorfrac, data = glow, Hess = TRUE)
m_clm <- ordinal::clm(raterisk ~ age + priorfrac, data = glow)
m_vglm <- VGAM::vglm(raterisk ~ age + priorfrac,
                     family = cumulative(parallel = TRUE), data = glow) # parallel = od

```

Resúmenes.

- `summary(m_ord)` da un **único bloque de pendientes** (común a los cortes) y los **umbrales** (Intercepts / zeta);
- `broom::tidy(m_ord)` los ordena (marca `coef.type = coefficient / scale`)
- `exp(coef(m_ord))` da los **odds ratios acumulados**.

`polr` no imprime p -valores; se sacan del valor t o con `ordinal::clm` (que sí los da).

Bondad de ajuste y comparación.

- Verosimilitud y criterios: `logLik()`, `AIC()`, `BIC()`, `deviance()`.
- **Modelos anidados** con el LRT (`anova`, `lmtest::lrtest`).
- Y clave aquí, el **supuesto de proporcionalidad de odds** (líneas paralelas) se contrasta con `brant::brant(m_ord)` o `ordinal::nominal_test(m_clm)`.
- Comparar con el **nominal** por AIC/BIC indica si compensa la parsimonia.

Ajustes y predicciones.

- `predict(m_ord, type = "probs")` da la **matriz de probabilidades**,
- `predict(m_ord, type = "class")` la categoría modal, y
- `ggeffects::ggpredict(m_ord, terms = "age[all]")` las dibuja.

```
library(ggeffects)
plot(ggpredict(m_ord, terms = "age [all]"))
```

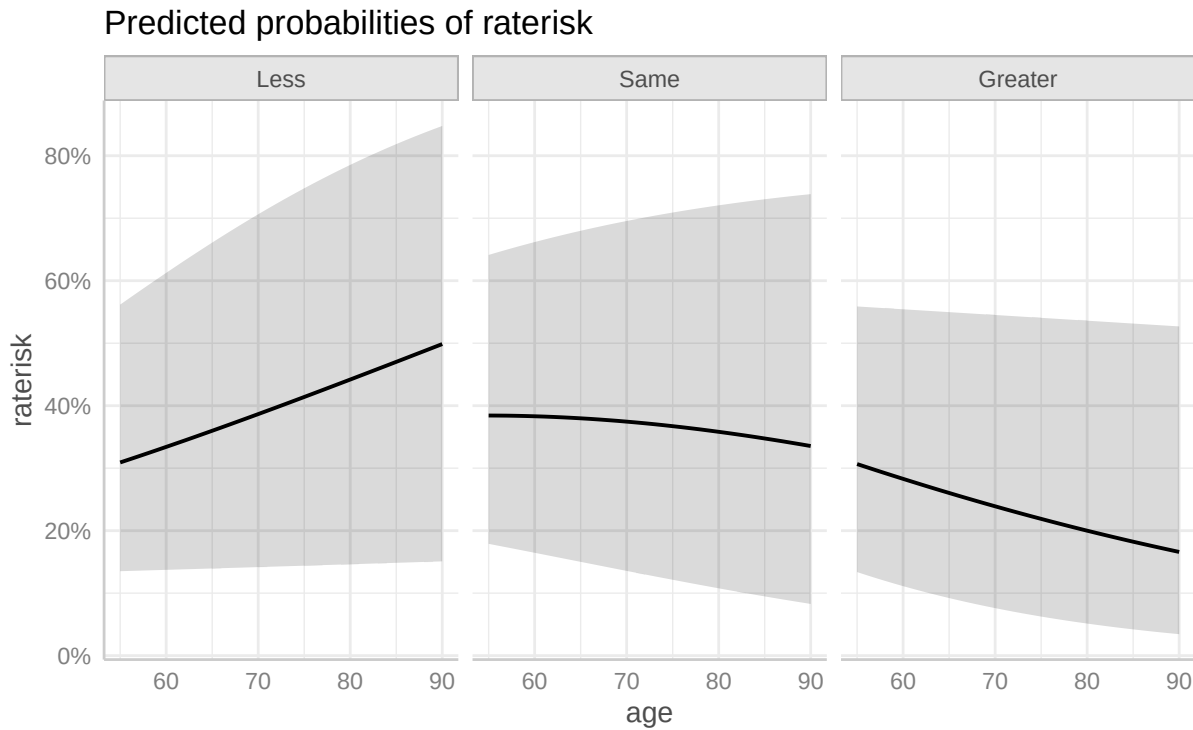


Figura 23: Probabilidad predicha de cada nivel de raterisk según la edad (odds proporcionales, priorfrac = No). Las curvas están ligadas por un único efecto de la edad común a todos los cortes.

La Figura 23 lo deja ver: frente a las **curvas libres** del nominal (Figura 22), aquí están **ligadas** por un único efecto de la edad —la categoría central (**Same**) forma una joroba y las extremas son monótonas—. Si ambas imágenes se parecen, el supuesto de proporcionalidad es razonable y ganamos parsimonia; si difieren mucho, el ordinal estaría forzando.

Con ambos modelos ajustados, la comparación numérica resume las **discrepancias** entre nominal y ordinal:

```
# Ajuste y parsimonia (el ordinal usa menos parámetros)
ajuste <- tibble::tibble(
  Modelo      = c("Nominal (multinom)", "Ordinal (polr)"),
  Parametros  = c(length(coef(m_nom)), length(coef(m_ord)) + length(m_ord$zeta)),
  logLik      = c(as.numeric(logLik(m_nom)), as.numeric(logLik(m_ord))),
```

```

  AIC      = c(AIC(m_nom), AIC(m_ord)),
  BIC      = c(BIC(m_nom), BIC(m_ord))
)

# Odds ratios: el nominal da uno por categoría; el ordinal, uno (acumulado) común
or_nom <- exp(coef(m_nom))      # filas Less/Greater x (Intercept, age, priorfracYes)
or_ord <- exp(coef(m_ord))      # age, priorfracYes (acumulado)
discrepancias <- tibble::tibble(
  Covariable      = c("age", "priorfracYes"),
  `Nominal Less|Same` = or_nom["Less", c("age", "priorfracYes")],
  `Nominal Greater|Same` = or_nom["Greater", c("age", "priorfracYes")],
  `Ordinal (acumulado)` = or_ord[c("age", "priorfracYes")]
)

ajuste

```

```

# A tibble: 2 x 5
  Modelo      Parametros logLik   AIC   BIC
  <chr>          <int>   <dbl> <dbl> <dbl>
1 Nominal (multinom)      6 -536. 1084. 1109.
2 Ordinal (polr)          4 -536. 1081. 1098.

```

```
discrepancias
```

```

# A tibble: 2 x 4
  Covariable   `Nominal Less|Same` `Nominal Greater|Same` `Ordinal (acumulado)`
  <chr>          <dbl>          <dbl>          <dbl>
1 age            1.01            0.978            0.977
2 priorfracYes    0.654            2.23            2.46

```

La tabla resume el contraste. En **ajuste**, el ordinal emplea **4 parámetros** (dos umbrales + dos pendientes) frente a los **6** del nominal (dos bloques de tres); si la proporcionalidad de odds se sostiene, su AIC/BIC salen **competitivos o mejores** pese a algo menos de verosimilitud. En **efectos**, la diferencia estructural es nítida: el nominal estima **un OR por categoría** y el ordinal los **colapsa en uno solo, acumulado**. Para **priorfrac**, el nominal da **0,65** sobre Less y **2,23** sobre Greater (frente a Same): léidos en el orden, ambos empujan **hacia arriba** (más Greater, menos Less), justo lo que resume el OR acumulado **2,46**. Para **age** ocurre lo simétrico (**1,01** y **0,98** → empuje **hacia abajo**, hacia Less; acumulado **0,98**). Como las dos lecturas nominales son **coherentes con el orden** en ambas covariables, el resumen ordinal

es fiel y la Figura 23 se parece a la Figura 22; si un OR claramente > 1 conviviera con otro < 1 **incoherentes** con el orden, el ordinal estaría forzando —y el **test de líneas paralelas** lo delataría—.

i Puente a la Unidad 4

La **Unidad 4** introduce los **efectos aleatorios**, sobre la respuesta **binaria**, que es donde los desarrollamos. Añadirlos a un **ordinal** como este es una extensión directa (un *ordinal mixto*, con `ordinal::clmm`): cada grupo desvía los umbrales y el resto del modelo no cambia. El hilo ordinal, además, reaparece en la **Unidad 5**: modelar el **tiempo discreto a un evento** es, de hecho, un ordinal *continuation-ratio*, allí con **fragilidad** de centro.

! Tarea del estudiante (1.3) — resuelve las Preguntas 3 y 4 del caso

Sobre la cohorte de tu equipo, responde dos preguntas del cliente (**Unidad 6**).

Pregunta 3 — tipo de fractura (`clase_nom`).

1. **Identifica y ajusta (RA3).** Reconoce que el tipo es una respuesta **nominal** (sin orden) y ajusta el modelo adecuado (`nnet::multinom`) con `x1` y `x2`.
2. **Interpreta y predice.** Lee los efectos como **odds ratios relativos** frente a la categoría base (¿hacia qué tipo empuja cada factor?), predice el tipo más probable para un par de perfiles y evalúa (p. ej. **Brier multiclase**).

Pregunta 4 — gravedad (`sever_ord`).

3. **Identifica y ajusta (RA3).** Reconoce que la gravedad es una respuesta **ordinal** y ajusta el modelo de **odds proporcionales**; comprueba el supuesto de proporcionalidad.
4. **Interpreta y responde.** Lee el **odds ratio acumulado** común (¿el tratamiento hace las fracturas menos graves?) y responde al cliente.

Apoyo — control de calidad (RA2, RA4). Como técnica de bondad de ajuste, agrupa la binaria de la Pregunta 1 en su versión **binomial** y aplica el **contraste**² (deviance y Pearson), interpretándolo y discutiendo el efecto del tamaño de celda.

4 · Efectos Aleatorios y Modelos Mixtos

Hasta aquí, cada coeficiente ha sido un **efecto fijo**: un único número válido para toda la población. Pero los datos rara vez son una masa homogénea —los pacientes se agrupan en centros, los alumnos en aulas, las medidas repetidas en cada sujeto—, y los grupos **difieren entre sí**, tanto en su nivel de partida como en cómo responden a las covariables. Ignorar ese

agrupamiento (o gastarse un parámetro por grupo) lleva a problemas conocidos; los **efectos aleatorios** son la vía intermedia: modelar la **variación entre grupos** como una distribución de la que cada grupo es una muestra. Es el primero de los dos hilos en espiral del curso —los efectos mixtos—, que abrimos aquí sobre el GLMM logístico y reactivaremos en cada caso.

En nuestros datos ese agrupamiento tiene nombre: la variable **centro**. Tanto en **GLOW** como en la **cohorte** simulada, cada paciente pertenece a un centro hospitalario, y los centros **no son intercambiables**: difieren en **criterios diagnósticos** —cómo se define y registra una fractura, qué umbrales se aplican— y en **factores de entorno** ligados a su **ubicación** —perfil de las pacientes, exposición, hábitos, recursos del área—. Dos mujeres con la misma edad y los mismos antecedentes pueden, por tanto, tener un riesgo algo distinto según **dónde** se las atienda.

En la Figura 24 se muestran las diferencias entre centros respecto a las tasas de fractura, en las bases de datos **cohorte** y **GLOW**. De partida, se aprecia cierta variabilidad de base entre los centros, mucho más notoria en **cohorte** que en **GLOW**.

```
library(patchwork)
lims      <- c(0, 0.62)
col_coh   <- "#2c7fb8"  # azul   → cohorte
col_glow  <- "#e6810a"  # naranja → GLOW

prop_coh <- cohorte |>
  group_by(centro) |>
  summarise(prop = mean(ever), n = n(), .groups = "drop")
p_coh <- ggplot(prop_coh, aes(reorder(factor(centro), prop), prop)) +
  geom_hline(yintercept = mean(cohorte$ever), linetype = "dashed", color = "grey50") +
  geom_col(fill = col_coh) +
  coord_flip() + scale_y_continuous(limits = lims) +
  labs(x = "Centro", y = "Proporción de fractura (ever)", title = "Cohorte (24 centros)")

prop_glow <- glow |>
  mutate(frac01 = as.integer(fracture == "Yes")) |>
  group_by(site_id) |>
  summarise(prop = mean(frac01), n = n(), .groups = "drop")
p_glow <- ggplot(prop_glow, aes(reorder(factor(site_id), prop), prop)) +
  geom_hline(yintercept = mean(glow$fracture == "Yes"), linetype = "dashed", color = "grey50") +
  geom_col(fill = col_glow) +
  coord_flip() + scale_y_continuous(limits = c(0, 0.35)) +
  labs(x = "Centro (site_id)", y = "Proporción de fractura", title = "GLOW (6 centros)")

p_coh + p_glow
```

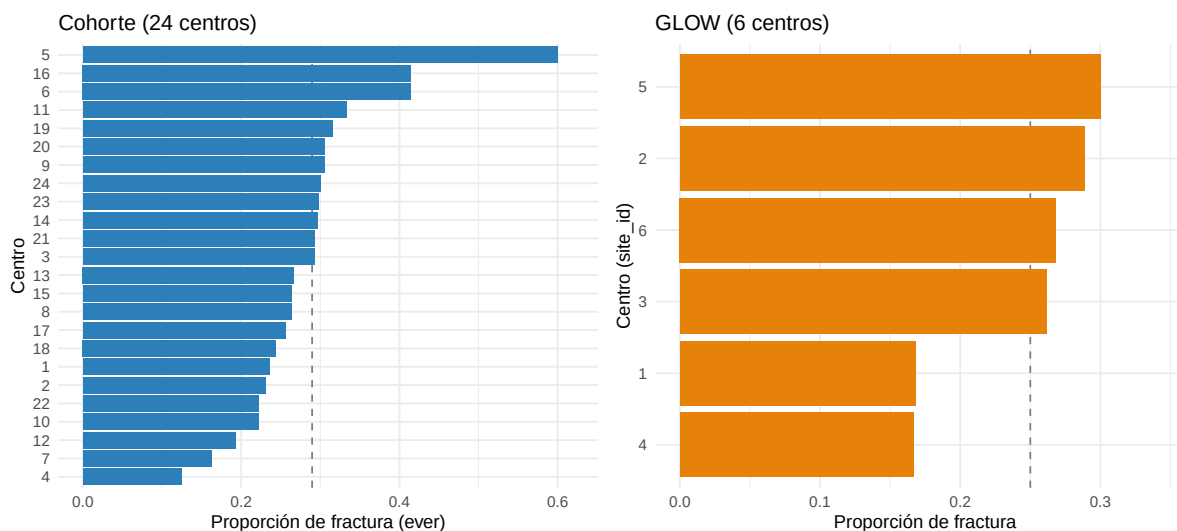


Figura 24: Proporción observada de fractura por centro. Izquierda (azul): cohorte simulada (24 centros). Derecha (naranja): GLOW (6 centros, datos reales). Línea discontinua: proporción global. Ambos paneles comparten el eje de proporción: la cohorte se despliega ($\sim 0,12$ a $\sim 0,60$; los centros estructuran el riesgo), mientras que en GLOW los seis se apiñan (el centro apenas aporta, ICC $0,7\%$). El color distingue cada conjunto, no codifica ninguna variable.

Ahora bien, el centro **no nos interesa como predictor**: no queremos —ni tendría sentido clínico— predecir si una paciente se fracturará **según a qué hospital pertenezca**, ni en los seis centros de GLOW, ni en los veinticuatro de la cohorte, pues no son un fin en sí mismos; son una **muestra** de los muchos centros posibles. Lo que **sí** debemos reconocer es que el centro induce **agrupamiento**: las pacientes de un mismo centro se **parecen más entre sí** de lo que explicaría el azar —no son del todo independientes— y, a la vez, hay **diferencias sistemáticas entre centros** —unos con un riesgo basal más alto que otros—. Gastar un coeficiente **fijo** por centro malgastaría parámetros y no generalizaría a centros nuevos; ignorarlo falsearía la precisión.

La vía adecuada es modelar esa variación **entre centros** como **efectos aleatorios** —el objeto de esta unidad—: estimamos **cuánto varían** los centros (una varianza), no el valor concreto de cada uno. Trabajamos sobre la **cohorte** simulada, cuya verdad conocida permite comprobar si lo recuperamos bien.

i Sobre esta unidad

RA implicados:

- **RA6 • Efectos aleatorios.** Ajustar e interpretar GLMM (intercepto y pendiente): componentes de varianza, ICC, *shrinkage* y condicional vs marginal.

- **RA4 · Diagnóstico y bondad de ajuste** (del mixto). Residuos simulados (DHARMa), R^2 marginal/condicional y solidez del ajuste.

(Agresti, 2015; Bates et al., 2015)

4.1 Qué son y para qué sirven

La idea central es un cambio de pregunta. Un efecto **fijo** responde «¿cuál es el efecto?» con **un número para toda la población**. Un efecto **aleatorio** no estima un número por grupo, sino la **distribución** de la que esos números proceden: en lugar de «¿cuál es el efecto del grupo 7?», pregunta «¿cuánto varían los efectos entre grupos?» —su **varianza**—. Se modela la **dispersión** entre grupos (en nuestro caso, **centros**), no cada grupo por separado.

Dos cosas pueden variar de un grupo a otro:

- El **nivel de base** → un **intercepto aleatorio** u_{0j} : cada centro parte de un riesgo basal distinto (la curva se **desplaza** arriba o abajo).
- La **sensibilidad a una covariable** → una **pendiente aleatoria** u_{1j} : en cada centro, el efecto de cierta covariable continua es distinto (la curva se **inclina** de forma diferente).

¿Para qué sirven? Cuatro cosas concretas:

1. **Respetar la no independencia**. Las observaciones de un mismo grupo se parecen entre sí; ignorarlo da errores estándar **demasiado pequeños** (falsa precisión). El efecto aleatorio modela esa correlación intragrupo correctamente.
2. **Partial pooling (tomar prestado)**. Las estimaciones de cada grupo se **encogen** hacia la media global, tanto más cuanto menos datos aporta el grupo: estabiliza los grupos pequeños sin imponer a todos un único valor rígido.
3. **Cuantificar la heterogeneidad**. Las **varianzas** σ^2 son, ellas mismas, la respuesta a una pregunta sustantiva: ¿cuánto difieren los grupos en su nivel, o en el efecto de una covariable?
4. **Generalizar**. Al tratar los grupos como una **muestra** de una población de grupos, las conclusiones valen para grupos **no observados**, no solo para los de la muestra.

En una frase: el efecto fijo dice *cuánto*; el efecto aleatorio dice *cuánto varía ese cuánto entre grupos* —y lo resume en una varianza—.

Los efectos aleatorios se pueden incorporar en un modelo lineal (generalizado) a través de la interceptación, de la pendiente o de ambas. Veamos sus implicaciones y las modelizaciones asociadas.

4.2 Intercepto aleatorio

Cuando hablamos de una afectación en la interceptación, nos estamos refiriendo a la modelización del **nivel de base**. Para tratar este nivel de base, considerando un posible efecto de agrupación relativo a **centro** en nuestra base de datos, hay tres estrategias:

- **Ignorarlo** (*complete pooling*): un solo intercepto para todos. Subestima la incertidumbre y puede sesgar los efectos.
- **Un parámetro por centro** (*no pooling*): tantos interceptos como centros. Gasta muchos grados de libertad y no generaliza a centros nuevos.
- **Intercepto aleatorio** (*partial pooling*): los interceptos de centro son **extracciones de una distribución**, $u_j \sim \mathcal{N}(0, \sigma_u^2)$; no estimamos cada u_j , sino su **varianza** σ_u^2 , asumiendo que todos están centrados en torno a una interceptación media global (y por lo tanto estas desviaciones de la interceptación global tienen media cero).

El modelo, para la binaria de la cohorte, es un **GLMM logístico**:

$$\text{logit Pr}(\text{ever}_{ij} = 1) = \beta_0 + \beta_1 x_{1ij} + \beta_2 x_{2ij} + u_j, \quad u_j \sim \mathcal{N}(0, \sigma_u^2).$$

Ese término u_j no es una abstracción: ya teníamos su **huella visual** en la cohorte. La Figura 24 mostraba que la proporción de fractura **varía de un centro a otro** —de $\approx 0,12$ a $\approx 0,60$ (en torno a una media global explícita con la vertical punteada)—, justo la dispersión de **niveles basales** que u_j recoge (y que en GLOW era mínima). Hay que tener en cuenta que en general, una proporción por centro mezclaría el nivel basal con la **composición** de covariables del centro; si estas estuvieran correladas con **centro**, el gráfico estaría confundiendo efectos.

En **forma matricial** —la notación habitual de los modelos mixtos, que conviene reconocer desde ya—, el predictor lineal de todas las observaciones se escribe

$$\eta = \mathbf{X}\beta + \mathbf{Z}\mathbf{u}, \quad \mathbf{u} \sim \mathcal{N}(\mathbf{0}, \mathbf{G}), \quad g(\mu) = \eta,$$

donde $\mathbf{X}$ ($n \times p$) y β son el diseño y los coeficientes **fijos**; $\mathbf{Z}$ ($n \times q$) y $\mathbf{u}$ son el diseño y los efectos **aleatorios**, con $\mathbf{G}$ su matriz de covarianzas; y g es el enlace (aquí, logit).

Para el **intercepto aleatorio**, $\mathbf{u} = (u_1, \dots, u_J)^\top$ son los J interceptos de centro, $\mathbf{Z}$ es la matriz **indicadora** que asigna cada observación a su centro, y $\mathbf{G} = \sigma_u^2 \mathbf{I}_J$. La parte $\mathbf{X}\beta$ es lo de siempre; lo nuevo es el término $\mathbf{Z}\mathbf{u}$.

Intercepto aleatorio en cohorte

Para interpretar los resultados que genera un modelo mixto, nada mejor que ajustarlo en unos datos y ver qué produce. Utilizamos **cohorte** para intentar predecir la respuesta binaria relativa a si se dió o no una fractura, **evento**, en función de la fragilidad ósea (x_1) y el

tratamiento preventivo (x2) recibido por cada paciente, teniendo en cuenta el **centro** clínico de referencia.

La primera tarea pasa pues, por inspeccionar los datos. Así, acompañamos al gráfico inicial en el que representamos las probabilidades de fractura por centros en Figura 24, con la Figura 25, donde mostramos la relación de la respuesta con las predictivas x1 y x2.

```
cohorte |>
  mutate(Desenlace = factor(evento, levels = c(0, 1), labels = c("Sin fractura", "Fractura"),
    Tratamiento = factor(x2, levels = c(0, 1), labels = c("No", "Sí"))) |>
  ggplot(aes(x = x1, y = Desenlace, fill = Tratamiento)) +
  geom_boxplot(alpha = 0.75, outlier.alpha = 0.25) +
  labs(x = "Fragilidad ósea (x1, en z)", y = "Desenlace", fill = "Tratamiento")
```

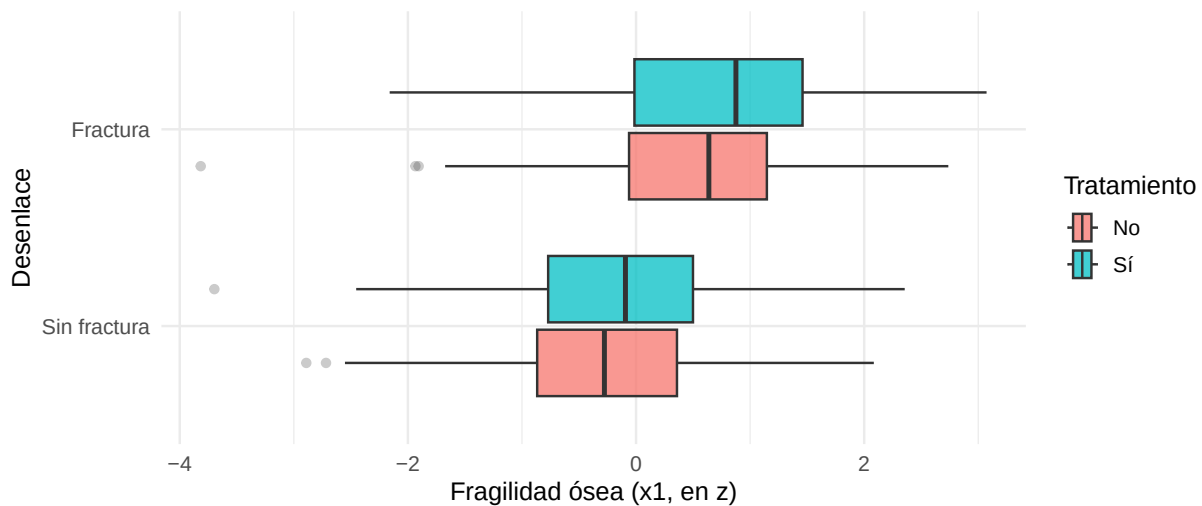


Figura 25: Fragilidad ósea (x1) según el desenlace, por brazo de tratamiento (x2). Quienes fracturan presentan mayor fragilidad; entre las tratadas, las que llegan a fracturar son aún más frágiles (el desplazamiento de umbral propio de un factor protector).

El patrón en la Figura 25 es **nítido**: las cajas de **Fractura** están claramente **desplazadas a la derecha** —mayor fragilidad— respecto a las de **Sin fractura** (la mediana de x1 pasa de $\approx -0,19$ a $\approx 0,69$), que es el efecto de la fragilidad sobre el riesgo. Y dentro de cada desenlace, las **tratadas** (x2 = Sí) quedan algo **más a la derecha** que las no tratadas: entre las que **fracturan**, las tratadas son **aún más frágiles** (mediana $\approx 0,88$ frente a $\approx 0,64$). Tiene sentido —como el tratamiento **protege**, solo «caen» las de fragilidad más alta—: es el **desplazamiento de umbral** propio de un factor protector.

Ajustemos pues, el modelo de interceptación aleatoria, pero también el modelo *pooled*, en el que ignoramos el centro, pues de este modo podremos valorar el efecto de incorporar el **centro**.

💡 En R. Ajustar un GLMM de intercepto aleatorio

Mismo `glmer` que un `glm`, declarando el efecto aleatorio con `( ... | grupo )`. Para un **intercepto aleatorio** por centro:

```
library(lme4)
m_int <- glmer(ever ~ x1 + x2 + (1 | centro), family = binomial, data = cohorte)
summary(m_int)          # añade el bloque 'Random effects' con la varianza de centro
```

La sintaxis completa —pendientes, cruzados, anidados— en la [Sección 4.5](#).

Ajuste de los modelos pooled e intercepto aleatorio

```
library(lme4)
m_pool <- glm (ever ~ x1 + x2,                      family = binomial, data = cohorte) # compl
m_int  <- glmer(ever ~ x1 + x2 + (1 | centro),      family = binomial, data = cohorte) # inte

summary(m_pool)
```

Call:

```
glm(formula = ever ~ x1 + x2, family = binomial, data = cohorte)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-0.84845	0.10582	-8.018	1.08e-15 ***
x1	1.01615	0.09679	10.499	< 2e-16 ***
x2	-0.63341	0.17022	-3.721	0.000198 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 1055.58 on 876 degrees of freedom
Residual deviance: 903.66 on 874 degrees of freedom
AIC: 909.66

Number of Fisher Scoring iterations: 4

Cómo se lee la salida de `glmer`

El resumen del ajuste `glmer` es:

```
summary(m_int)
```

```
Generalized linear mixed model fit by maximum likelihood (Laplace
Approximation) [glmerMod]
Family: binomial ( logit )
Formula: ever ~ x1 + x2 + (1 | centro)
Data: cohorte
```

AIC	BIC	logLik	-2*log(L)	df.resid
899.0	918.1	-445.5	891.0	873

```
Scaled residuals:
```

Min	1Q	Median	3Q	Max
-2.0791	-0.5968	-0.3734	0.6784	10.9504

```
Random effects:
```

Groups	Name	Variance	Std.Dev.
centro	(Intercept)	0.1959	0.4426

```
Number of obs: 877, groups: centro, 24
```

```
Fixed effects:
```

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-0.8955	0.1422	-6.297	3.04e-10 ***
x1	1.0803	0.1017	10.620	< 2e-16 ***
x2	-0.6605	0.1741	-3.793	0.000149 ***

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Correlation of Fixed Effects:
```

```
(Intr) x1
x1 -0.189
x2 -0.422 -0.140
```

El bloque de **efectos fijos** es idéntico al de `glm` —coeficientes con su error estándar, z y p , que se interpretan igual (log-odds, $e^\beta = \text{OR}$)—.

La novedad es el bloque **Random effects**, encima: ahí están las **varianzas** y desviaciones típicas de los efectos aleatorios —en `m_int`, una sola fila, `centro (Intercept)`— y el número de grupos. `glm` no tiene ese bloque: estima un único intercepto y nada más.

Con el ajuste delante, recorreremos las **tres lecturas** de un intercepto aleatorio.

1. Componentes de varianza

La varianza σ_u^2 resume cuánto varían los grupos entre sí. Surge un nuevo índice, común en modelos mixtos: el ICC. En la escala latente logística, el **coeficiente de correlación intraclase (ICC)** mide qué fracción de la variación total ocurre **entre** centros.

El **coeficiente de correlación intraclase (ICC)** mide qué fracción de la variación total ocurre **entre** grupos (y no dentro de ellos); equivalentemente, es la **correlación** entre dos observaciones del **mismo** grupo. En un modelo con intercepto aleatorio, descompone la varianza en una parte **entre grupos** (σ_u^2) y una parte **dentro** (residual, σ_ε^2):

$$\text{ICC} = \frac{\sigma_u^2}{\sigma_u^2 + \sigma_\varepsilon^2} \in [0, 1].$$

ICC $\rightarrow 0$ indica grupos prácticamente indistinguibles (el agrupamiento apenas importa); ICC $\rightarrow 1$, grupos muy heterogéneos y observaciones fuertemente correladas dentro de cada uno.

En un **GLMM logístico** no hay una varianza residual libre: en la **escala latente**, el término de error sigue una logística estándar, cuya varianza es **fija**, $\pi^2/3 \approx 3,29$. Por eso el **ICC latente** se calcula como

$$\text{ICC} = \frac{\sigma_u^2}{\sigma_u^2 + \pi^2/3}.$$

Con enlace **probit** la varianza latente es 1, así que $\text{ICC} = \sigma_u^2/(\sigma_u^2 + 1)$.

💡 En R. Calcular el ICC de un GLMM

A mano, desde la varianza de **VarCorr**. En la escala latente de un GLMM **logístico** la varianza residual es fija, $\pi^2/3$ (con **probit**, 1):

```
vc <- as.data.frame(VarCorr(m_int))
s2_u <- vc$vcov[1] # sigma_u^2 (varianza del intercepto de centro)
s2_u / (s2_u + pi^2 / 3) # ICC latente (logit)
```

Con **performance**, directo —y devuelve dos versiones—:

```
library(performance)
icc(m_int)
```

- **Adjusted ICC**: $\sigma_u^2/(\sigma_u^2 + \sigma_{\text{resid}}^2)$, solo efectos aleatorios. Es el que **coincide** con la fórmula manual y el que interpretamos como «cuánto pesa el centro frente a las diferencias entre pacientes».
- **Conditional ICC**: mete además la varianza de los **efectos fijos** en el denominador, así que sale **menor**.

Vale limpio para un **intercepto aleatorio**; con **pendiente aleatoria** el ICC depende de la covariable y conviene usar `performance::icc()`.

En nuestro ejemplo, el ICC responde a una pregunta concreta: de toda la variación en el riesgo de fractura, ¿cuánta se debe a en qué clínica te atienden, y cuánta a las diferencias entre pacientes (su fragilidad, su tratamiento, su azar individual)?

Comparemos pues, la varianza del efecto aleatorio y el ICC, y puesto que los datos los hemos simulado con unos valores dados, podemos contrastar las estimaciones con la verdad del DGP (*Data Generating Process*).

```
# matriz de varianzas-covarianzas de los efectos aleatorios
vc <- as.data.frame(VarCorr(m_int))
vc$vcov[1]      # varianza  sigma_u^2
```

```
[1] 0.1959143
```

```
vc$sdcor[1]     # desviación sigma_u
```

```
[1] 0.4426221
```

```
# y de ahí el ICC latente:
vc$vcov[1] / (vc$vcov[1] + pi^2/3)
```

```
[1] 0.05620383
```

```
# verdad simulada
attr(cohorte, "verdad")$binaria[c("sigma_u", "icc_latente")]
```

```
$sigma_u
[1] 0.7
```

```
$icc_latente
[1] 0.1296342
```

Aquí el modelo estima $\hat{\sigma}_u \approx 0,44$ ($\text{ICC} \approx 0,06$), **por debajo** de la verdad del DGP ($\sigma_u = 0,70$, $\text{ICC} = 0,13$): la **subestimación** típica de las varianzas con respuesta binaria y pocos grupos (24).

Es pues, un resultado que merece comentarse: el ICC cuantifica cuánta de la variación del riesgo se debe al centro (6–13 % aquí) frente a las diferencias entre pacientes (el resto). El centro pesa poco, pero no nada: conviene modelarlo —para no falsear la precisión— aunque el grueso del riesgo se juegue a nivel de paciente.

2. Condicional vs marginal

Con un enlace **no lineal** como el logit, el efecto de una covariable tiene **dos versiones que no coinciden** (en un modelo lineal sí lo harían). Geométricamente se entiende bien con la Figura 27: cada centro tiene su propia curva logística con la **misma pendiente** —el efecto **condicional**, «para un centro dado», que es el que estima **glmer**— pero **desplazada** por su intercepto u_j . Al **promediar** esas curvas sobre la población de centros, la sigmoide sale **más plana**, y su pendiente —el efecto **marginal**, poblacional— es **menor en magnitud**.

Comparemos pues los coeficientes, de nuevo entre los dos modelos y con la verdad del DGP.

```
# verdad
attr(cohorte,"verdad")$binaria[c("beta")]
```

```
$beta
  x1    x2
0.85 -0.65
```

```
# coeficientes estimados
sp <- coef(summary(m_pool)) # tabla de efectos fijos del pooled
si <- coef(summary(m_int))  # tabla de efectos fijos del mixto

coefs <- rbind(
  data.frame(modelo = "pooled (glm)", term = rownames(sp),
             estimate = sp[, "Estimate"], se = sp[, "Std. Error"]),
  data.frame(modelo = "mixto (glmer)", term = rownames(si),
             estimate = si[, "Estimate"], se = si[, "Std. Error"])
)

library(ggplot2)
ggplot(coefs, aes(estimate, term, color = modelo)) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "grey60") +
```

```
geom_pointrange(aes(xmin = estimate - 1.96 * se, xmax = estimate + 1.96 * se),
  position = position_dodge(width = 0.5)) +
labs(x = "Coeficiente (log-odds),  $\pm 1,96 \cdot EE$ ", y = NULL, color = NULL)
```

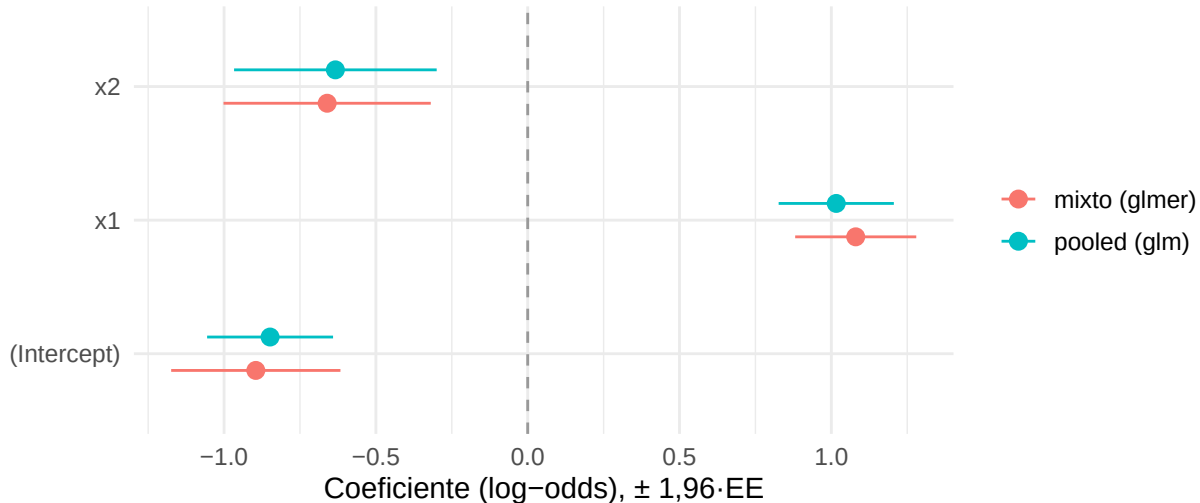


Figura 26: Efectos fijos del modelo *pooled* (glm) frente al mixto de intercepto aleatorio (glmer): punto = coeficiente (log-odds) y bigotes = $\pm 1,96 \cdot EE$ (intervalo de Wald al 95 %). Dos lecturas a la vez: los coeficientes del mixto son algo **mayores en magnitud** (la des-atenuación marginal→condicional) y sus bigotes, **más anchos** —errores estándar honestos que tienen en cuenta el agrupamiento por centro, que el *pooled* ignoraba—.

Se ve en los números: `m_int` da $x_1 = 1,08$, $x_2 = -0,66$, algo **mayores** que el *pooled* ($x_1 = 1,02$, $x_2 = -0,63$) —la atenuación marginal→condicional, leve aquí porque σ_u es pequeño, $\beta_{\text{marg}} \approx \beta_{\text{cond}} / \sqrt{1 + 0,346 \sigma_u^2}$ —. Y, sobre todo, los **errores estándar crecen** en el mixto (el del intercepto pasa de 0,106 a 0,142): `m_int` **tiene en cuenta** el agrupamiento que el *pooled* ignoraba, que daba una precisión **falsamente alta**.

```
b0 <- 0; b1 <- 1.5; sigma_u <- 2 # sigma_u exagerado para que el aplanamiento se
x <- seq(-6, 6, length.out = 200)

# Familia de curvas condicionales: varios centros (distinto u_j), MISMA pendiente
cond_fam <- tidyr::expand_grid(x = x, u = c(-4, -2, 0, 2, 4)) |>
  dplyr::mutate(p = plogis(b0 + b1 * x + u))

# Curva marginal: promedio de plogis(b0 + b1*x + u) sobre u ~ N(0, sigma_u^2)
ug <- seq(-4 * sigma_u, 4 * sigma_u, length.out = 401)
```

```

w <- dnorm(ug, 0, sigma_u); w <- w / sum(w)
destacadas <- dplyr::bind_rows(
  tibble::tibble(x = x, p = plogis(b0 + b1 * x),
                 tipo = "condicional (un centro)"),
  tibble::tibble(x = x, p = sapply(x, \(xi) sum(w * plogis(b0 + b1 * xi + ug))),
                 tipo = "marginal (promedio)")
)

ggplot() +
  geom_line(data = cond_fam, aes(x, p, group = u), color = "grey75", linewidth = 0.5) +
  geom_line(data = destacadas, aes(x, p, color = tipo), linewidth = 1.2) +
  scale_color_manual(values = c("condicional (un centro)" = "#2c7fb8",
                                "marginal (promedio)" = "#e6810a")) +
  labs(x = "covariable", y = "Probabilidad de fractura", color = NULL) +
  theme(legend.position = "top")

```

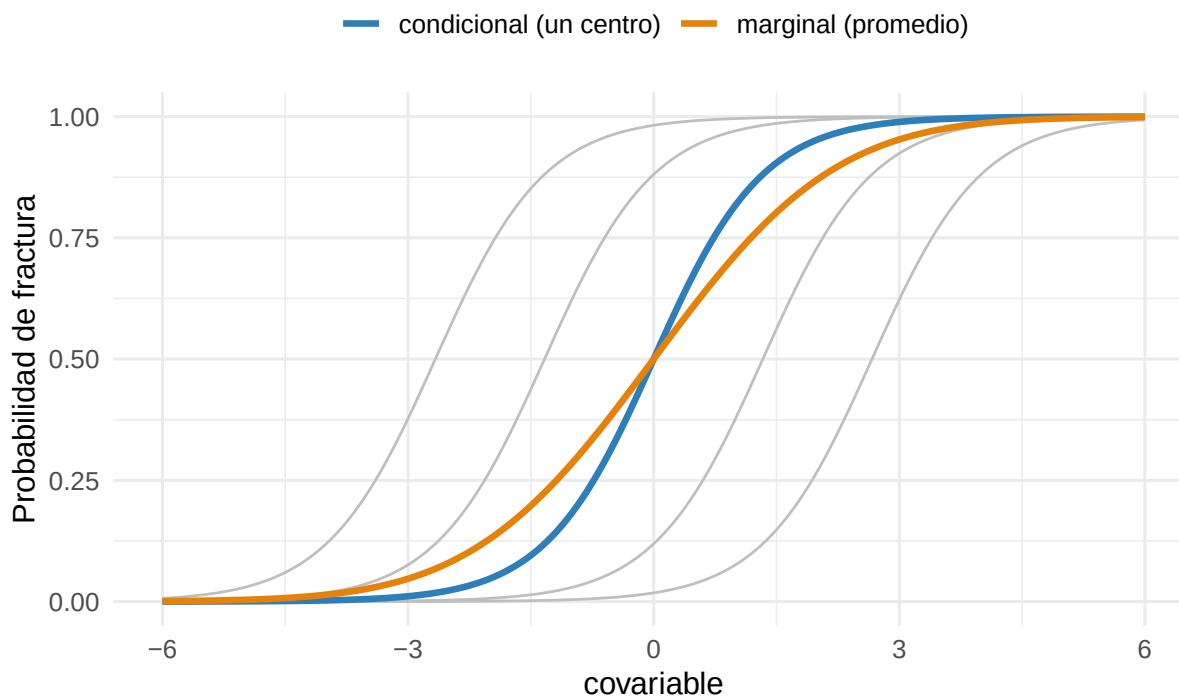


Figura 27: Por qué *condicional* *marginal* con enlace no lineal. En gris, varias curvas logísticas con la misma pendiente desplazadas por el intercepto de centro; en azul, una de ellas (un centro típico: el efecto condicional, lo que estima glmer). Su promedio sobre los centros (naranja) es la curva marginal, más plana: la pendiente poblacional es menor que la de cada centro. El efecto se ha exagerado (σ_u grande) para hacerlo visible; con σ_u pequeño, como en la cohorte, ambas curvas casi coinciden.

3. Encogimiento o *shrinkage* (*partial pooling*).

Los interceptos de centro $\hat{u}_j$ **no** son las desviaciones crudas de cada centro (las del *no pooling*), sino un **compromiso** entre los **datos del propio centro** y la **media global** —el 0 hacia el que $u_j \sim \mathcal{N}(0, \sigma_u^2)$ los empuja—, ponderado por **cuánta información** aporta el centro: con muchos datos apenas se encoge; con pocos, su estimación ruidosa se **arrima a 0**, tomando fuerza prestada del conjunto.



En R. Obtener los efectos aleatorios estimados (`ranef`)

`ranef` devuelve los efectos aleatorios **predichos** (las desviaciones $\hat{u}_j$ de cada grupo, ya **encogidas** hacia 0):

```
ranef(m_int)$centro # data.frame: una fila por centro, columna (Intercept) =
```

Para acompañarlos de su **incertidumbre** (y poder dibujar intervalos), pide las varianzas condicionales y pásalo a `data.frame`:

```
re <- as.data.frame(ranef(m_int, condVar = TRUE))
# columnas: grpvar (factor de agrupamiento), term, grp (centro), condval (u_j), condsd (
```

Con eso, el *caterpillar* es inmediato (puntos = `condval`, bigotes = $\pm 1.96 \cdot \text{condsd}$):

```
library(ggplot2)
ggplot(re, aes(reorder(grp, condval), condval)) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_pointrange(aes(ymin = condval - 1.96 * condsd, ymax = condval + 1.96 * condsd)) +
  coord_flip() +
  labs(x = "Centro", y = "Intercepto aleatorio (log-odds)")
```

Dos matices: `ranef` da **desviaciones** respecto a la media (centradas en 0); si quieres el **coeficiente por grupo** completo (fijo + aleatorio), usa `coef(m_int)$centro`.

Y en un modelo con **pendiente aleatoria** (`(1 + x1 | centro)`), `ranef(m_int)$centro` trae **dos** columnas —(Intercept) y `x1`—, una desviación por efecto aleatorio.

El *caterpillar* o gráfico de oruga en la Figura 28 lo hace visible:

```
re_coh <- as.data.frame(ranef(m_int, condVar = TRUE)) # columnas: grp, condval, condsd
ggplot(re_coh, aes(reorder(grp, condval), condval)) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_pointrange(aes(ymin = condval - 1.96 * condsd,
                      ymax = condval + 1.96 * condsd), linewidth = 0.3) +
```

```
coord_flip() +
labs(x = "Centro", y = "Intercepto aleatorio (log-odds)")
```

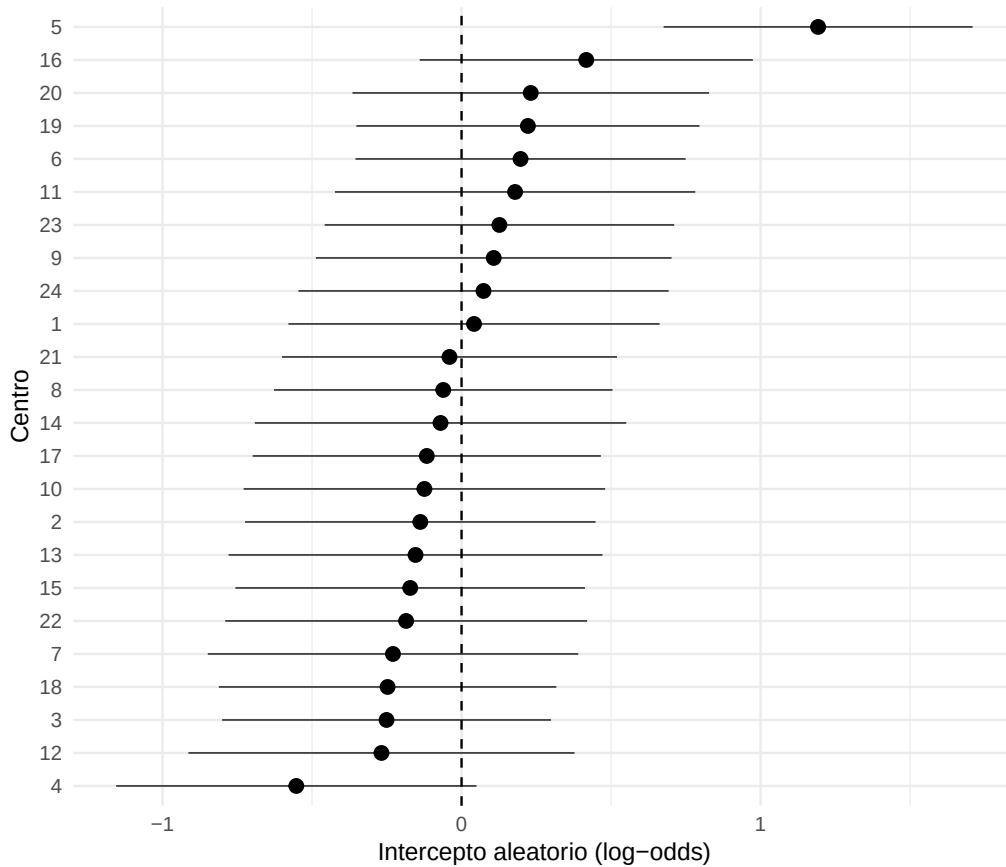


Figura 28: Efecto de centro modelizado en la cohorte: interceptos aleatorios de los 24 centros (desviación en log-odds respecto al riesgo medio) con IC 95%, ordenados. La dispersión del conjunto es lo que el mixto resume en σ_u ; los centros cuyo intervalo no cruza el 0 se separan del promedio.

En la Figura 28 cada punto es la desviación estimada de un centro respecto al riesgo medio; el **shrinkage** acerca a 0 a los centros con **intervalos más anchos** (menos datos), y la **amplitud** del conjunto es la traducción visual de $\hat{\sigma}_u$. Un centro con dos pacientes no dispara un intercepto extremo: se estabiliza.

Para comparar finalmente los dos modelos, con y sin efecto aleatorio para centro, nos basamos en las medidas residuales que obtenemos:

```
# 1) Varianza entre centros: sigma_u^2 (y sigma_u)
vc <- as.data.frame(VarCorr(m_int))
s2_u <- vc$vcov[1]          # sigma_u^2
s_u <- vc$sdcor[1]          # sigma_u
c(sigma2_u = s2_u, sigma_u = s_u)
```

```
sigma2_u  sigma_u
0.1959143 0.4426221
```

```
# 2) Varianza residual (latente, fija) e ICC
resid_var <- pi^2 / 3      # ~3.29 (en logit)
icc <- s2_u / (s2_u + resid_var)
c(var_residual = resid_var, ICC = icc)
```

```
var_residual  ICC
3.28986813    0.05620383
```

```
# (equivalente con paquete: performance::icc(m_int) -> 'adjusted ICC')
```

```
# 3) AIC de los dos modelos (menor = mejor)
AIC(m_pool, m_int)          # data.frame con df y AIC de cada uno
```

```
      df      AIC
m_pool 3 909.6627
m_int  4 898.9709
```

¿Por qué preferimos el intercepto aleatorio? Lo dicen sus propios resultados, sin recurrir a la verdad del proceso generador. El modelo estima una **varianza entre centros no nula**, $\hat{\sigma}_u^2 \approx 0,19$ ($\hat{\sigma}_u \approx 0,44$): hay **heterogeneidad real** entre clínicas, justo lo que el *pooled* da por inexistente ($\sigma_u^2 = 0$). En la escala latente, esa varianza convive con una **residual fija** $\pi^2/3 \approx 3,29$, de modo que el **ICC** $\approx 0,06$: alrededor de un **6 %** de la variación del riesgo es **entre centros** y el **94 %** restante, **entre pacientes**. Es una fracción **modesta pero suficiente** para invalidar el supuesto de independencia del *pooled* (de ahí que sus errores estándar fueran demasiado pequeños). Y la comparación formal cierra el argumento: el **AIC del mixto es menor** que el del *pooled* (898.97 frente a 909.66), así que el único parámetro extra —la varianza σ_u^2 — **compensa con creces**. En suma: aunque el grueso del riesgo se juega a nivel de paciente, existe una capa de variación **entre clínicas que es real**; modelarla

(mixto) da una descripción más fiel y una inferencia honesta, mientras que ignorarla (*pooled*) infla la precisión.

Queda abierta una pregunta: ¿pesa la **fragilidad ósea** lo mismo en todas las clínicas, o su efecto sobre el riesgo de fractura es **más fuerte en unas que en otras**? Lo vemos a continuación la [Sección 4.3](#).

4.3 Pendiente aleatoria: cuando el efecto varía por grupo

El intercepto aleatorio deja que el **nivel** cambie por centro, pero asume que el **efecto** de cada covariable es el mismo en todos. A menudo no lo es: la edad, por ejemplo, puede pesar más en unos centros que en otros. Eso se modela con una **pendiente aleatoria**, añadiendo una desviación de grupo al coeficiente:

$$\text{logit Pr}(\text{ever}_{ij} = 1) = \beta_0 + \beta_1 x_{1ij} + \beta_2 x_{2ij} + u_{0j} + u_{1j} x_{1ij}, \quad \begin{pmatrix} u_{0j} \\ u_{1j} \end{pmatrix} \sim \mathcal{N}(\mathbf{0}, \Sigma).$$

Cómo se lee:

- β_1 es el efecto **medio** de x_1 en la población de centros; el efecto **en el centro** j es $\beta_1 + u_{1j}$.
- $\sigma_{u_1}^2$ (la varianza de las pendientes) dice **cuánto varía** ese efecto entre centros: si es ≈ 0 , x_1 actúa casi igual en todos; si es grande, en unos centros pesa mucho y en otros poco.
- La **correlación** ρ entre u_{0j} y u_{1j} liga nivel y sensibilidad: $\rho > 0$ indica que los centros con mayor riesgo basal tienen, además, un efecto de x_1 más fuerte.

Las pendientes aleatorias son **más exigentes** con los datos (necesitan suficientes grupos y variación de x_1 dentro de cada uno) y es frecuente que el ajuste no converja o resulte **singular**; por eso se empieza por el intercepto y se añade la pendiente solo si está justificada.

En forma matricial, una pendiente aleatoria solo **añade columnas a $\mathbf{Z}$** (una por efecto aleatorio y grupo) y agranda **$\mathbf{G}$** , que pasa a tener un bloque Σ de 2×2 por grupo.

La pendiente aleatoria admite **dos formas**, y conviene distinguirlas:

- **Solo pendiente**, (0 + x1 | centro): la **sensibilidad** a x1 varía por centro, pero el **nivel basal** es común a todos. Hipótesis **poco habitual** —rara vez varía el efecto sin variar el nivel— y a menudo da ajustes **singulares**.
- **Intercepto + pendiente**, (1 + x1 | centro): **ambos** varían y pueden estar **correlados** (la ρ entre u_{0j} y u_{1j}). Es la forma **estándar**: si dejamos variar la pendiente, lo natural es permitir también que el nivel cambie, y además, si tenemos ya indicios de interceptación aleatoria de un ajuste previo, lo natural es incorporar también la pendiente para comprobar si el ajuste mejora.

¿Tiene sentido en la cohorte? Ya previamente concluimos con el sentido de incluir intercepto aleatorio. Veamos si ganamos algo más al incorporar la pendiente. Para verlo, agrupamos `x1` en tramos y, en cada centro, calculamos la proporción de fracturas por tramo: si el **efecto** de `x1` variara por centro, las **pendientes** de esas líneas diferirían de un panel a otro.

```
library(tidyverse)
n_bins <- 4
br <- seq(min(cohorte$x1), max(cohorte$x1), length.out = n_bins + 1)
mid <- (head(br, -1) + tail(br, -1)) / 2 # punto medio de cada bin (posición
cohorte |>
  mutate(bin = cut(x1, breaks = br, include.lowest = TRUE), # bins de igual longitud
         x1_bin = mid[as.integer(bin)]) |>
  group_by(centro, x1_bin) |>
  summarise(prop = mean(evento), .groups = "drop") |> # proporción de fracturas
  ggplot(aes(x1_bin, prop, color = factor(centro), group = centro)) +
  geom_line(alpha = 0.5) +
  geom_point(size = 1.6, alpha = 0.8) +
  facet_wrap(~ centro, ncol = 6) + # 6 columnas × 4 filas
  labs(x = "x1 (fragilidad ósea)", y = "Proporción de fracturas") +
  guides(color = "none") # sin leyenda de centros
```

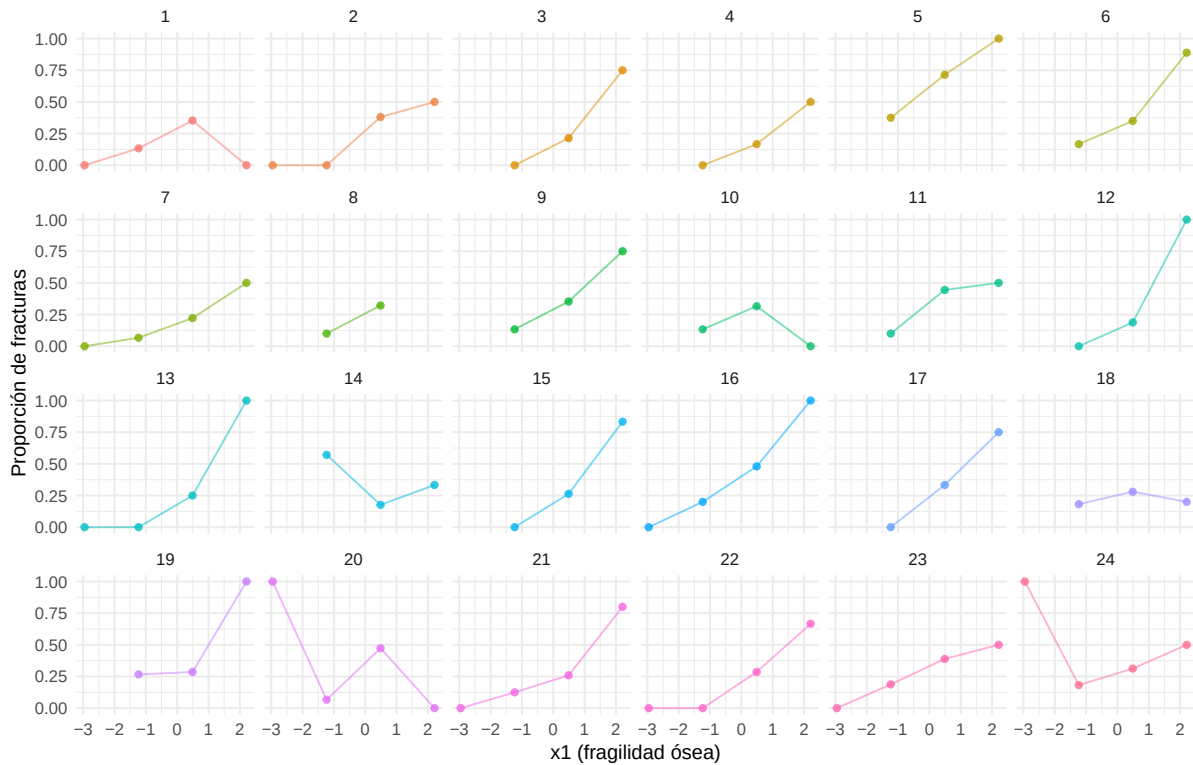


Figura 29: Proporción de fracturas frente a la fragilidad ósea (x_1), en bins de igual longitud, con una faceta por centro (rejilla 6×4). La tendencia creciente común refleja el efecto (fijo) de x_1 ; las diferencias de nivel entre paneles, el intercepto aleatorio. Con pocas pacientes por centro y bin las curvas son ruidosas, así que las diferencias de pendiente no son interpretables a ojo.

En la Figura 29 se aprecia con claridad una **tendencia creciente común** —el efecto de x_1 , compartido por todos los centros— y **diferencias de nivel** entre paneles (unos sistemáticamente más altos): la huella del **intercepto aleatorio**. Las diferencias de **pendiente** (algún panel parece plano o decreciente) son, en cambio, **difíciles de separar del ruido de muestreo**: con tan pocas pacientes por centro y tramo, cada punto se apoya en pocos casos y oscila con facilidad. A ojo no podemos decidir si hay pendientes aleatorias reales; lo **contrastaremos formalmente** a continuación, ajustando los modelos.



En R. Añadir una pendiente aleatoria

```
m_slope0 <- glmer(ever ~ x1 + x2 + (0 + x1 | centro), family = binomial, data = cohorte)
m_slope <- glmer(ever ~ x1 + x2 + (1 + x1 | centro), family = binomial, data = cohorte)
VarCorr(m_slope) # sigma_u0, sigma_u1 y su correlacion rho
```

Añadimos pues pendiente aleatoria al modelo.

```
m_slope <- glmer(ever ~ x1 + x2 + (1 + x1 | centro), family = binomial, data = cohorte)

VarCorr(m_slope)          # sigma_u0, sigma_u1 y su correlacion
```

Groups Name	Std.Dev.	Corr
centro (Intercept)	0.53421	
x1	0.39714	-0.797

En el modelo **intercepto + pendiente**, VarCorr da una desviación de pendiente $\hat{\sigma}_{u_1} \approx 0,40$ con una **correlación casi extrema** entre los efectos aleatorios (intercepto y pendiente) $\rho \approx -0,80$ —señales de un ajuste **inestable**, frecuente cuando la pendiente no aporta señal real—. La pregunta, entonces, no es *¿se puede?* sino *¿merece la pena?*; y eso se decide **comparando**.

La estrategia de selección, sistematizada.

Con efectos aleatorios se procede **de menos a más** y se contrasta cada salto:

1. **Empezar por el intercepto** ((1 | g)): casi siempre justificado cuando hay agrupamiento.
2. **Añadir la pendiente** ((1 + x | g)) **solo si** mejora el ajuste de forma apreciable.
3. **Comparar** por criterios de información (**AIC/BIC**, menor es mejor) y por **LRT** (**anova**) entre modelos anidados; vigilar avisos de **ajuste singular** o de no convergencia.
4. Ante la duda, **parsimonia**: el modelo más simple que describe bien los datos.

💡 En R. Comparar y elegir

```
AIC(m_pool, m_int, m_slope)  # escalera de complejidad (menor AIC, mejor)
anova(m_int, m_slope)        # LRT: ¿aporta la pendiente sobre el intercepto?
```

```
AIC(m_pool, m_int, m_slope)  # pooled < intercepto < intercepto + pendiente
```

	df	AIC
m_pool	3	909.6627
m_int	4	898.9709
m_slope	6	898.0549

```
anova(m_int, m_slope)        # LRT de la pendiente
```

```

Data: cohorte
Models:
m_int: ever ~ x1 + x2 + (1 | centro)
m_slope: ever ~ x1 + x2 + (1 + x1 | centro)
      npar    AIC    BIC  logLik -2*log(L)  Chisq Df Pr(>Chisq)
m_int      4 898.97 918.08 -445.49    890.97
m_slope    6 898.05 926.71 -443.03    886.05 4.916  2    0.08561 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

Los números **deciden**: el AIC apenas cambia al añadir la pendiente ($899,0 \rightarrow 898,1$, despreciable) y el **LRT** frente a `m_int` no es significativo ($p \approx 0,08$, 2 g.l.). Como el DGP **no** incluye pendiente aleatoria, ese $\hat{\sigma}_{u_1} = 0,40$ era **ruido sobreajustado**.

Conclusión: nos quedamos con el **intercepto aleatorio** (`m_int`) —mejora claramente al *pooled* (Sección 4.2) y la pendiente no se justifica—.

4.4 Bondad de ajuste y diagnóstico del modelo mixto

Ajustado el modelo, ¿**ajusta bien**? Un GLMM hereda las dificultades del GLM binario —el χ^2 de la deviance no es un test de bondad de ajuste válido con respuesta **individual** (Sección 3.2)— y añade la del efecto aleatorio: los residuos ordinarios (Pearson, deviance) se **enredan** con los u_j y dejan de ser informativos. El instrumental, por tanto, es el que afinamos en la **Unidad 2**, ahora **sobre el modelo mixto**: lo que se **extrae** del ajuste, los **residuos simulados**, una medida de **varianza explicada** adaptada a los mixtos y unos **chequeos del propio ajuste**.

Lo que se extrae del ajuste.

Casi todo el diagnóstico se apoya en cantidades que se sacan del modelo ya ajustado —coeficientes, varianzas, valores ajustados, predicciones y residuos—:

💡 En R. Extraer de un GLMM ajustado `m`

Coefficientes y varianzas:

- `fixef(m)` — efectos **fijos** (β); `vcov(m)`, su matriz de covarianzas (de donde salen los EE).
- `ranef(m)` — efectos **aleatorios** predichos (u_j , encogidos); `coef(m)` — el coeficiente **por grupo** (fijo + aleatorio).
- `VarCorr(m)` — **varianzas y correlaciones** de los efectos aleatorios (σ_u, ρ).
- `confint(m, method = "Wald")` (o "profile", más fiable) — IC de los β y de σ_u .

Ajustes y predicciones:

- `fitted(m)` — probabilidades **ajustadas** (incluyen el efecto aleatorio de cada grupo).
- `predict(m, type = "response")` — predicción en probabilidad; el argumento `re.form` es la clave:
 - `re.form = NULL` (por defecto) → **condicional** al grupo (usa su u_j).
 - `re.form = NA` (o `~0`) → **marginal/poblacional** (ignora los efectos aleatorios: «un centro medio»).
- `predict(m, newdata = nd, type = "response", re.form = ~0)` — para **perfiles nuevos** o centros no observados.
- `residuals(m, type = "pearson")` (o `"deviance"`).

Ajuste global:

- `summary(m)` — todo junto (efectos fijos + bloque Random effects); `AIC(m)`, `BIC(m)`, `logLik(m)`, `deviance(m)`.
- Salida ordenada con `broom.mixed::tidy(m)`, `glance(m)`, `augment(m)`.

Residuos simulados (DHARMa).

Con respuesta binaria y efectos aleatorios, los residuos de Pearson o deviance crudos **no** se leen (el caso individual de la [Sección 2.6](#)). La solución, ya conocida, son los **residuos simulados** de DHARMa (Hartig, 2024): el paquete **simula** muchas respuestas desde el modelo ajustado y, comparando cada observación con su distribución simulada, produce **residuos cuantil escalados** que, bajo un modelo correcto, son **uniformes** $U(0, 1)$ —y eso vale **igual para un GLMM**—. Estos residuos nos permiten testar uniformidad de los residuos (con Kolmogorov-Smirnov), sobre/infra-dispersión (se testa que el parámetro de dispersión es 1), exceso de valores atípicos y heterocedasticidad.



En R. Residuos simulados (DHARMa) en modelos mixtos

DHARMa genera **residuos cuantil escalados** —uniformes $U(0, 1)$ bajo un modelo correcto— por **simulación**, y funciona igual para `glmer`/`glmmTMB` que para `glm`. El flujo es: simular una vez y reutilizar el objeto.

```
library(DHARMa)
sim <- simulateResiduals(m, n = 1000) # m = modelo glmer/glmmTMB
plot(sim)                             # Q-Q (uniformidad, KS) + residuos vs predicho
```

Tests formales sobre `sim`:

```
testUniformity(sim)      # KS: ¿residuos uniformes? (ajuste global)
testDispersion(sim)     # sobre/infradispersión (cociente ~1 = ok)
testOutliers(sim)       # exceso de valores atípicos
testQuantiles(sim)      # patrón / heterocedasticidad (cuantiles vs predicho)
```

Lo propio de un mixto:

- **Residuos frente a una covariable o al grupo**, para detectar estructura no modelada (no linealidad, un centro desviado):

```
plotResiduals(sim, form = datos$x1)      # vs un predictor -> ¿forma funcional?
plotResiduals(sim, form = datos$centro)  # vs el factor de agrupamiento
```

- **Agregando por grupo** —un residuo por centro, para mirar el nivel del efecto aleatorio—:

```
sim_g <- recalculateResiduals(sim, group = datos$centro)
plot(sim_g)
```

- DHARMA calcula los residuos **condicionando** a los efectos aleatorios estimados (lo habitual); para diagnosticar la **estructura aleatoria** en sí, la agregación por grupo de arriba es la vía práctica.

Notas: con respuesta **binaria** o de **conteo** (muchos empates) DHARMA lo gestiona con *randomización* (`integerResponse = TRUE`, por defecto lo detecta); y `refit = TRUE` calcula los residuos por **bootstrap paramétrico** —más exacto con **pocos grupos**, pero lento—.

Se revisan pues el **Q–Q** (uniformidad, test KS), los **residuos frente a lo predicho** (patrones, dispersión, *outliers*) y, lo propio de un mixto, **agrupando por centro** para mirar el nivel del efecto aleatorio:

```
library(DHARMA)
sim <- simulateResiduals(m_int)
plot(sim)
```

DHARMA residual

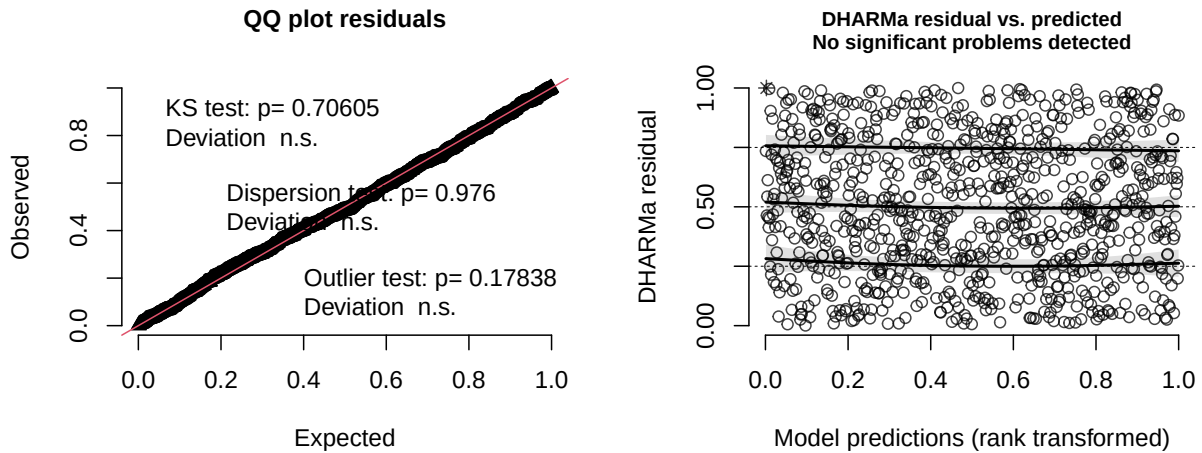
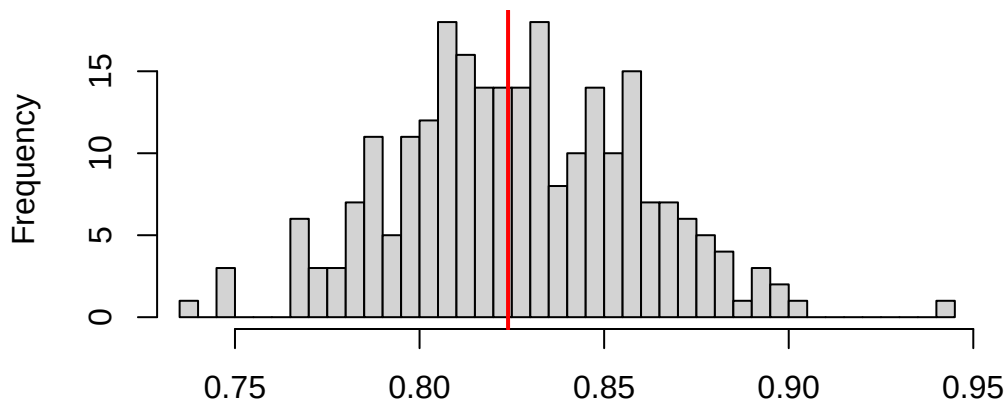


Figura 30: Residuos simulados (DHARMA) del GLMM de intercepto aleatorio. Izquierda: Q-Q de los residuos escalados, que bajo un buen ajuste siguen la diagonal uniforme. Derecha: residuos frente a lo predicho, que deberían repartirse sin patrón. Curvatura en el Q-Q o un abanico a la derecha señalarían forma funcional inadecuada o dispersión mal modelada.

```
testDispersion(sim)
```

```
# sobre/infradispersión
```

DHARMA nonparametric dispersion test via sd of residuals fitted vs. simulated



Simulated values, red line = fitted model. p -value (two.sided) = 0.976

DHARMA nonparametric dispersion test via sd of residuals fitted vs.

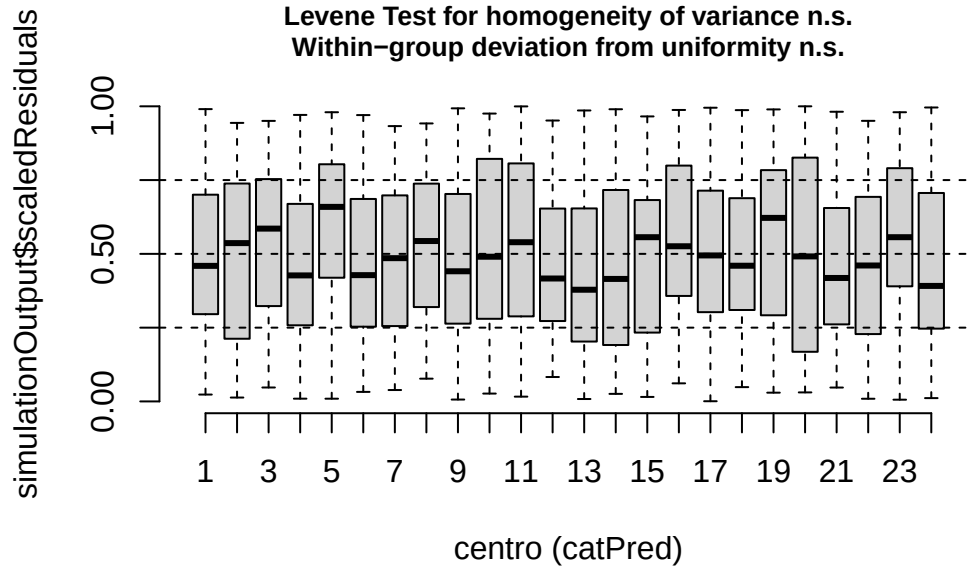

```

simulated

data:  simulationOutput
dispersion = 0.99685, p-value = 0.976
alternative hypothesis: two.sided

plotResiduals(sim, form = cohorte$centro) # residuos agregados por centro

```



Conclusión: no hay problemas. El Q–Q se pega a la diagonal y los tres tests (Kolmogorov-Smirnov, Dispersion y outliers) salen **no significativos** (KS $p = 0,71$, dispersión $p = 0,98$, outliers $p = 0,18$; DHARMA rotula «*no significant problems detected*») —el intercepto aleatorio reproduce bien los datos, también al **agrupar por centro**—. Si fallara, lo veríamos como una **curvatura** sistemática en el Q–Q, **sobredispersión** o un grupo de centros desviado, que apuntarían a una **covariable omitida**, una **forma funcional** inadecuada o una **estructura aleatoria** insuficiente.

¿Cuánto explica el modelo? R^2 marginal y condicional.

En un modelo mixto la varianza explicada se parte en dos (Nakagawa): el R^2 **marginal** —solo los **efectos fijos**— y el R^2 **condicional** —fijos **más** aleatorios—. La **diferencia** $R^2_{\text{condicional}} - R^2_{\text{marginal}} = \frac{\sigma_u^2}{\sigma_f^2 + \sigma_u^2 + \sigma_\varepsilon^2}$ es el **ICC no ajustado**: la fracción de la varianza total que aporta el **agrupamiento** (los centros) más allá de los efectos fijos. Es justo el *Unadjusted/Conditional ICC* que devuelve `performance::icc(m)`.

```
performance::r2(m_int)      # R2 marginal (fijos) y condicional (fijos + aleatorios)
```

```
# R2 for Mixed Models
```

```
Conditional R2: 0.311  
Marginal R2: 0.270
```

```
performance::icc(m_int)     # icc no ajustado = R2(cond)-R2(marg)
```

```
# Intraclass Correlation Coefficient
```

```
Adjusted ICC: 0.056  
Unadjusted ICC: 0.041
```

En la cohorte, $R^2_{\text{marginal}} = 0,27$ y $R^2_{\text{condicional}} = 0,31$: los efectos fijos (x_1, x_2) explican el **grueso** (27 %) y la estructura de centro **añade unos 4 puntos** (hasta el 31 %) —una contribución **modesta**, coherente con el ICC pequeño—.

El ajuste, ¿es sólido? Singularidad y convergencia.

Un GLMM puede **converger mal** o quedar **singular** —típico al pedir más estructura aleatoria de la que los datos sostienen, como la pendiente casi singular de la [Sección 4.3](#)—. Conviene comprobarlo siempre:

```
isSingular(m_int)           # TRUE = ajuste singular (estructura aleatoria no sos
```

```
[1] FALSE
```

```
performance::check_singularity(m_int)
```

```
[1] FALSE
```

Aquí `isSingular(m_int)` devuelve **FALSE**: el intercepto aleatorio está **bien sostenido** por los datos (a diferencia de lo que ocurría al forzar la pendiente en la [Sección 4.3](#)). Un ajuste **singular** (o con avisos de convergencia) **no sería de fiar** y obligaría a **simplificar** la parte aleatoria. Para una panorámica de golpe, `performance::check_model(m_int)` reúne varios diagnósticos —incluida la **normalidad de los efectos aleatorios** ($u_j \sim \mathcal{N}$)— en un solo gráfico.

💡 En R. Bondad de ajuste y solidez de un GLMM

Para los **residuos**, el tip de DHARMa de arriba. El resto del diagnóstico:

```
# Varianza explicada (Nakagawa)
performance::r2(m_int)      # R2 marginal (fijos) y condicional (fijos + aleatorios)
performance::icc(m_int)     # ICC ajustado y no ajustado (= R2_cond - R2_marg)

# Solidez del ajuste
performance::check_model(m_int) # panel de diagnósticos del mixto
isSingular(m_int)             # ¿ajuste singular? (estructura aleatoria no sostenida)
```

4.5 Extensiones: estructuras aleatorias más ricas

La parte aleatoria puede crecer más allá de un intercepto. Conviene reconocer las estructuras habituales —aunque en la cohorte, con pacientes simplemente agrupadas en centros, no todas tengan cabida— porque aparecen en cuanto el diseño se complica.

Correlados vs no correlados. Al dejar variar intercepto y pendiente, $(1 + x \mid g)$ estima además su **correlación** ρ . Que $\rho > 0$ implicaría que los centros de mayor riesgo basal responden, además, más a x . Si esa correlación no interesa o **desestabiliza** el ajuste (correlaciones de ± 1 , ajustes singulares), la especificación $(1 + x \parallel g)$ los deja **independientes** (sin ρ), que es un modelo más parco, pero recomendable cuando no tenemos bastante información (datos) para estimarla.

Cruzados vs anidados. Dos agrupamientos pueden combinarse de dos maneras, y la diferencia es **de diseño**, no de sintaxis:

- **Anidado:** cada nivel de un factor pertenece a **uno solo** del otro. Pacientes **centros** **regiones**: un centro está en una única región. Se escribe $(1 \mid \text{region/centro})$. Y un **anidamiento lo es por diseño**: si los centros se numeran 1, 2, ... **dentro** de cada región (de modo que el «centro 1» de una región no es el de otra), hay que declararlo anidado —o usar identificadores **únicos**— o lme4 confundiría centros distintos que comparten etiqueta.
- **Cruzado:** los factores se combinan **libremente** y ninguno anida en el otro —una misma paciente valorada por **varios médicos** (médicos y pacientes se cruzan), o la rejilla **centro** $\times$ **periodo**—. Se escribe $(1 \mid g1) + (1 \mid g2)$.

Regla práctica:

- *¿cada nivel de A vive dentro de un único nivel de B?* $\rightarrow$ **anidado**;
- *¿se combinan sin restricción?* $\rightarrow$ **cruzado**.

💡 En R. La sintaxis (|) para cada estructura

Un GLMM binomial se ajusta con `lme4::glmer` (o `glmmTMB::glmmTMB`, más rápido y flexible), con `family = binomial`, **igual para datos individuales y agrupados** — solo cambia la respuesta, como en `glm`—:

- **Individual** (0/1 por sujeto): `glmer(y ~ x1 + x2 + (1 | g), family = binomial, data = d)`.
- **Agrupado** (éxitos/ensayos por fila): `glmer(cbind exitos, fracasos) ~ x1 + x2 + (1 | g), family = binomial, data = d` (o la proporción con `weights = m`).

La parte (... | grupo) declara **qué** varía por grupo:

- (1 | g) — **intercepto** aleatorio por g.
- (1 + x | g) (o (x | g)) — intercepto y pendiente de x, **correlados**.
- (0 + x | g) — **solo** pendiente de x (sin intercepto aleatorio).
- (1 + x || g) — intercepto y pendiente, **sin** correlación.
- (1 | g1) + (1 | g2) — dos agrupamientos **cruzados**; (1 | g1/g2) para **anidados**.

En la cohorte la estructura es **simple** —pacientes en centros, un solo nivel—, así que estas extensiones quedan como repertorio para diseños más ricos (las reactivaremos en el Caso 2). El **protocolo de selección** de la [Sección 4.3](#) sigue valiendo: empezar simple, añadir estructura solo si AIC/LRT la respaldan, y vigilar los ajustes singulares.

4.6 El modelo mixto sobre datos reales: GLOW por centro

La cohorte tiene la ventaja de la **verdad conocida**, pero 24 centros son muchos para *verlos*. GLOW ofrece el contrapunto **real e intuitivo**: 500 mujeres seguidas en **6 centros** (`site_id`). Las pacientes de un mismo centro comparten contexto asistencial, así que cabe el **mismo GLMM logístico** con intercepto aleatorio por centro, ahora sobre la respuesta `real fracture`:

```
library(lme4)
glow_m <- glow |> mutate(site_id = factor(site_id))

m_glow_pool <- glm (fracture ~ age + priorfrac, family = binomial, data =
m_glow_mix <- glmer(fracture ~ age + priorfrac + (1 | site_id), family = binomial, data =

summary(m_glow_mix)
```

Generalized linear mixed model fit by maximum likelihood (Laplace
 Approximation) [glmerMod]
 Family: binomial (logit)
 Formula: fracture ~ age + priorfrac + (1 | site_id)
 Data: glow_m

AIC	BIC	logLik	-2*log(L)	df.resid
536.2	553.1	-264.1	528.2	496

Scaled residuals:

Min	1Q	Median	3Q	Max
-1.21215	-0.55258	-0.43669	-0.06166	2.62011

Random effects:

Groups	Name	Variance	Std.Dev.
site_id	(Intercept)	0.02237	0.1496

Number of obs: 500, groups: site_id, 6

Fixed effects:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-4.20529	0.85331	-4.928	8.3e-07 ***
age	0.04085	0.01223	3.341	0.000835 ***
priorfracYes	0.84935	0.23558	3.605	0.000312 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Correlation of Fixed Effects:

	(Intr)	age
age	-0.985	
priorfracYes	0.165	-0.258

```
# Varianza entre centros e ICC latente
vc <- as.data.frame(VarCorr(m_glow_mix))
c(sigma_u = sqrt(vc$vcov[1]), ICC = vc$vcov[1] / (vc$vcov[1] + pi^2 / 3))
```

sigma_u	ICC
0.149575578	0.006754597

```
# Efectos fijos: agrupado (glm) vs mixto (glmer)
cbind(pooled = coef(m_glow_pool), mixto = fixef(m_glow_mix))
```

	pooled	mixto
(Intercept)	-4.21429456	-4.20528747
age	0.04119282	0.04085258
priorfracYes	0.83883503	0.84934528

```
re_glow <- as.data.frame(ranef(m_glow_mix, condVar = TRUE)) # grp, condval, condsd
ggplot(re_glow, aes(reorder(grp, condval), condval)) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_pointrange(aes(ymin = condval - 1.96 * condsd,
                     ymax = condval + 1.96 * condsd)) +
  coord_flip() +
  labs(x = "Centro (site_id)", y = "Intercepto aleatorio (log-odds)")
```

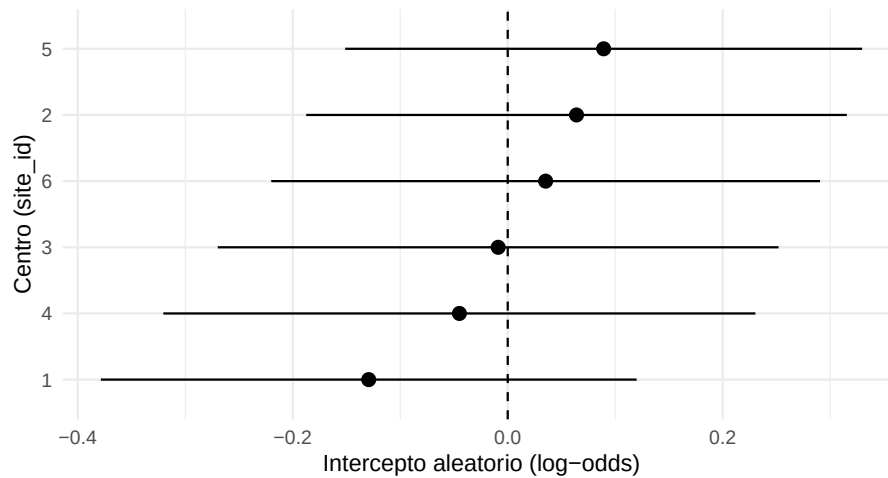


Figura 31: Efecto de centro modelizado en GLOW: intercepto aleatorio de cada uno de los 6 centros (desviación en log-odds respecto al riesgo medio) con IC 95%. Los seis se apiñan en torno a 0 y todos los intervalos lo cruzan: la variación entre centros es mínima (ICC latente 0,7%).

Y el resultado es **claro**: la variación entre centros es **mínima**. El modelo estima $\hat{\sigma}_u = 0,15$ (log-odds) y un **ICC latente de apenas 0,7 %** —casi todo el riesgo está **dentro** de los centros, no entre ellos—. El *caterpillar* lo confirma: los **seis** interceptos se apiñan en torno a 0 (de $-0,13$ a $+0,09$) y **todos** los intervalos lo cruzan, así que ninguno se separa del riesgo medio. Y como el efecto aleatorio es tan pequeño, los efectos fijos del mixto son **prácticamente idénticos** a los del *pooled* (edad $\approx +0,04$ por año; fractura previa $\widehat{OR} \approx e^{0,85} \approx 2,3$). En GLOW, **agrupar por centro apenas aporta**: un hallazgo legítimo —no toda jerarquía esconde heterogeneidad— y el contraste perfecto con la cohorte, donde el centro **sí** estructuraba el riesgo ($\hat{\sigma}_u = 0,44$, ICC $\approx 6\%$).

Esta base hace explícitos dos **límites** que la cohorte disimulaba. Con **solo 6 grupos**, la varianza σ_u se estima con **mucha incertidumbre** —por eso la cohorte, con 24 centros y verdad conocida, sigue siendo el banco de pruebas riguroso—; y un $\hat{\sigma}_u$ tan pequeño (y, en otros casos, **pegado al borde**) es el tipo de situación de **frontera** que el enfoque bayesiano maneja con más suavidad (callout al cierre). GLOW gana en **intuición** (pocos centros, reales, dibujables); la cohorte, en **rigor**.

i Puente a la Unidad 5

Esta misma cohorte guarda una pregunta que aquí hemos dejado fuera: no solo **si** se fractura, sino **cuándo**. Las variables **tiempo** (periodo de la revisión) y **evento** (1 = fractura observada, 0 = **censura**) abren el **análisis de supervivencia**, que la **Unidad 5** aborda **sin salir del marco GLM**: el tiempo discreto se reescribe como una regresión binomial con enlace **cloglog** sobre datos **persona-periodo**. Y el hilo mixto **vuelve** allí: el mismo intercepto aleatorio por centro reaparece como **fragilidad** —`glmer(..., family = binomial("cloglog"))`—, de modo que los efectos aleatorios que hemos introducido aquí se reactivan sobre el **reloj de la supervivencia** ([Sección 5.3](#)).

i Los efectos aleatorios encajan mejor en el enfoque bayesiano

La distinción **fijo vs aleatorio** es, en parte, una incomodidad **frecuentista**: los aleatorios se tratan como variables latentes que hay que **integrar** de la verosimilitud (con aproximaciones —Laplace, cuadratura—), y su varianza puede quedar **pegada al borde** —los ajustes *singulares* o el $\hat{\sigma}_u$ subestimado que vimos—. En el marco **bayesiano** no hay tal dicotomía: **todos** los parámetros tienen distribución, y un efecto aleatorio es solo un parámetro con un **prior jerárquico** ($u_j \sim \mathcal{N}(0, \sigma_u^2)$, con su propio prior para σ_u^2). El *partial pooling* **emerge solo** de esa jerarquía, la incertidumbre de la varianza se propaga entera, los problemas de frontera se manejan con suavidad y estructuras complejas (como el nominal mixto) encajan sin forzar. Herramientas: **brms**, **rstanarm**, **MCMCglmm**.

! Tarea del estudiante (1.4) — resuelve la Pregunta 5 del caso

Responde la **Pregunta 5** del cliente ([Unidad 6](#)) sobre la cohorte de tu equipo: *¿cuánto del riesgo depende de la clínica, hay centros mejores o peores, y qué esperar de uno nuevo?*
Tareas:

1. **Reconoce el agrupamiento (RA6)**. Ajusta el riesgo de fractura con **intercepto aleatorio por centro** (`glmer, (1 | centro)`) y compáralo con la versión *pooled* (`glm`): interpreta el **ICC** (cuánto pesa la clínica) y explica por qué el *pooled* **atenúa** los efectos (condicional vs marginal), contrastándolos con la **verdad del DGP**.
2. **Centros mejores y peores**. Interpreta el *caterpillar* de los interceptos por centro (`ranef`) y el *shrinkage*: qué clínicas se desvían y por qué las extremas se encogen.

3. **¿Pendiente aleatoria?** Prueba $(1 + x1 \mid \text{centro})$ y razona —varianza estimada, correlación con el intercepto, convergencia— si los datos la justifican.
4. **Diagnóstico (RA4).** Residuos simulados (`DHARMA::simulateResiduals`, Q–Q y dispersión), R^2 **marginal** y **condicional** (`performance::r2`) y `isSingular` para confirmar que el ajuste es sólido.
5. **Responde al cliente**, incluida la expectativa para **una clínica nueva** (predicción marginal).

Opcional (dato real): repite el punto 1 sobre GLOW con $(1 \mid \text{site_id})$ y comenta por qué allí, con tan pocos centros, el agrupamiento apenas aporta.

5 · Supervivencia: del hazard al GLM en tiempo discreto

Las unidades anteriores respondían a «¿ocurre el evento?»: una fractura, sí o no, modelada con la logística. Pero a esa pregunta le falta otra —«¿y **cuándo**?»—. Pasar de la ocurrencia al **tiempo hasta el evento** abre el **análisis de supervivencia**, el segundo hilo en espiral del curso. Sin salir del marco GLM, el tiempo-a-evento en **tiempo discreto** se reescribe como una regresión binomial con enlace `cloglog`, y nos permitirá integrar efectos de agrupación con modelos mixtos como los que ya vimos en la [Unidad 4](#).

Los datos: un seguimiento por periodos.

GLOW se queda corto aquí: registra si hubo fractura en el primer año, pero no **cuándo** ni a quién se dejó de observar. Por eso trabajamos con una **cohorte simulada** —inspirada en el mismo contexto clínico (fracturas en mujeres seguidas en varias clínicas) y descrita al detalle, con todas sus variables y su proceso generador, en la Presentación del caso— que añade justo lo que falta: **el tiempo**.

Y lo añade de una forma muy concreta que conviene fijar desde el principio, porque determina toda la teoría que sigue: a cada mujer no la observamos de forma continua, sino en una serie de revisiones $t = 1, 2, \dots, 6$ (una al año). En cada revisión a la que llega sin haberse fracturado —es decir, mientras sigue *en riesgo*— anotamos una sola cosa: ¿fractura en este periodo, sí o no? La secuencia continúa hasta que ocurre una de dos cosas: o registramos la fractura (y ahí termina el seguimiento), o la mujer sale del estudio sin haberla tenido (**censura**). Dicho de otro modo, el dato de cada paciente es una secuencia de respuestas binarias, periodo a periodo: eso es, literalmente, un **proceso estocástico en tiempo discreto**, y es la materia prima con la que construiremos el modelo.

Resumido en variables, de cada paciente tenemos:

- el **tiempo hasta la fractura o salida del estudio** —por finalización sin rotura— (tiempo, la revisión en que se detecta),

- si llegamos a observar el evento o no (**evento**: 1 = fractura observada; 0 = **censura**, p. ej. fin administrativo del seguimiento)
- y sus **covariables de base** —constantes a lo largo del seguimiento—, con lectura clínica: **x1** es un marcador de **fragilidad ósea** (baja densidad mineral, estandarizada en puntuación z), un factor de riesgo continuo —a mayor valor, hueso más frágil y más riesgo a fractura ($\beta_1 > 0$)—; **x2** es un **factor binario protector** —recibir o no un tratamiento preventivo, $\beta_2 < 0$ —; y **centro** es la **clínica** a la que pertenece cada paciente —el agrupamiento que trataremos como **fragilidad inicial**, con una interceptación aleatoria—. Al ser simulada, conocemos la verdad del proceso generador y podremos comprobar que el ajuste la recupera.

Qué cambia respecto a antes.

Hasta ahora estimábamos la **probabilidad** de un desenlace binario. Ahora el objeto es **dinámico**, e interesa:

- el **riesgo (*hazard*) a lo largo del tiempo**: ¿qué riesgo de fracturar hay en cada periodo, entre quienes llegan sin fractura?,
- la **curva de supervivencia** $S(t)$: ¿qué fracción de la muestra/población sigue sin fractura pasado t ?
- y cómo las covariables **aceleran o retrasan** el evento (los *hazard ratios*).

Y además, aparece un ingrediente nuevo, la **censura**: hay que aprovechar la información de las pacientes a las que **no** vimos fracturar en la ventana de observación, sin sesgarla. Todo ello, como veremos, cabe dentro del GLM.

Comenzaremos definiendo y describiendo los conceptos básicos en el análisis de supervivencia, y seguiremos implementando este análisis de un modo no paramétrico y después a través de un modelo lineal generalizado binario.

i Sobre esta unidad

RA implicados:

- **RA7 · Tiempo-a-evento y censura.** Reformular la supervivencia en tiempo discreto como un GLM cloglog persona-periodo, tratar la censura y predecir hazard y supervivencia.
- **RA3 · Interpretación en la escala correcta** (vía hazard ratios).

(Dobson & Barnett, 2018, cap. de supervivencia; Aitkin & Clayton, 1980; Allison, 1982; Cox, 1972; Prentice & Gloeckler, 1978)

5.1 La función de supervivencia y la función de riesgo en tiempo discreto

Dado que nuestros datos son un **seguimiento por periodos**, describimos el tiempo-a-evento con las herramientas que le corresponden: las de **tiempo discreto**. No necesitaremos ninguna otra.

Sea T el **tiempo hasta la fractura**, medido en **número de revisión**. Como no observamos el instante exacto, sino solo el periodo en que se detecta el evento, T es una **variable aleatoria discreta** con valores en $\{1, 2, \dots\}$. Toda su información se resume en dos funciones —conocer una determina la otra—:

- **Función de riesgo por periodo (hazard discreto).** De los sujetos que llegan al periodo t sin evento (el **conjunto de riesgo** en t), h_t es la **fracción que fractura durante ese periodo**:

$$h_t = \Pr(T = t \mid T \geq t) \in [0, 1].$$

Es exactamente la respuesta a la pregunta que hacemos en cada revisión —entre quienes llegan sin fractura, ¿qué proporción fractura *ahora*?—. Nótese lo importante: h_t es una **probabilidad** (acotada por 1). En lenguaje de tablas de vida, la **probabilidad condicional de fallo** q_t .

- **Función de supervivencia discreta.** $S_t = \Pr(T > t)$, la probabilidad de “seguir sin fractura” pasado el periodo t . Se reconstruye periodo a periodo, encadenando la probabilidad de *no* fallar en cada uno:

$$S_t = \prod_{k=1}^t (1 - h_k).$$

Se interpreta directamente ($S_5 = 0.8$: el 80 % llega sin fractura más allá de la quinta revisión), cumple $S_0 = 1$ y es **no creciente**.

Ambas se enlazan por $h_t = 1 - \frac{S_t}{S_{t-1}}$: el hazard es la **caída relativa** de la supervivencia de un periodo al siguiente. Y la probabilidad de que la fractura caiga **exactamente** en el periodo t combina las dos ideas —sobrevivir a los anteriores y fallar en t —:

$$\Pr(T = t) = h_t \prod_{k < t} (1 - h_k).$$

i Un apunte

El hazard por periodo h_t es una **probabilidad**, no una tasa: esto es lo propio del tiempo discreto y es justo lo que nos permitirá tratarlo con un GLM binomial. (El análisis en tiempo *continuo*, donde el riesgo es una tasa instantánea, es el terreno del modelo de Cox; lo veremos en detalle en el Caso 3.)

La conexión con Bernoulli (y con el GLM).

En nuestros datos, cada periodo en riesgo es un **ensayo de Bernoulli** con probabilidad h_t : el sujeto “falla” (evento) o “sobrevive” ese periodo. Una paciente **censurada** es aquella que sale del seguimiento sin haber tenido una fractura (se acaba el periodo de observación); la censura no está vinculada al riesgo de fractura.

Disponer los datos con **una fila por sujeto y periodo en riesgo** (persona-periodo) convierte el problema en una **secuencia de ensayos de Bernoulli** con probabilidad h_t . La verosimilitud de supervivencia se **factoriza** exactamente como un producto de Bernoulli sobre los periodos,

$$P(T = t) = \begin{cases} h_t \prod_{s < t} (1 - h_s) & \text{si evento en } t \\ \prod_{s \leq c} (1 - h_s) & \text{si censura} \end{cases}$$

así que un **GLM binomial ordinario** (con enlace cloglog, [Sección 5.3](#)) maximiza la verosimilitud correcta, sin maquinaria extra.

i ¿Hace falta un efecto aleatorio por paciente?

Aunque cada sujeto aporte **varias filas** en el formato persona-periodo, **no**. Las filas de un mismo sujeto son **condicionalmente independientes** dadas las covariables y los interceptos por periodo —es la factorización de la verosimilitud de arriba, no una aproximación—, de modo que un GLM ordinario ya maximiza la verosimilitud correcta. Los efectos aleatorios entran en **otro nivel**: el de la **clínica (centro)**, como *fragilidad compartida* (la ajustamos en la [Sección 5.3](#)). Una *fragilidad individual* por paciente sería una extensión posible —no un requisito— y, además, difícil de identificar con tan pocos periodos por sujeto.

5.2 Precedentes en clave de supervivencia: Kaplan–Meier y Cox

El análisis de supervivencia tiene un instrumental **propio**, anterior al GLM, que conviene conocer antes de construir el nuestro: aporta el contexto y el vocabulario. Lo recorremos de lo más descriptivo a lo más estructurado.

Primero, **Kaplan–Meier**: la forma no paramétrica de *dejar hablar a los datos*, que dibuja la curva de supervivencia y compara grupos sin suponer nada sobre el riesgo. Después, los **modelos de riesgos proporcionales**, el marco común que permite pasar de *describir* a *explicar* —cómo las covariables modulan el riesgo— y del que cuelga nuestro GLM. Su versión clásica en tiempo continuo es el **modelo de Cox**, que aquí solo presentamos como referencia: su desarrollo queda para el Caso 3.

Con estos precedentes tendremos el vocabulario para construir, en la [Sección 5.3](#), un GLM que —al ser la cohorte simulada— podremos **validar contra la verdad del proceso generador**. Y ese GLM no se queda en describir: predice el riesgo base y la supervivencia periodo a periodo, incorpora la fragilidad entre clínicas como efecto aleatorio y puede relajar la proporcionalidad cuando haga falta, todo dentro del único marco que ya manejamos.

Kaplan–Meier: dejar hablar a los datos

En supervivencia, lo primero es **mirar los datos**, sin comprometerse todavía con ningún modelo. Antes de imponer una fórmula al riesgo o un efecto a las covariables, interesa ver la forma de la curva de supervivencia —cómo y cuándo cae— y si dos grupos (tratamiento/control) se separan. Ese es el terreno de los métodos **no paramétricos**: en lugar de estimar los parámetros de un modelo, leen la curva directamente de los datos, con el mínimo de supuestos. Su herramienta de referencia —y casi siempre el primer paso de cualquier análisis de supervivencia— es el estimador de **Kaplan–Meier**.

Antes de nada, conviene **ver la forma de los datos**. La cohorte tiene **una fila por mujer**: sus covariables de base (`x1`, `x2`, `centro`), el **periodo** en que se observa el desenlace (`tiempo`) y si este fue **fractura o censura** (`evento`: 1 = fractura, 0 = censura).

```
head(cohorte[, c("id", "centro", "x1", "x2", "tiempo", "evento")])
```

	id	centro	x1	x2	tiempo	evento
1	1	1	-1.6736304	0	4	1
2	2	1	1.0929380	1	3	1
3	3	1	-0.3526607	1	6	0
4	4	1	0.2976267	0	4	0
5	5	1	-0.3236454	1	6	0
6	6	1	-1.6879930	1	4	0

El estimador de **Kaplan–Meier** (Kaplan & Meier, 1958) reconstruye la curva de supervivencia **sin imponer ninguna forma funcional**: en cada instante con eventos multiplica la fracción que “sobrevive”,

$$\hat{S}(t) = \prod_{t_k \leq t} \left(1 - \frac{d_k}{n_k}\right),$$

con d_k eventos y n_k sujetos en riesgo en t_k . Fíjate en que es exactamente el producto $\prod(1 - h_k)$ de la [Sección 5.1](#), con el hazard estimado de forma empírica, $\hat{h}_k = d_k/n_k$: Kaplan–Meier es el **hermano no paramétrico** de la supervivencia discreta $S_t = \prod(1 - h_k)$, y es especialmente útil para comparar curvas de supervivencia entre grupos/perfiles. Su visualización nos permite responder preguntas descriptivas del tipo : ¿qué grupo sobrevive más?, ¿en qué momento empieza a agravarse el riesgo en un perfil?, ...

```
library(survival)
library(survminer)

km <- survfit(Surv(tiempo, evento) ~ x2, data = cohorte)
ggsurvplot(km, data = cohorte, conf.int = TRUE, pval = TRUE,
            legend.title = "Tratamiento (x2)",
            legend.labs = c("No", "Sí"),
            xlab = "Periodo de revisión", ylab = "Supervivencia S(t)")
```

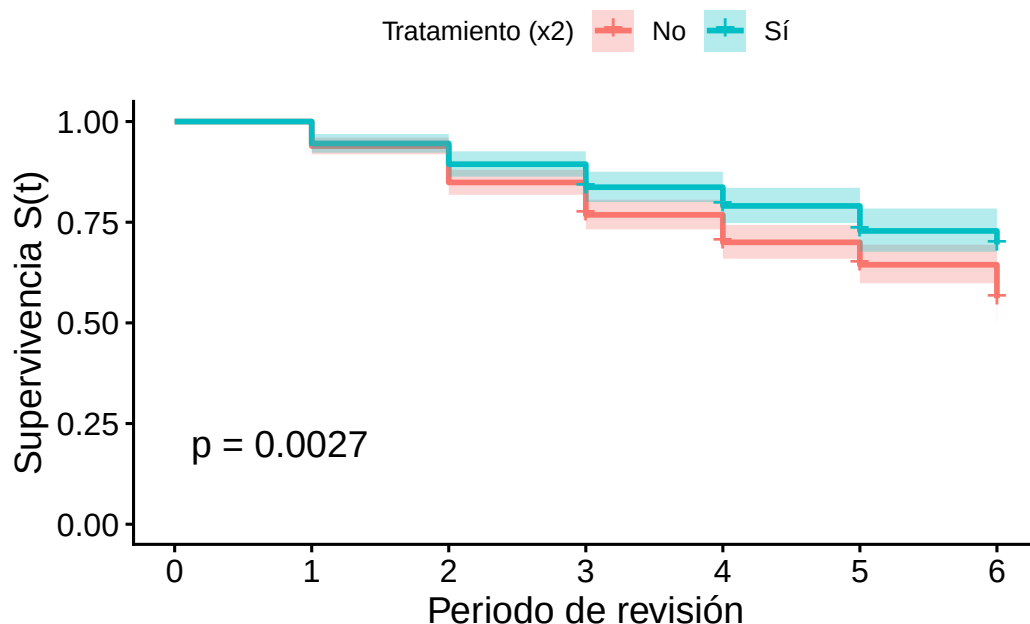


Figura 32: Kaplan–Meier de la cohorte por brazo de tratamiento (x2): describe la caída de $S(t)$ sin modelo, y el log-rank contrasta los dos grupos. KM solo admite grupos discretos —no la fragilidad continua x1— y no entrega hazard ratios; para eso hace falta un modelo.

En la cohorte el patrón es el esperado: las tratadas ($x2 = \text{Sí}$) se mantienen por encima de las no tratadas y la separación crece con los periodos —arrancan casi juntas y, al sexto, la supervivencia ronda 0.73 frente a ≈ 0.56 —.

¿Es esa separación real o podría ser ruido muestral?

Para contrastarlo se usa la prueba **log-rank** (Mantel, 1966), el contraste no paramétrico que acompaña a Kaplan–Meier. Su idea: en cada periodo con eventos cuenta cuántas fracturas se observan en un grupo y cuántas cabría esperar si las dos curvas fueran iguales (bajo H_0 : misma supervivencia en ambos grupos), y acumula esas diferencias “observado — esperado” a lo largo

del seguimiento. Si se apilan de forma sistemática en un grupo, la separación difícilmente es casual y el p -valor sale pequeño. No supone ninguna forma funcional —por eso es el compañero natural de KM—, pero, como la propia curva, no mide el tamaño del efecto ni ajusta por covariables: solo responde *sí/no* a “¿difieren los grupos?”.

En R, el p -valor que `ggsurvplot(..., pval = TRUE)` imprime sobre la curva es justo el del log-rank; el contraste explícito se obtiene con `survdifff`:

```
survdifff(Surv(tiempo, evento) ~ x2, data = cohorte)
```

Sobre la cohorte da $p = 0.0027$: la ventaja de las tratadas **no** es ruido.

Con todo, KM **describe** y **compara**, pero se queda como una primera herramienta descriptiva: **no ajusta por covariables**, no admite predictores **continuos** (la fragilidad `x1` se queda fuera) y no entrega un **tamaño de efecto** interpretable. Para eso hace falta una **regresión**.

Riesgos proporcionales, en tiempo discreto

Esa regresión exige **imponer una estructura**: decir cómo entran las covariables en el riesgo. La estructura que adoptamos —y que nos llevará directamente al GLM binario— es la de **riesgos proporcionales**.

La idea es que **una covariable multiplica el riesgo de cada periodo por un factor constante** —el mismo en la primera revisión que en la sexta—, de modo que su efecto se resume en un solo número. Falta precisar *sobre qué cantidad* actúa ese factor. La respuesta sale de la propia S_t . Como $S_t = \prod_{k \leq t} (1 - h_k)$, tomando logaritmos cada periodo aporta “ $-\log(1 - h_k) \geq 0$ ” a la caída de la supervivencia acumulada; llamemos a eso el **aporte de riesgo** del periodo. La hipótesis de riesgos proporcionales dice que la covariable multiplica ese aporte por un factor común $\exp(\mathbf{x}'\beta)$:

$$-\log(1 - h_t(\mathbf{x})) = \exp(\mathbf{x}'\beta) [-\log(1 - h_{0t})],$$

donde h_{0t} es el riesgo del periodo para la paciente de **referencia** o basal (en la que no hay tratamiento, $\mathbf{x} = \mathbf{0}$). Y ahora el paso clave: tomando log a ambos lados, aparece el enlace **clog-log**,

$$\underbrace{\log(-\log(1 - h_t))}_{\text{cloglog}(h_t)} = \underbrace{\log(-\log(1 - h_{0t}))}_{\alpha_t} + \mathbf{x}'\beta.$$

Es decir: la hipótesis de riesgos proporcionales, escrita sobre el hazard por periodo, *es* un GLM binomial con enlace cloglog —un intercepto α_t por revisión (el riesgo base) más un β por covariable—. No hemos salido del tiempo discreto ni del GLM binario: el enlace no se *elige*, se *deduce* del supuesto (Allison, 1982; Prentice & Gloeckler, 1978). Y en el modelo que ajustaremos, estimar el hazard base será, literalmente, estimar esos α_t —uno por revisión—; en la [Sección 5.3](#) veremos que crecen, recuperando el riesgo base creciente del simulador.

Dos lecturas inmediatas de ese factor constante:

- **Sobre la supervivencia.** Si el aporte de cada periodo se multiplica por $\exp(\mathbf{x}'\beta)$, la supervivencia entera se eleva a esa potencia: $S_t(\mathbf{x}) = S_{0t}^{\exp(\mathbf{x}'\beta)}$. En la escala cloglog de la supervivencia, las curvas de dos grupos van **paralelas** (separadas por una distancia fija) y nunca se cruzan —esa constancia es el supuesto—.
- **El hazard ratio.** Cada $\exp(\beta_k)$ es el factor por el que se multiplica el riesgo ante un aumento unitario de x_k : > 1 aumenta el riesgo, < 1 protege (reduce riesgo), $= 1$ indiferente. Un solo número describe el efecto en los seis periodos.

La Figura 33 lo hace visible: a la izquierda, en la escala cloglog, esa misma diferencia es una distancia vertical fija —las curvas paralelas de los riesgos proporcionales—; a la derecha, en la escala del hazard, la brecha entre grupos se ensancha (el efecto es multiplicativo).

```
beta <- log(2) # efecto de la covariable: HR = exp(beta)
base <- tibble(periodo = 1:6,
               h0 = c(0.05, 0.07, 0.09, 0.11, 0.13, 0.15)) # hazard base creciente (ilustración)

base |>
  mutate(`Referencia (x = 0)` = h0,
         `Grupo (x = 1)`      = 1 - (1 - h0)^exp(beta)) |> # PH discreto exacto
  pivot_longer(c(`Referencia (x = 0)`, `Grupo (x = 1)`),
               names_to = "grupo", values_to = "hazard") |>
  mutate(`Escala hazard: h_t` = hazard,
         `Escala cloglog: log(-log(1 - h_t))` = log(-log(1 - hazard))) |>
  pivot_longer(c(`Escala hazard: h_t`, `Escala cloglog: log(-log(1 - h_t))`),
               names_to = "escala", values_to = "valor") |>
  ggplot(aes(periodo, valor, color = grupo)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  facet_wrap(~escala, scales = "free_y") +
  scale_color_manual(values = c("Referencia (x = 0)" = "grey55",
                                "Grupo (x = 1)"      = "#2c7fb8")) +
  labs(x = "Periodo de revisión", y = NULL, color = NULL) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "top")
```

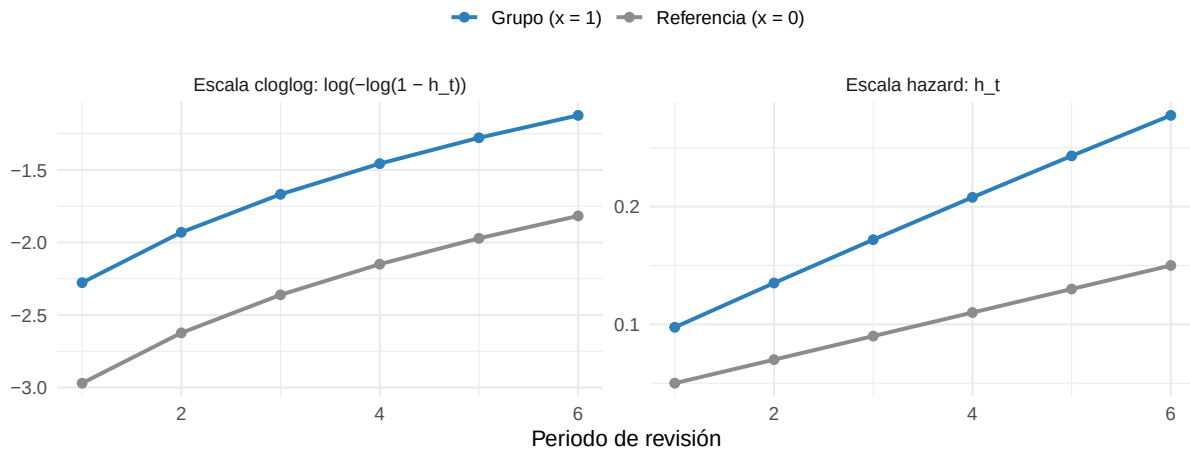


Figura 33: Qué significa ‘riesgos proporcionales’ en tiempo discreto (riesgo base creciente de juguete, $HR = 2$). Izquierda (escala cloglog): ese efecto es una distancia vertical fija (); las curvas van paralelas y nunca se cruzan. Si esa distancia cambiara con los periodos, la proporcionalidad fallaría. Derecha (hazard h_t): el grupo con la covariable alta es el de referencia con su aporte de riesgo escalado por $\exp(\)$ cada periodo —la brecha se ensancha porque el efecto es multiplicativo—.

i Cuándo falla la proporcionalidad

El supuesto de proporcionalidad se puede romper de dos formas.

- En un efecto que se desvanece, el hazard ratio se mueve hacia 1 sin llegar a cruzarlo: la ventaja se diluye y las curvas convergen, pero no se cortan. Es lo que ocurre con una vacuna cuya protección decae con el tiempo, o por depleción de susceptibles —los pacientes más frágiles tienen el evento pronto y queda una población seleccionada más robusta, de modo que el efecto del factor de riesgo se atenúa—.
- En curvas que se cruzan, el hazard ratio pasa de > 1 a < 1 y la ventaja se invierte. El caso de manual es cirugía frente a tratamiento conservador: riesgo alto al principio (mortalidad perioperatoria) y menor después, así que las curvas de supervivencia se cortan.

La **señal práctica**: si las curvas de Kaplan–Meier convergen sin tocarse, es un efecto que se desvanece; si se cortan, es un cruce. En cualquiera de los dos casos —o si el efecto de una covariable cambia con el tiempo, o si el tratamiento tiene un efecto inicial fuerte que luego desaparece— un único hazard ratio deja de describir bien el efecto, y el modelo de Cox básico (o nuestro GLM con un solo HR) puede ser inadecuado: hay que recurrir a sus extensiones —coeficientes que varían en el tiempo, estratificación—.

¿Es razonable asumir proporcionalidad aquí?

La proporcionalidad es un supuesto, pero **se puede comprobar**, y con una covariable a mano el chequeo es directo. Para la binaria **x2**, la propia curva de Kaplan–Meier ya da una primera respuesta: bajo proporcionalidad las curvas de los dos grupos **no se cruzan** y se separan de forma ordenada —justo lo que veíamos, las tratadas siempre por encima con la brecha creciendo—, así que no hay señal de alarma. El chequeo gráfico más fino es el **diagrama log–log**: representar ‘ $\log(-\log \hat{S}(t))$ ’ frente a ‘ $\log t$ ’ —es decir, el **cloglog** de la supervivencia de Kaplan–Meier— debería dar curvas aproximadamente paralelas si el supuesto se cumple. Que el diagnóstico natural sea precisamente el **cloglog** no es casualidad: es el enlace del modelo que construiremos.

```
ggsurvplot(km, data = cohorte, fun = "cloglog",
            legend.title = "Tratamiento", legend.labs = c("No", "Sí"),
            xlab = "log(periodo)", ylab = "log(-log S(t))")
```

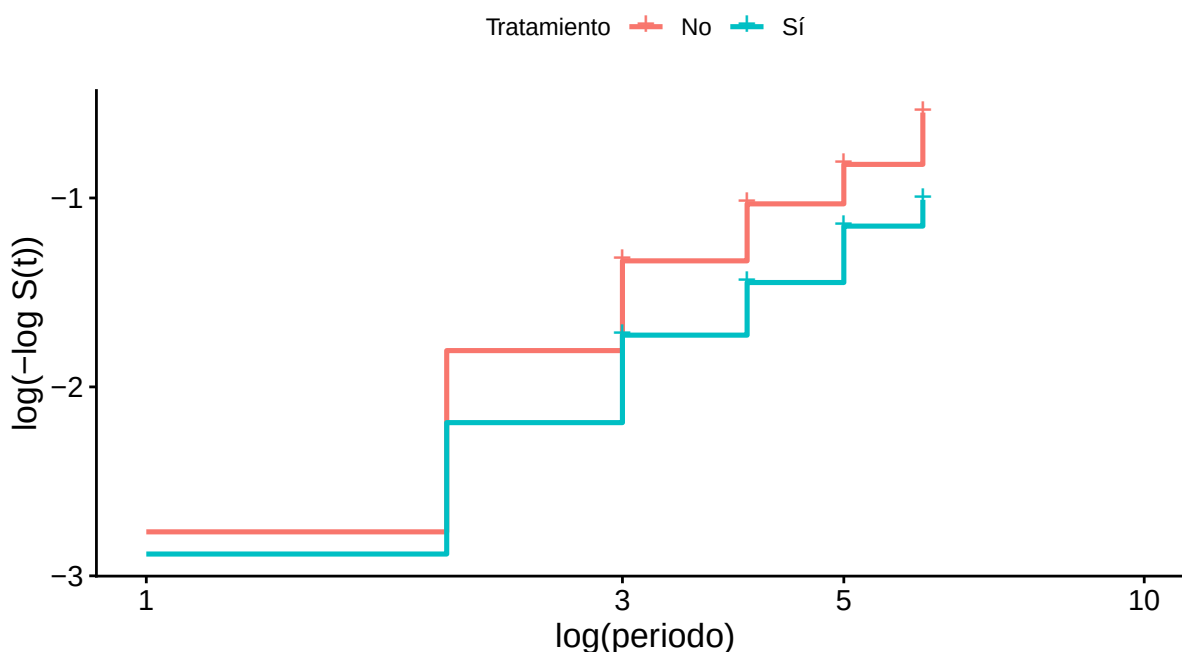


Figura 34: Diagrama log-log de Kaplan-Meier por tratamiento: $\log(-\log S(t))$ frente a $\log(\text{tiempo})$. Bajo riesgos proporcionales las dos curvas son aproximadamente paralelas. Con solo seis periodos el chequeo es grueso, pero no se aprecia cruce ni convergencia, así que la proporcionalidad para **x2** es asumible.

Para la covariable **continua x1**, Kaplan–Meier no sirve directamente (habría que **categorizarla** en grupos); ahí el chequeo general son los **residuos de Schoenfeld** del modelo ajustado —una tendencia temporal en ellos delata la no-proporcionalidad—, que dejamos para cuando tengamos el ajuste.

💡 En R. Estimar y dibujar la curva de Kaplan–Meier

Hace falta el paquete **survival** (y **survminer** para dibujarla). Los datos van **a nivel de paciente** —una fila por mujer—, con el **tiempo** hasta el evento y el **indicador** de evento/censura (1 = evento, 0 = censura). La respuesta se construye con `Surv(tiempo, evento)` y la curva se estima con `survfit`:

```
library(survival); library(survminer)
km <- survfit(Surv(tiempo, evento) ~ x2, data = cohorte) # ~ 1 para la curva global
ggsurvplot(km, data = cohorte, pval = TRUE)             # añade la prueba log-rank
```

A la derecha de `~` va la variable de grupo (aquí `x2`), y `evento` debe codificarse 1 = evento, 0 = censura; con `fun = "cloglog"` se obtiene el diagrama log–log de arriba.

Cox: el mismo supuesto en tiempo continuo

El supuesto de riesgos proporcionales que acabamos de escribir en tiempo discreto tiene una versión célebre en **tiempo continuo**: el **modelo de Cox** (Cox, 1972), el estándar del análisis de supervivencia. Aquí **no** entramos en su maquinaria —la verosimilitud parcial y su carácter semiparamétrico se ven en detalle en el Caso 3—; aquí nos basta una idea: nuestro GLM binario es su contrapartida en tiempo discreto y comparte con él la lectura de los efectos como *hazard ratios*. Reencontraremos a Cox, ya con su maquinaria, en el Caso 3.

Con Kaplan–Meier para describir y el marco de riesgos proporcionales para explicar, ya tenemos todo lo que necesitamos. En la [Sección 5.3](#) construimos nuestro modelo dentro del GLM, lo ajustamos, comprobamos que recupera la verdad del simulador y derivamos las conclusiones clínicas.

5.3 La modelización GLM del problema de supervivencia

Las técnicas de la [Sección 5.2](#) eran el precedente; ahora construimos nuestro modelo, y lo hacemos como en la práctica: a partir de los datos, sin suponer que conocemos el proceso que los generó. De las 877 mujeres de la cohorte tenemos tres piezas —un **evento** binario (fractura sí/no), un **tiempo** discreto (la revisión en que se observa) y una **censura** (seguimientos que terminan sin fractura)— y el objetivo es **integrarlas** en un modelo que estime el riesgo, respete la censura y dé efectos interpretables, todo **dentro del marco GLM**. La sección lo construye paso a paso: qué es la variable de interés y qué supuestos hacemos, qué enlace usamos y por qué, cómo se ajusta e interpreta sobre la cohorte, qué aporta como modelo de supervivencia y qué conclusiones clínicas se derivan.

De los datos al modelo: qué es T y qué asumimos

En la práctica, a partir de los datos observados hay que construir un modelo razonable. ¿Qué tenemos, exactamente, de cada paciente de la cohorte?

- un **indicador de fractura** (ocurrió / no ocurrió), **ever**,
- el **periodo de revisión** en que se observó el desenlace, **tiempo**, y
- una marca de **censura** (si el seguimiento terminó sin fractura), **evento**.

La variable de interés es un tiempo discreto.

Llamemos T al **tiempo hasta la fractura**. Como no vemos el instante exacto, sino solo la revisión en que se detecta, T es una variable aleatoria discreta que toma valores en los periodos $\{1, 2, \dots, 6\}$. De cada paciente observamos su último periodo y si terminó en **evento** ($\delta = 1$) o en **censura** ($\delta = 0$, y entonces solo sabemos que T es mayor que ese periodo).

No modelamos T directamente, sino su hazard.

La distribución de T con censura es incómoda de manejar; en cambio, el **hazard discreto** —la probabilidad de fracturar en la revisión t entre quienes llegan sin fractura a ella ([Sección 5.1](#))— se deja estimar de forma natural:

$$h_t = \Pr(T = t \mid T \geq t)$$

Esto es así porque descompone la historia de cada paciente en una **secuencia de preguntas binarias**: en cada revisión a la que llega, ¿fractura, sí o no? Y una secuencia de respuestas ‘sí/no’ es justo lo que un GLM binomial sabe modelar.

Esa descomposición nos pide **reorganizar los datos**. En la **cohorte** tenemos una fila por mujer —su **tiempo** y su **evento**—; el hazard por periodo, en cambio, vive en el **formato persona-periodo** que anticipamos en la [Sección 5.1: una fila por paciente y revisión en riesgo](#), con una respuesta binaria $y = 1$ solo en la revisión donde se detecta la fractura. La **censura encaja sin esfuerzo**: una paciente censurada aporta las filas de los periodos que sí se observaron (en todas, $y = 0$) y deja de aportar después —no hay que inventarle un valor de T —.

La transformación la hace `expandir_persona_periodo()` (definida en la Presentación del caso): recorre cada mujer y genera sus filas, periodo a periodo, hasta el último que se observó.

```
pp <- expandir_persona_periodo(cohorte) # de 1 fila por mujer a 1 fila por mujer y periodo
head(pp, 8)
```

	id	centro	x1	x2	periodo	y
1	1	1	-1.6736304	0	1	0
2	1	1	-1.6736304	0	2	0

3	1	1	-1.6736304	0	3	0
4	1	1	-1.6736304	0	4	1
5	2	1	1.0929380	1	1	0
6	2	1	1.0929380	1	2	0
7	2	1	1.0929380	1	3	1
8	3	1	-0.3526607	1	1	0

Compárese con la **cohorte** de antes: una mujer que fractura en su tercera revisión pasa a ser **tres filas** ($y = 0, 0, 1$); una censurada en el cuarto periodo sin fractura, **cuatro filas** todas con $y = 0$. Cada fila es el **ensayo de Bernoulli** de un periodo, y sobre este banco ajustamos el GLM.

Qué asumimos:

- **Independencia condicional.** Las distintas filas de una misma paciente cuentan como ensayos independientes dadas las covariables y el periodo —no porque sean sujetos distintos, sino por la factorización de la verosimilitud de supervivencia (el «punto fino» de la [Sección 5.1](#))—. Por eso basta un GLM ordinario, sin efecto aleatorio de paciente.
- **Censura no informativa.** El motivo de abandonar el seguimiento **no** está ligado al riesgo de fracturar; bajo ese supuesto, la censura no introduce sesgo.
- **Riesgos proporcionales.** Asumimos que las covariables **multiplican** el hazard por un factor **constante en el tiempo** (el supuesto de la [Sección 5.2](#)). Esto permite resumir cada efecto en un único hazard ratio y, como veremos, **fija el enlace**. Si no se cumpliera, se relajaría con interacciones covariable $\times$ periodo.
- *Forma del riesgo base libre.* No imponemos ninguna fórmula a cómo cambia el riesgo con las revisiones: dejamos **un parámetro por periodo**, α_t (uno por revisión). Es lo más flexible posible —el análogo discreto de “no especificar $h_0(t)$ ” que hacía Cox—.

Con esas piezas, el modelo casi se escribe solo: un **GLM binomial** para el hazard de cada periodo, con predictor lineal

$$\eta_t(\mathbf{x}) = \alpha_t + \mathbf{x}'\beta$$

—un intercepto por revisión (α_t , el riesgo base) más los efectos de las covariables (β)—. Queda **una sola decisión**: el **enlace** que conecta el hazard (una probabilidad en $[0, 1]$) con ese predictor. Y no es libre: el supuesto de **riesgos proporcionales** lo determina —y ya lo dedujimos en la [Sección 5.2](#)—.

El enlace complementary log-log y por qué es el natural

El hazard es una **probabilidad**; el predictor lineal $\alpha_t + \mathbf{x}'\beta$ vive en toda la recta real. Ya sabemos, por la [Sección 5.2](#), qué **enlace** los conecta bajo riesgos proporcionales: el **complementary log-log**,

$$\text{cloglog}(p) = \log(-\log(1-p)) = \eta, \quad p = 1 - \exp(-\exp(\eta)).$$

A diferencia del logit y el probit, es **asimétrico** (se aproxima a 1 más despacio que a 0).

```
tibble(eta = seq(-4, 4, by = 0.02)) |>
  mutate(logit = plogis(eta),
         cloglog = 1 - exp(-exp(eta))) |>
  pivot_longer(c(logit, cloglog), names_to = "enlace", values_to = "p") |>
  ggplot(aes(eta, p, color = enlace)) +
  geom_line(linewidth = 1) +
  labs(x = expression(eta), y = expression(p == g^{-1}(eta)), color = "Enlace")
```

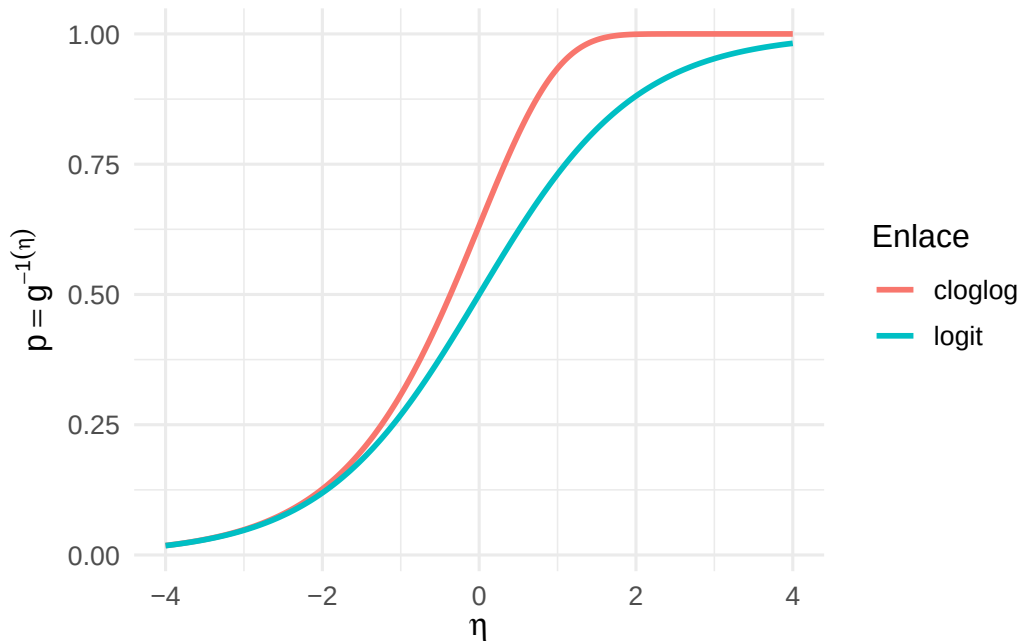


Figura 35: De los enlaces comparados en la Unidad 2, aquí retenemos solo logit y cloglog —los relevantes en supervivencia discreta—. La asimetría del cloglog (se aproxima a 1 más despacio que a 0) es lo que lo hace el enlace natural de los riesgos proporcionales.

Recordemos por qué este enlace y no el logit: como dedujimos en la [Sección 5.2](#), escribir la hipótesis de riesgos proporcionales sobre el hazard por periodo da **exactamente** $\text{cloglog}(h_t) = \alpha_t + \mathbf{x}'\beta$, con un intercepto α_t que absorbe el riesgo base (Allison, 1982; Prentice & Gloeckler, 1978). Con **logit** se obtiene la aproximación “logística discreta”, razonable cuando el riesgo por periodo es pequeño, pero solo el **cloglog** **preserva exactamente** los riesgos proporcionales subyacentes —y por eso reproduce los **mismos hazard ratios que Cox**—.

Ajuste en tiempo discreto e interpretación

Con los datos ya en formato **persona-periodo** (`pp`), ajustar el hazard es, literalmente, un **GLM binomial con enlace cloglog**. Construido el modelo, ahora aprovechamos que la cohorte es **simulada** para hacer algo que con datos reales no podríamos: **contrastar las estimaciones con la verdad** del proceso generador —la validación que anticipábamos—.

```
m_pp <- glm(y ~ periodo + x1 + x2, family = binomial("cloglog"), data = pp)
summary(m_pp)
```

Call:

```
glm(formula = y ~ periodo + x1 + x2, family = binomial("cloglog"),
    data = pp)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-2.97695	0.15603	-19.079	< 2e-16 ***
periodo2	0.38666	0.18723	2.065	0.038909 *
periodo3	0.49718	0.18982	2.619	0.008814 **
periodo4	0.44472	0.21705	2.049	0.040472 *
periodo5	0.58974	0.24694	2.388	0.016931 *
periodo6	0.77511	0.30201	2.567	0.010272 *
x1	0.84153	0.06866	12.256	< 2e-16 ***
x2	-0.45364	0.13269	-3.419	0.000629 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 1810.7 on 3427 degrees of freedom
Residual deviance: 1640.8 on 3420 degrees of freedom
AIC: 1656.8

Number of Fisher Scoring iterations: 6

```
exp(coef(m_pp)[c("x1", "x2")]) # hazard ratios estimados
```

```
      x1      x2
2.3199063 0.6353109
```

```
exp(attr(cohorte, "verdad")$binaria$beta)    # HR verdaderos:  $e^{0.85}$ ,  $e^{-0.65}$ 
```

```
      x1      x2
2.3396469 0.5220458
```

Los coeficientes de **periodo** son los **interceptos por periodo** α_t , y aquí **crecen** (0.39, 0.50, ..., 0.78 sobre el periodo 1): el modelo recupera el **riesgo base creciente** del DGP. Los de x1 y x2, exponenciados, son **hazard ratios**: x1 da $\widehat{HR} \approx 2.32$, que **clava** la verdad ($e^{0.85} \approx 2.34$), y x2 da ≈ 0.64 , algo **atenuado** frente a su verdad ($e^{-0.65} \approx 0.52$) —el efecto protector se queda corto, en parte por omitir la fragilidad—. La **censura** no exige nada extra: un sujeto censurado simplemente **deja de aportar filas** tras su último periodo en riesgo, sin un $y = 1$; aporta lo que se observó (que no falló mientras estuvo) y nada más.

El riesgo base por periodo y la supervivencia que implica:

```
base <- tibble(periodo = factor(levels(pp$periodo), levels = levels(pp$periodo)),
               x1 = 0, x2 = 0)
base$h0 <- predict(m_pp, newdata = base, type = "response") # hazard base por periodo
base$t   <- seq_len(nrow(base))
base$S   <- cumprod(1 - base$h0)                          # supervivencia base

pH <- ggplot(base, aes(t, h0)) +
  geom_col(fill = "grey70") +
  labs(x = "Periodo", y = expression(hazard~base~h[0][t]), title = "Riesgo base por periodo")
pS <- ggplot(base, aes(t, S)) +
  geom_line(linewidth = 1) + geom_point(size = 2) +
  scale_y_continuous(limits = c(0, 1)) +
  labs(x = "Periodo", y = expression(supervivencia~S[t]), title = "Supervivencia base")
pH + pS
```

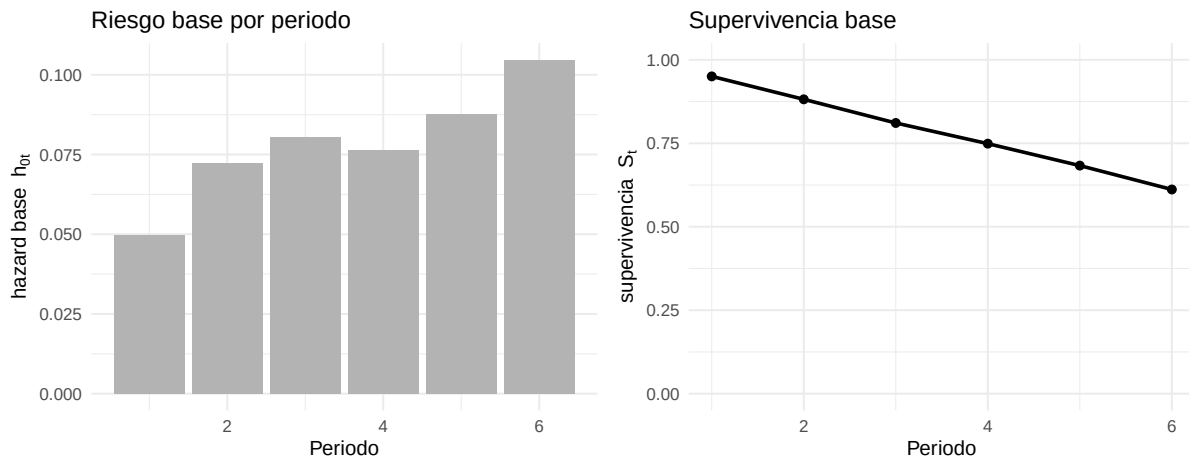


Figura 36: Riesgo base por periodo (h_{0t} , evaluado en $x_1 = x_2 = 0$) y la supervivencia base $S_t = \prod(1 - h_{0k})$ que implica. En este DGP el riesgo base es creciente, de modo que la supervivencia cae cada vez más deprisa.

Fragilidad: el centro como efecto aleatorio.

Las pacientes se agrupan en clínicas; añadir un **intercepto aleatorio por centro** —la *fragilidad*, en el lenguaje de la supervivencia— es, otra vez, un **GLMM cloglog**, y reactiva la [Unidad 4](#) sobre el reloj de supervivencia. Y, como pedíamos, se ajusta **sin bayesiana**, con `lme4`:

```
library(lme4)
m_frail <- glmer(y ~ periodo + x1 + x2 + (1 | centro),
                family = binomial("cloglog"), data = pp)
VarCorr(m_frail) # sigma de la fragilidad (verdad del DGP: 0.70)
```

Groups	Name	Std.Dev.
centro	(Intercept)	0.367

```
fixef(m_frail)[c("x1", "x2")] # efectos fijos, ahora condicionales al centro
```

	x1	x2
	0.8879172	-0.4681168

`VarCorr` estima la **varianza de la fragilidad**: $\hat{\sigma}_u \approx 0.37$, **por debajo** de la verdad (0.70) —la misma subestimación de las varianzas que con datos binarios y pocos grupos (24 centros)—; mide cuánto varía el riesgo base de unas clínicas a otras. Al **condicionar al centro**, los

efectos crecen algo en magnitud ($x_1 : 0.84 \rightarrow 0.89$; $x_2 : -0.45 \rightarrow -0.47$): es el salto marginal→condicional de 1.4, ahora sobre el hazard, aunque leve porque $\hat{\sigma}_u$ es pequeño.



En R. El GLM de supervivencia en persona-periodo

```
pp <- expandir_persona_periodo(cohorte) # 1 fila por p
m <- glm(y ~ periodo + x1 + x2, family = binomial("cloglog"), data = pp) # hazard discr
```

- **Formato persona-periodo** — $y = 1$ solo en el periodo del evento; la **censura se trata sola** (la paciente aporta filas $y = 0$ hasta su último periodo observado, sin fila de evento).
- **El factor periodo** estima el **hazard base** α_t (un intercepto por revisión); las covariables entran como en cualquier GLM y su $e^{\hat{\beta}}$ es un **hazard ratio** con **cloglog** (un **odds ratio** si se usa **binomial("logit")**, la logística de tiempo discreto).
- **Del hazard a la supervivencia** — `predict(m, newdata, type = "response")` da $\hat{h}_t$ por periodo; la supervivencia se acumula con `cumprod(1 - h)` (y el riesgo acumulado es $1 - S$).
- **Fragilidad** — un intercepto aleatorio por grupo, `+(1 | centro)`, lo convierte en un **GLMM cloglog**: `lme4::glmer(..., family = binomial("cloglog"))`, con `VarCorr(m)` para $\hat{\sigma}_u$ y `fixef(m)` para los efectos condicionales.

Cloglog frente a logit: ¿cuánto importa el enlace?

Elegimos el **cloglog** por una razón estructural: preserva los riesgos proporcionales. Pero los datos persona-periodo son binarios, así que nada impide ajustarlos con el enlace por defecto, el **logit** —la *regresión logística de tiempo discreto*—. La diferencia es de **interpretación**: con **cloglog** los $\exp(\beta)$ son **hazard ratios** (riesgos proporcionales); con **logit** son **odds ratios** (un supuesto de *odds proporcionales*). Ajustemos los dos y comparémoslos:

```
m_pp_logit <- glm(y ~ periodo + x1 + x2, family = binomial("logit"), data = pp)

# cloglog -> hazard ratios ; logit -> odds ratios
cbind(cloglog_HR = exp(coef(m_pp)[c("x1", "x2")]),
      logit_OR    = exp(coef(m_pp_logit)[c("x1", "x2")]))
```

```
      cloglog_HR  logit_OR
x1  2.3199063  2.4420551
x2  0.6353109  0.6116684
```

```
c(AIC_cloglog = AIC(m_pp), AIC_logit = AIC(m_pp_logit))
```

AIC_cloglog	AIC_logit
1656.826	1657.275

Como vemos en la tabla, la diferencia en los coeficientes estimados para ambos modelos es mínima: para la fragilidad, HR = 2.32 (cloglog) frente a OR = 2.44 (logit); para el tratamiento, 0.64 frente a 0.61. El patrón es el esperado —el odds ratio se aleja un poco más de 1 que el hazard ratio (algo más alto cuando el efecto es de riesgo, algo más bajo cuando es protector), porque con hazards por periodo que son moderados, odds y riesgo se parecen pero no son idénticos—. Y los AIC casi empatan (1656.8 con cloglog frente a 1657.3 con logit), con una ventaja mínima para el cloglog. En estos datos, el enlace apenas mueve los resultados.

```
base2 <- tibble(periodo = factor(levels(pp$periodo), levels(pp$periodo)), x1 = 0, x2 = 0)
pred2 <- base2 |>
  mutate(cloglog = predict(m_pp,      newdata = base2, type = "response"),
         logit    = predict(m_pp_logit, newdata = base2, type = "response")) |>
  pivot_longer(c(cloglog, logit), names_to = "enlace", values_to = "h") |>
  mutate(t = as.integer(periodo)) |>
  arrange(enlace, t) |>
  group_by(enlace) |>
  mutate(S = cumprod(1 - h)) |>
  ungroup()

ggplot(pred2, aes(t, S, color = enlace)) +
  geom_line(linewidth = 1) + geom_point(size = 2) +
  scale_y_continuous(limits = c(0, 1)) +
  labs(x = "Periodo", y = "Supervivencia base S(t)", color = "Enlace")
```

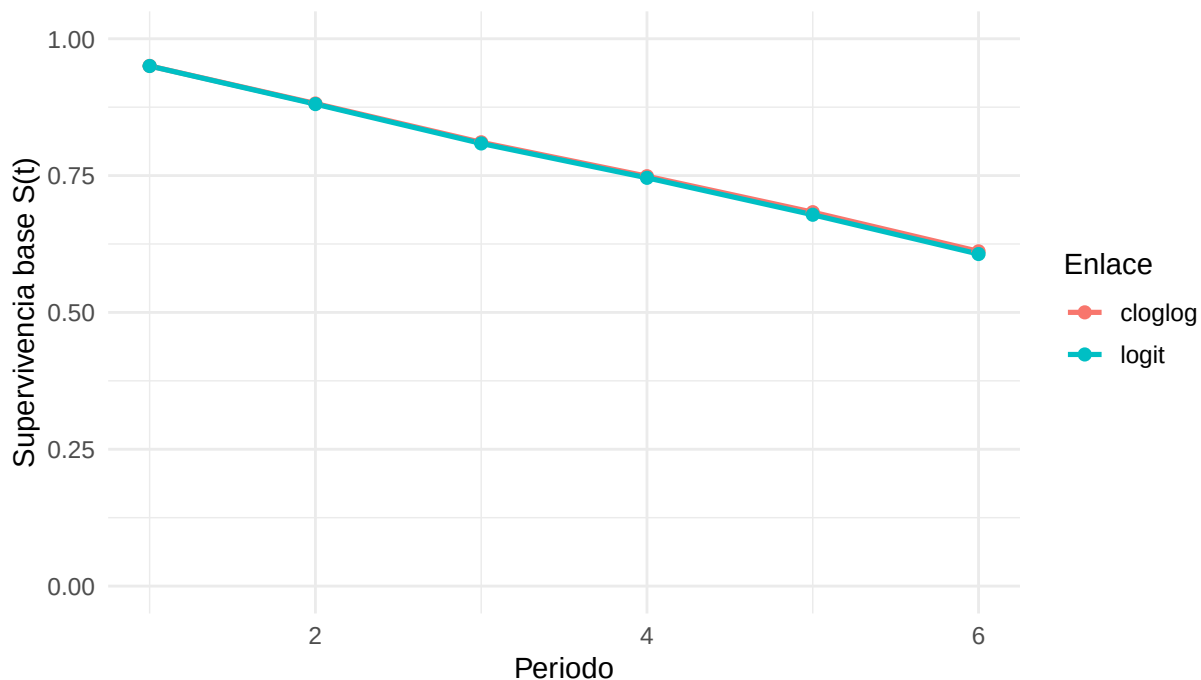


Figura 37: Supervivencia base ($x_1 = x_2 = 0$) reconstruida desde el GLM con enlace cloglog y con logit. Las dos curvas son casi indistinguibles: con hazards por periodo moderados ambos enlaces dan prácticamente lo mismo. La preferencia por el cloglog es estructural (preserva los riesgos proporcionales y reproduce a Cox), no numérica.

El gráfico lo confirma: la supervivencia base reconstruida con uno y otro enlace es casi la misma curva. La conclusión es doble. Numéricamente, aquí da casi igual; pero conceptualmente la elección no es indiferente: nos quedamos con el **cloglog** por lo que significa —es el único que preserva los riesgos proporcionales y, por tanto, el que reproduce a Cox y permite leer los efectos como hazard ratios—. El **logit** sería la opción si quisiéramos razonar en odds.

Qué aporta el GLM en supervivencia

Merece la pena recoger lo que hemos ganado al plantear la supervivencia como un GLM binomial con enlace **cloglog**, sin salir del marco que ya manejábamos:

- Modela el riesgo base: los interceptos α_t son el hazard de cada periodo, así que el modelo no solo estima los efectos, sino que predice el hazard y la supervivencia periodo a periodo (lo que hicimos en los gráficos anteriores).
- Trata la censura sin nada extra: una paciente censurada aporta sus filas observadas y deja de aportar después; la verosimilitud es la correcta por construcción.

- Maneja el tiempo discreto de forma exacta: cuando muchas mujeres fracturan en el mismo periodo, el formato persona-periodo lo recoge tal cual, sin aproximaciones.
- Se extiende a la fragilidad: añadir `(1 | centro)` es un GLMM `cloglog` que se ajusta con `glmer`, reactivando los efectos mixtos de la [Unidad 4](#) —como vimos arriba—.

Todo ello leyendo los efectos como *hazard ratios*, la escala del estándar clínico. Ese estándar es el modelo de Cox, la versión en tiempo continuo del mismo supuesto de riesgos proporcionales, que desarrollaremos —y compararemos formalmente con este GLM— en el Caso 3. Aquí la validación la da el propio simulador: los *hazard ratios* estimados recuperan los valores verdaderos del DGP ($\approx 2,3$ y $0,64$ frente a $e^{0,85}$ y $e^{-0,65}$), que es cuanto necesitamos para confiar en el modelo.

Conclusiones para la práctica clínica

Traducidas a quien trata pacientes, las estimaciones del modelo dicen:

- **La fragilidad ósea manda.** Cada desviación típica de baja densidad mineral **multiplifica por $\sim 2,3$** el riesgo de fractura en cada periodo: es el factor dominante. Identificar y vigilar de cerca a las pacientes de alta fragilidad es la palanca de mayor impacto.
- **El tratamiento ayuda, pero no basta por sí solo.** Reduce el riesgo alrededor de un tercio (HR $0,64$), un efecto real y estable en el tiempo, pero parcial: en pacientes muy frágiles el riesgo sigue siendo alto pese al tratamiento. Conviene combinarlo con el manejo de la fragilidad.
- **El riesgo no se relaja con el seguimiento.** El hazard base es creciente: la probabilidad de fractura por periodo aumenta con el tiempo, así que la vigilancia no debe disminuir en las revisiones tardías.
- **Importa el centro.** La *fragilidad* entre clínicas indica que el riesgo basal varía de un centro a otro: parte del riesgo individual depende del contexto asistencial, no solo de la paciente.
- **Un solo número resume el efecto.** Al cumplirse los riesgos proporcionales, el hazard ratio es estable en los seis periodos: una única cifra ($\times 2,3$ por fragilidad; $\times 0,64$ por tratamiento) describe el efecto durante todo el seguimiento, lo que facilita comunicárselo a la paciente.

El siguiente gráfico hace tangible la decisión clínica: el **riesgo acumulado de fractura** ($1 - S_t$) a lo largo de los periodos para tres perfiles de paciente.

```
arquetipos <- tibble(
  arquetipo = factor(c("Alta fragilidad, sin tratamiento",
    "Alta fragilidad, con tratamiento",
    "Fragilidad media, con tratamiento"),
    levels = c("Alta fragilidad, sin tratamiento",
```

```

                                "Alta fragilidad, con tratamiento",
                                "Fragilidad media, con tratamiento")),
  x1 = c(1, 1, 0), x2 = c(0, 1, 1))

grid <- tidyr::expand_grid(periodo = factor(levels(pp$periodo), levels(pp$periodo)),
                          arquetipo = arquetipos$arquetipo) |>
  left_join(arquetipos, by = "arquetipo")
grid$h <- predict(m_pp, newdata = grid, type = "response")
grid <- grid |>
  mutate(t = as.integer(periodo)) |>
  arrange(arquetipo, t) |>
  group_by(arquetipo) |>
  mutate(S = cumprod(1 - h), riesgo_acum = 1 - S) |>
  ungroup()

ggplot(grid, aes(t, riesgo_acum, color = arquetipo)) +
  geom_line(linewidth = 1) + geom_point(size = 2) +
  scale_y_continuous(labels = scales::percent) +
  labs(x = "Periodo de revisión", y = "Riesgo acumulado de fractura (1 - S)",
       color = NULL) +
  theme(legend.position = "top")

```

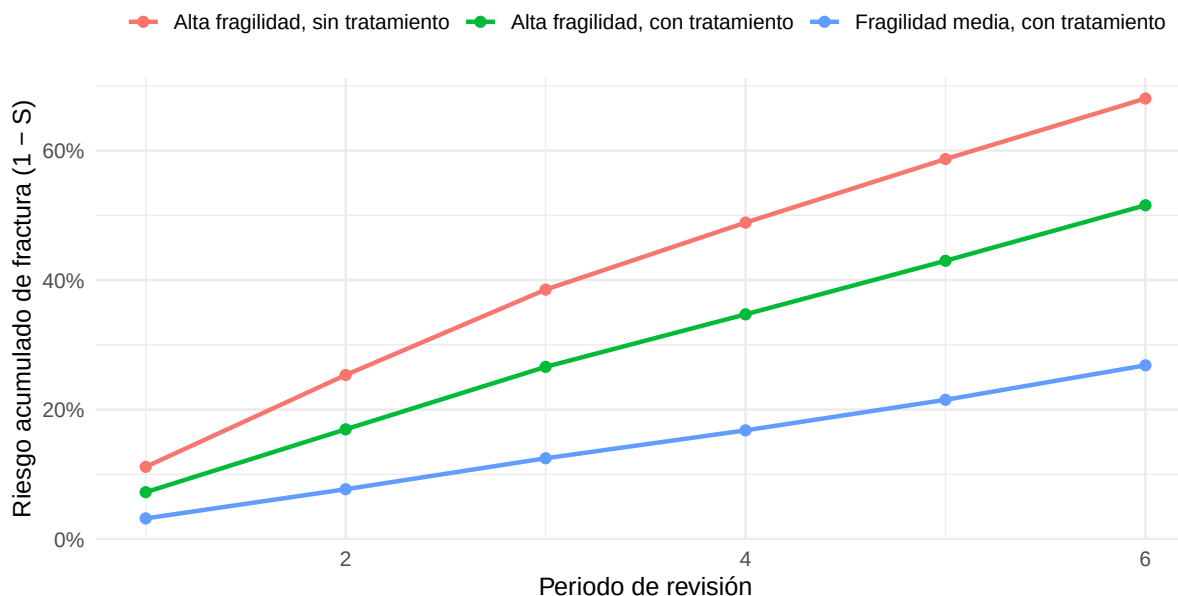


Figura 38: Riesgo acumulado de fractura ($1 - S_t$) predicho por el GLM cloglog para tres perfiles. La distancia vertical entre ‘alta fragilidad sin tratar’ y ‘con tratamiento’ es la reducción de riesgo absoluta atribuible al tratamiento; crece con el seguimiento. Los valores exactos se leen sobre el eje al renderizar.

Los números ponen cifras a la decisión: tras los seis periodos, una paciente de alta fragilidad sin tratar acumula un riesgo de fractura de 68 %; con tratamiento baja a 52 % (unos 16 puntos menos); y si además su fragilidad fuera media, el riesgo caería a 27 %. La fragilidad pesa más que el fármaco —reducirla, cuando se puede, mueve más la aguja que el tratamiento por sí solo—.

Estas conclusiones son didácticas —descansan en una cohorte simulada—, pero el flujo es el real: de los hazard ratios y la curva de supervivencia a una recomendación accionable. Con datos reales, el mismo modelo daría las cifras sobre las que decidir.

! Tarea del estudiante (1.5) — resuelve la Pregunta 2 del caso

Responde la **Pregunta 2** del cliente ([Unidad 6](#)) sobre la cohorte de tu equipo: *¿cuándo se fracturan, cada cuánto conviene revisar, y retrasa el tratamiento la fractura?*

Tareas:

1. **Prepara y ajusta (RA7)**. Reconoce que es un problema de **tiempo-a-evento con censura**, expande a **persona-periodo (pp)** y ajusta el GLM con enlace **cloglog**; reporta los **hazard ratios** de x_1 y x_2 (con su IC).
2. **Cuándo, y cada cuánto revisar**. Representa el **hazard base por periodo** ($\hat{h}_{0t}$) y la **supervivencia** S_t ; a partir de su forma (¿riesgo creciente?, ¿se acelera

la caída?), propón al cliente un **calendario de revisiones**.

3. **Censura.** Explica por qué, siendo **no informativa**, no sesga la estimación: qué aporta —y qué no— una paciente censurada.
4. **Fragilidad de centro.** Añade (`1 | centro`) y comenta qué cambia (o no) en los efectos fijos y sus errores estándar.
5. **Predicción y respuesta.** Traza la **supervivencia** (o el riesgo acumulado $1 - S_t$) para un par de perfiles (fragilidad alta/media, con y sin tratamiento), di si el tratamiento **retrasa** la fractura y responde al cliente.

Control de calidad: contrasta tus estimaciones con la **verdad del DGP** (`attr(cohorte, "verdad")`) —ajusta a ciegas y valida al final—.

6 · Estudio de caso, exposición y evaluación

i Sobre esta unidad

Cierre del Caso 1: trabajo acompañado del estudio de caso y **exposiciones** con Q&A individual. No hay teoría nueva; se integra y se evalúa todo lo anterior. Esta unidad reúne la **maquinaria de evaluación** (actividades, instrucciones, rúbricas). **RA implicados:** RA8 (desarrollar, presentar y defender estudios de caso reproducibles) y, de forma acumulativa, RA1–RA7.

6.1 El estudio de caso (entregable)

El encargo

Te contrata la dirección médica de **OsteoRed**, una red de 24 clínicas de salud ósea que, dentro de un programa de prevención, ha seguido durante seis años a una cohorte de mujeres. No son estadísticos: te traen sus datos y sus preguntas, y esperan **respuestas y recomendaciones**, no una clase de modelos.

De cada mujer registraron su **fragilidad ósea** (un marcador de densidad mineral, estandarizado; `x1`), si seguía el **tratamiento preventivo** (`x2`), la **clínica** a la que acude (`centro`) y, a lo largo de hasta **seis revisiones anuales**, si sufrió una **fractura** y **cuándo** —o si terminó el seguimiento sin fracturarse— (`ever`, `tiempo`, `evento`); y, cuando la hubo, su **tipo** (`clase_nom`: A/B/C) y su **gravedad** (`sever_ord`: leve/moderada/grave).

Lo que la dirección de OsteoRed quiere saber

1. «¿Podemos anticipar qué mujeres acabarán fracturándose? ¿Influyen de verdad la fragilidad ósea y nuestro tratamiento, y cuánto? ¿Podríamos estimar el riesgo de una paciente nueva? Y el tratamiento, ¿protege por igual a todas o depende de lo frágil que esté cada una?»
2. «No nos basta con saber *si* se fracturan: necesitamos saber **cuándo**. ¿En qué momento del seguimiento se dispara el riesgo? ¿Cada cuánto deberíamos citarlas a revisión? ¿El tratamiento **retrasa** la fractura? Y cuidado: muchas terminan el seguimiento sin fracturarse, no queremos que eso distorsione las cuentas.»
3. «Cuando hay fractura, la clasificamos en tres **tipos** (A, B, C). ¿Los mismos factores empujan hacia unos tipos más que otros? ¿Podríamos anticipar el tipo más probable?»
4. «Cada fractura tiene una **gravedad** (leve, moderada, grave). ¿Qué la agrava? ¿Nuestro tratamiento, además de evitar fracturas, hace que las que ocurren sean **menos graves**?»
5. «Trabajamos en **24 clínicas** y sospechamos que no todas rinden igual. ¿El riesgo depende de la clínica o es todo cosa de la paciente? ¿Hay centros que lo hacen claramente mejor o peor? Si abrimos uno nuevo, ¿qué deberíamos esperar?»

Tu trabajo

Entregas un **informe de consultoría reproducible** que convierta esas preguntas en análisis. Nadie te dice qué modelo usar: **elegirlo y defenderlo es parte de la nota**. Para **cada** pregunta del cliente:

1. **Identifica el objetivo** y qué tipo de respuesta tienes entre manos.
2. **Propón un análisis adecuado y justifícalo**: por qué ese modelo y no otro, qué enlace, si procede una interacción, si hay que reconocer el agrupamiento por clínica o el tiempo con censura.
3. **Resuélvelo** con rigor, con las decisiones de selección y el diagnóstico que lo respalden.
4. **Interprétalo** en la escala que el cliente entienda: probabilidades, veces más riesgo, plazos...
5. **Responde al cliente** con una recomendación accionable.

De forma transversal, el informe incluye **predicción** donde el cliente la pide (el riesgo de una paciente nueva, el calendario de revisiones, qué esperar de una clínica nueva), el **control de calidad** de cada análisis y la **reproducibilidad**: un .qmd que corre de principio a fin (RA8).

El formato: un informe técnico

OsteoRed pregunta en términos **ejecutivos** (qué hacer, a quién vigilar, cada cuánto), pero tu entregable es un **informe técnico**, no un resumen ejecutivo. La diferencia está en el lector: un resumen ejecutivo va directo a conclusiones y recomendaciones, con lo técnico al mínimo; un informe técnico se escribe para alguien que podría **auditar o reproducir** tu trabajo, así que documenta el razonamiento completo —objetivo, método elegido y su justificación, supuestos, ajuste, diagnóstico e interpretación— antes de llegar a la recomendación.

En la práctica: traduce cada pregunta ejecutiva del cliente a uno o varios **objetivos analíticos precisos**; organiza el informe siguiendo esas preguntas y, dentro de cada una, recorre el ciclo (objetivo → método justificado → resultados → diagnóstico → interpretación → respuesta y recomendación); y **devuelve cada conclusión al lenguaje del cliente**. El rigor técnico y la lectura accionable conviven: lo técnico sostiene la recomendación, no la sustituye.

! Apartado de Reflexión sobre el uso de la IA (obligatorio)

Cierra el informe con un apartado de **Reflexión ética sobre la IA** que recoja una reflexión sobre el uso que se ha hecho de la IA para la realización del trabajo: qué herramientas usasteis y para qué, los **prompts relevantes**, y —sobre todo— cómo **verificasteis y validasteis** lo que produjeron (qué comprobasteis, qué corregisteis, qué descartasteis).

Es la evidencia de vuestro proceso y del uso responsable de la IA.

! Anexo de reproducibilidad (obligatorio)

Añade al final un **anexo de reproducibilidad** que permita a otra persona regenerar el informe: el **entorno** (`sessionInfo()` o `renv`, con las versiones de los paquetes), la **semilla** usada, las **instrucciones para ejecutar** el `.qmd` de principio a fin y la **procedencia y estructura de los datos** (cohorte y su expansión persona-periodo).

! Importante

Además, el líder del equipo ha de completar y presentar, en un documento adicional: - La **planificación y reparto** de tareas asumidas en el equipo (roles, tareas, cómo os organizasteis en el tiempo, qué revisiones o hitos hubo y cómo resolvisteis los bloqueos. - La evaluación de los compañeros, anotando el grado de implicación (del 0 al 100%) en el trabajo. - El checklist de tareas realizadas (según la propuesta especificada abajo) cumplimentado.

💡 Control de calidad interno: ajusta a ciegas y valida contra el DGP

Esto es tuyo, no del cliente. Como la cohorte es simulada, los parámetros que la generaron están en `attr(cohorte, "verdad")`: los efectos β de `x1` y `x2`, la varianza entre centros σ_u , el riesgo base por periodo α_t y los parámetros del nominal y del ordinal. La disciplina que se te pide: **ajusta y decide sin mirar esa “verdad”** y, solo al final, compárala con tus estimaciones, mostrando el contraste en el informe.

Ten presente que esta comprobación es un lujo **didáctico**, posible solo porque los datos son simulados. Con datos reales no existe tal “verdad”: allí la confianza en el modelo descansa en el diagnóstico, la validación fuera de muestra y el conocimiento del problema. Recógelos así en tu Reflexión.

💡 Cada equipo, una base de datos distinta

Cada equipo trabaja sobre **su propia cohorte**, generada con una configuración diferente —pero coherente— de los parámetros del DGP. Las diez configuraciones y los datos por equipo los produce el script `R/generar_datos Equipos.R`, que deja en `datos Equipos/equipo_XX/` la cohorte (`cohorte.rds / .csv`) y su expansión persona-periodo (`pp.rds / .csv`). El enunciado y las preguntas del cliente son los mismos para todos; lo que cambia de un equipo a otro son las **conclusiones numéricas** (efectos, riesgos, plazos). La clave con los parámetros verdaderos (`configuraciones Equipos.csv`) la guarda el/la docente.

6.2 Checklist de tareas

Antes de entregar, comprueba dos cosas: que has **respondido cada pregunta del cliente** y que no te has saltado ninguna de las **tareas que un buen analista** hace en un proyecto. Verifica cada casilla y marca las que hayas comprobado e incluido en la entrega.

¿Respondida cada pregunta del cliente?

- ☐ **P1 · Riesgo de fractura:** quién se fractura, papel de la fragilidad y del tratamiento, riesgo de una paciente nueva, y si el efecto del tratamiento depende de la fragilidad.
- ☐ **P2 · Cuándo ocurre:** evolución del riesgo a lo largo de las revisiones, calendario de seguimiento y efecto del tratamiento sobre los plazos, con la censura bien tratada.
- ☐ **P3 · Tipo de fractura:** qué factores empujan hacia cada tipo y anticipación del más probable.
- ☐ **P4 · Gravedad:** qué agrava la fractura y si el tratamiento la reduce.
- ☐ **P5 · Peso de la clínica:** cuánto del riesgo depende del centro, centros mejores o peores y qué esperar de uno nuevo.

Tareas analíticas del proyecto

Comprensión y datos

- ☐ Traducir cada pregunta del cliente a objetivos analíticos precisos.
- ☐ Explorar y depurar los datos (EDA): tipos de variable, escalas, atípicos y ausentes.

Modelización

- ☐ Identificar el tipo de respuesta y elegir un modelo adecuado a cada objetivo, justificándolo.
- ☐ Especificar bien: enlace, interacciones pertinentes, agrupamiento (efectos aleatorios), tiempo con censura, proporcionalidad.
- ☐ Comprobar la modificación de un efecto por otra variable cuando el cliente lo pregunta (interacción).
- ☐ Ajustar y comprobar la convergencia y la estabilidad del ajuste.

Evaluación

- ☐ Diagnosticar: residuos, calibración, separación, bondad de ajuste (χ^2), sobredispersión y los supuestos propios del modelo (proporcionalidad de odds en el ordinal, riesgos proporcionales en supervivencia, convergencia y no-singularidad en los mixtos).
- ☐ Evaluar la capacidad predictiva con la métrica adecuada: AUC y calibración (binaria), Brier multiclase (nominal), hazard/supervivencia por periodo.
- ☐ Seleccionar y comparar modelos con criterios (AIC/BIC) y contrastes (LRT), justificando la elección.

Resultados

- ☐ Interpretar los efectos en la escala propia de cada respuesta —odds ratios y efectos marginales (binaria), OR relativos (nominal), OR acumulado (ordinal), hazard ratios (supervivencia)—, en términos relativos y absolutos, con sus intervalos de confianza.
- ☐ Predecir donde el cliente lo pide (perfiles de paciente, paciente nueva, clínica nueva, calendario).
- ☐ Control de calidad: contrastar las estimaciones con la verdad del DGP.

Cierre

- ☐ Responder a cada pregunta con una recomendación accionable, en el lenguaje del cliente.
- ☐ Comunicar: informe técnico claro y ordenado, figuras legibles y referencias (APA 7).
- ☐ Reproducibilidad: el `.qmd` corre de principio a fin; semilla, entorno y datos documentados en el anexo.
- ☐ Incluir la reflexión sobre el uso de la IA.
- ☐ Adjuntar el documento de planificación y reparto de tareas, evaluación, valoración y checklist de tareas realizadas.

Reproducibilidad

```
sessionInfo()
```

```
R version 4.6.0 (2026-04-24)
Platform: x86_64-apple-darwin20
Running under: macOS Sequoia 15.7.7

Matrix products: default
BLAS:   /Library/Frameworks/R.framework/Versions/4.6-x86_64/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.6-x86_64/Resources/lib/libRlapack.dylib;

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: Europe/Madrid
tzcode source: internal

attached base packages:
[1] stats      graphics  grDevices  utils      datasets  methods   base

other attached packages:
[1] survminer_0.5.2      ggpubr_0.6.3          survival_3.8-6
[4] nnet_7.3-20           yardstick_1.4.0        ggeffects_2.3.2
[7] emmeans_2.0.3         pROC_1.19.0.1         car_3.1-5
[10] carData_3.0-6         marginaleffects_0.32.0 GGally_2.4.0
[13] MuMIn_1.48.19         lubridate_1.9.5        forcats_1.0.1
[16] stringr_1.6.0         dplyr_1.2.1           purrr_1.2.2
[19] readr_2.2.0           tidyr_1.3.2           tibble_3.3.1
[22] ggplot2_4.0.3         tidyverse_2.0.0        performance_0.17.0
[25] arm_1.15-3            lme4_2.0-1            Matrix_1.7-5
[28] MASS_7.3-65           DHARMA_0.5.0          see_0.14.0
[31] patchwork_1.3.2       aplore3_0.9           broom_1.0.13

loaded via a namespace (and not attached):
[1] RColorBrewer_1.1-3    rstudioapi_0.18.0     jsonlite_2.0.0
[4] shape_1.4.6.1         datawizard_1.3.1      magrittr_2.0.5
[7] TH.data_1.1-5         estimability_1.5.1    jomo_2.7-6
[10] farver_2.1.2          logistf_1.26.1        nloptr_2.2.1
[13] rmarkdown_2.31        vctrs_0.7.3           minqa_1.2.8
[16] rstatix_0.7.3         tidytext_0.5.9        htmltools_0.5.9
```

[19] haven_2.5.5	Formula_1.2-5	mitml_0.4-5
[22] plyr_1.8.9	sandwich_3.1-2	zoo_1.8-15
[25] mime_0.13	lifecycle_1.0.5	iterators_1.0.14
[28] pkgconfig_2.0.3	gap_1.15.2	R6_2.6.1
[31] fastmap_1.2.0	rbibutils_2.4.1	shiny_1.13.0
[34] digest_0.6.39	qgam_2.0.0	labeling_0.4.3
[37] timechange_0.4.0	abind_1.4-8	mgcv_1.9-4
[40] compiler_4.6.0	withr_3.0.2	doParallel_1.0.17
[43] S7_0.2.2	backports_1.5.1	ggstats_0.13.0
[46] ggsignif_0.6.4	pan_1.9	tools_4.6.0
[49] otl_0.2.0	httpuv_1.6.17	glue_1.8.1
[52] nlme_3.1-169	promises_1.5.0	grid_4.6.0
[55] checkmate_2.3.4	generics_0.1.4	operator.tools_1.6.3.1
[58] gtable_0.3.6	tzdb_0.5.0	formula.tools_1.7.1
[61] data.table_1.18.4	hms_1.1.4	utf8_1.2.6
[64] foreach_1.5.2	pillar_1.11.1	later_1.4.8
[67] splines_4.6.0	lattice_0.22-9	tidyselect_1.2.1
[70] knitr_1.51	reformulas_0.4.4	gridExtra_2.3
[73] stats4_4.6.0	xfun_0.58	stringi_1.8.7
[76] yaml_2.3.12	boot_1.3-32	evaluate_1.0.5
[79] codetools_0.2-20	cli_3.6.6	rpart_4.1.27
[82] xtable_1.8-8	Rdpack_2.6.6	Rcpp_1.1.1-1.1
[85] coda_0.19-4.1	parallel_4.6.0	gap.datasets_0.0.6
[88] glmnet_5.0	mvtnorm_1.4-0	scales_1.4.0
[91] insight_1.5.1	rlang_1.2.0	multcomp_1.4-31
[94] mice_3.19.0		

```
# renv::snapshot() # fijar el estado del entorno al cerrar el caso
```